[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 10

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 9 ...

- Exponential smoothing
  - Simple Exponential Smoothing
  - Holt's linear trend method (and damped trend method)
  - Holt-Winters method

Today

- ARIMA models

ARIMA models

- Besides Exponential Smoothing, ARIMA models are the most widely used approaches for forecasting.
- While Exponential Smoothing focuses on describing the trend and seasonality in data, ARIMA models try to describe the autocorrelation in the data.

Stationarity and differencing - recap

- A time series is stationary if its statistical properties (mean, variance, etc.) do not depend on the time t when it is observed.
- This means that time series with trend and season are not stationary, but white noise is.
- Interestingly, cyclic time series are stationary, since the lengths of the cycles are not equal.
- In general, a stationary time series has a plot which is approximately horizontal and has constant variance (except in case of cycles).
- Besides the time series plot, you can recognize non-stationary time series by their ACF plot as well: the ACF of non-stationary time series decreases slowly (and r_1 is often positive and large), while for stationary time series we quickly get to near zero autocorrelation.

- One way to transform non-stationary time series into stationary ones is to *difference* it, take the differences of consecutive observations.
- Differencing helps to stabilize the mean of the time series.
- In *R* we can use the *difference* function to difference a time series.
- Another transformation, taking the logarithm, could help in stabilizing the variance of the time series.

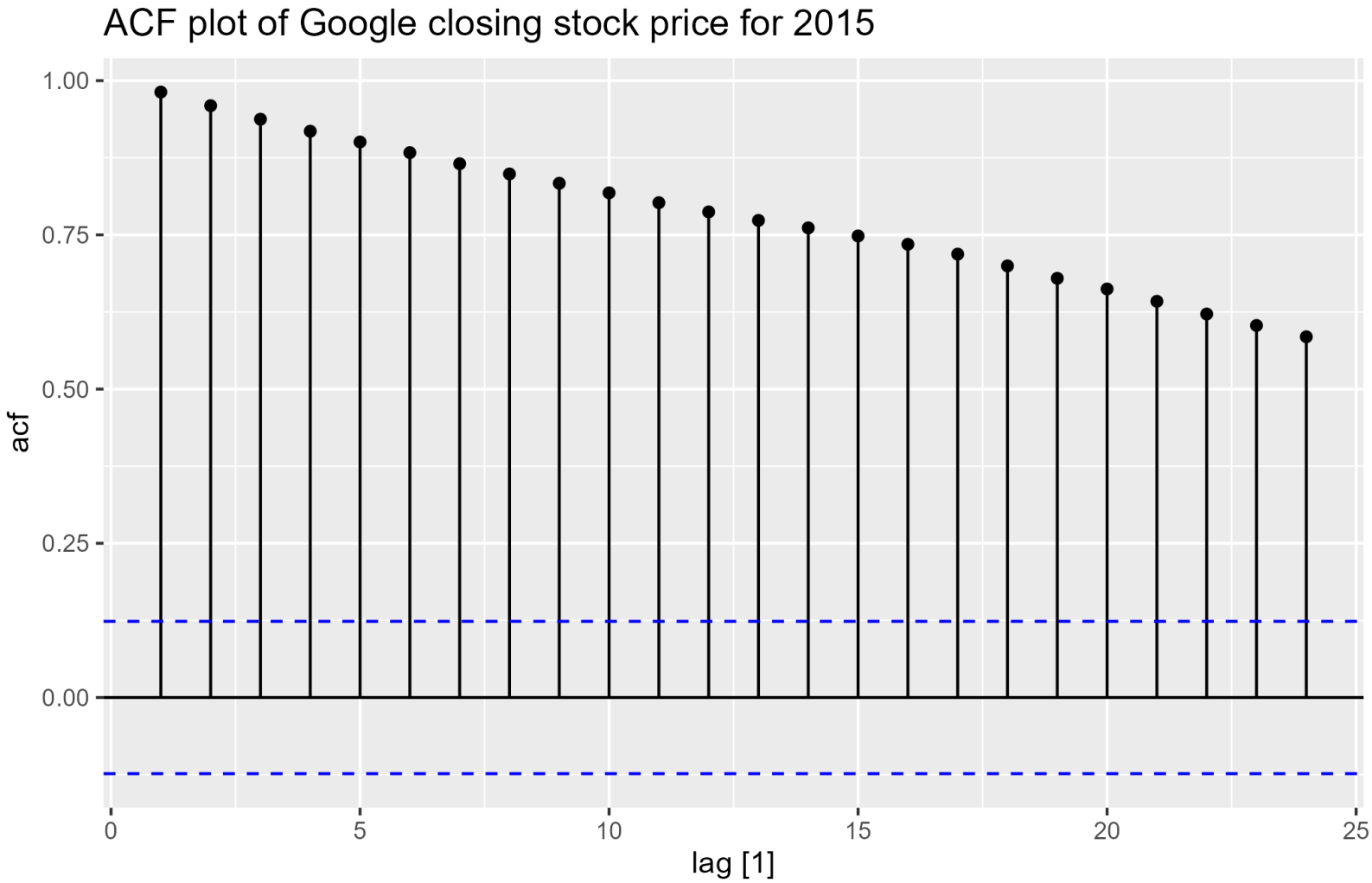
```r
google_2015 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2015)

google_2015 |>
  autoplot(Close) +
  geom_point() +
  labs(title = "Google closing stock price for 2015")
```



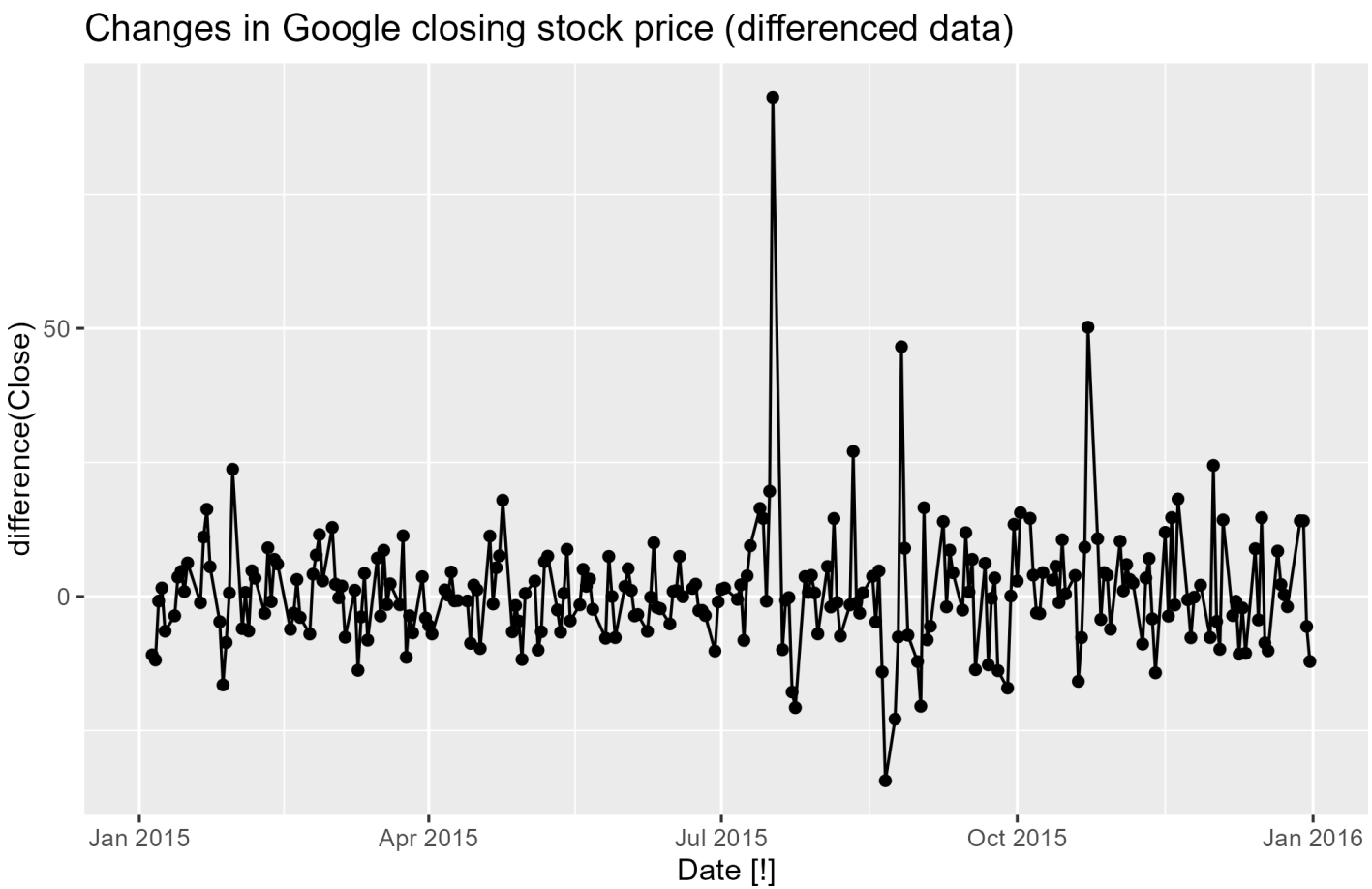
- Is it stationary?

```
google_2015 |>
  ACF(Close) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot of Google closing stock price for 2015")
```


ACF plot of Google closing stock price for 2015
1.00
0.75
0.50
0.25
0.00
acf
0
5
10
15
20
25
lag [1]

```
google_2015 |>
  mutate(CloseD = difference(Close)) -> google_2015


google_2015 |>
  autoplot(CloseD) +
  geom_point() +
  labs(title = "Changes in Google closing stock price (differenced data)")
```


Changes in Google closing stock price (differenced data)
difference(Close)
50
0
Jan 2015
Apr 2015
Jul 2015
Oct 2015
Jan 2016
Date [!]

```
google_2015 |>
  ACF(CloseD) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot of differenced Google closign stock price for 2015")

#is the differenced data really white noise?
google_2015 |>
  mutate(diff = difference(Close)) |>
  features(diff, ljung_box, lag = 10)
#p = 0.637 => yes, it is.
```

```
  Symbol lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 GOOG      7.91     0.637
```

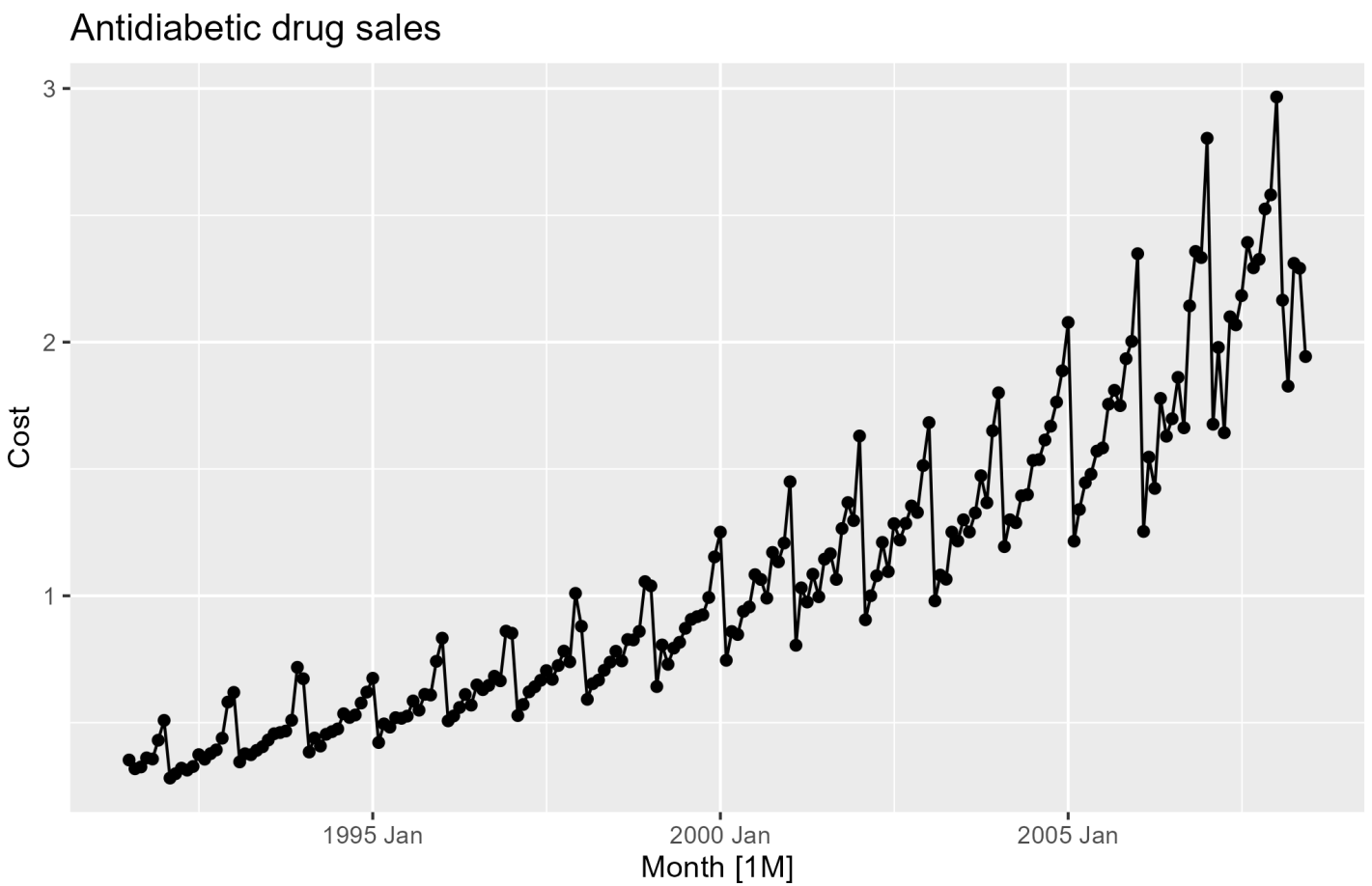
```
#check the non-differenced data
google_2015 |>
  features(Close, ljung_box, lag = 10) # this is not white noise
```

```
  Symbol lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 GOOG     2083.         0
```

- Differencing can be repeated, if we consider that the result is not stationary yet. We can have second-order differencing.
- We can also have seasonal differencing, where we take the difference between an observation and the previous observation from the same season.
- The *unitroot_kpss* feature will perform the KPSS statistical test to determine if the time series is stationary.
- The *unitroot_ndiffs* and *unitroot_nsdiffs* functions will tell you how many differencing and seasonal differencing is required.
- Getting the right number of differencing is important, doing it too many times might introduce false dynamics and autocorrelations.

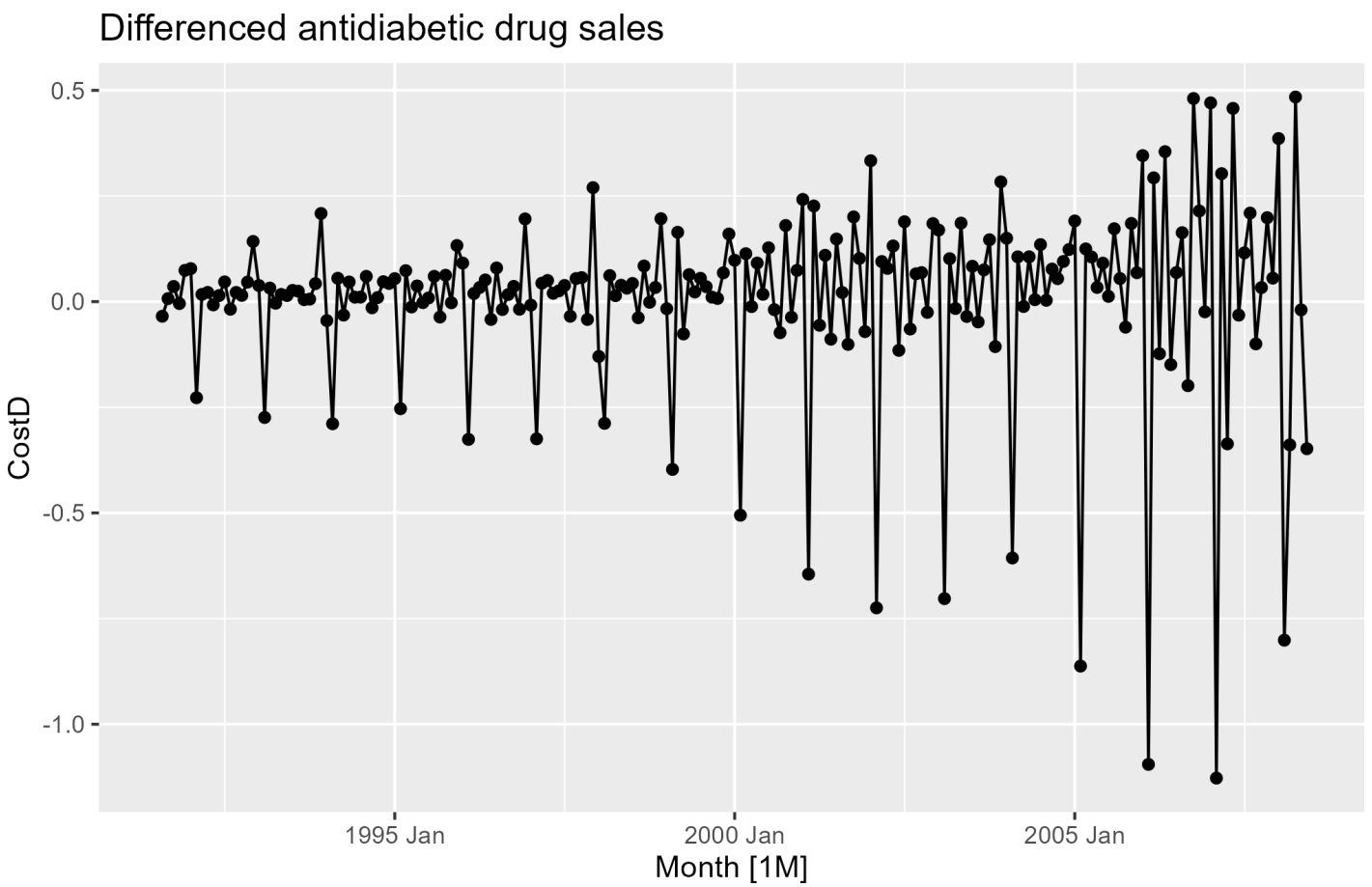
```
PBS |>
  filter(ATC2 == "A10") |>
  summarize(Cost = sum(Cost) / 10e6) -> a10

a10 |>
  autoplot() +
  geom_point() +
  labs(title = "Antidiabetic drug sales")
```


Antidiabetic drug sales
Cost
3
2
1
1995 Jan
2000 Jan
2005 Jan
Month [1M]

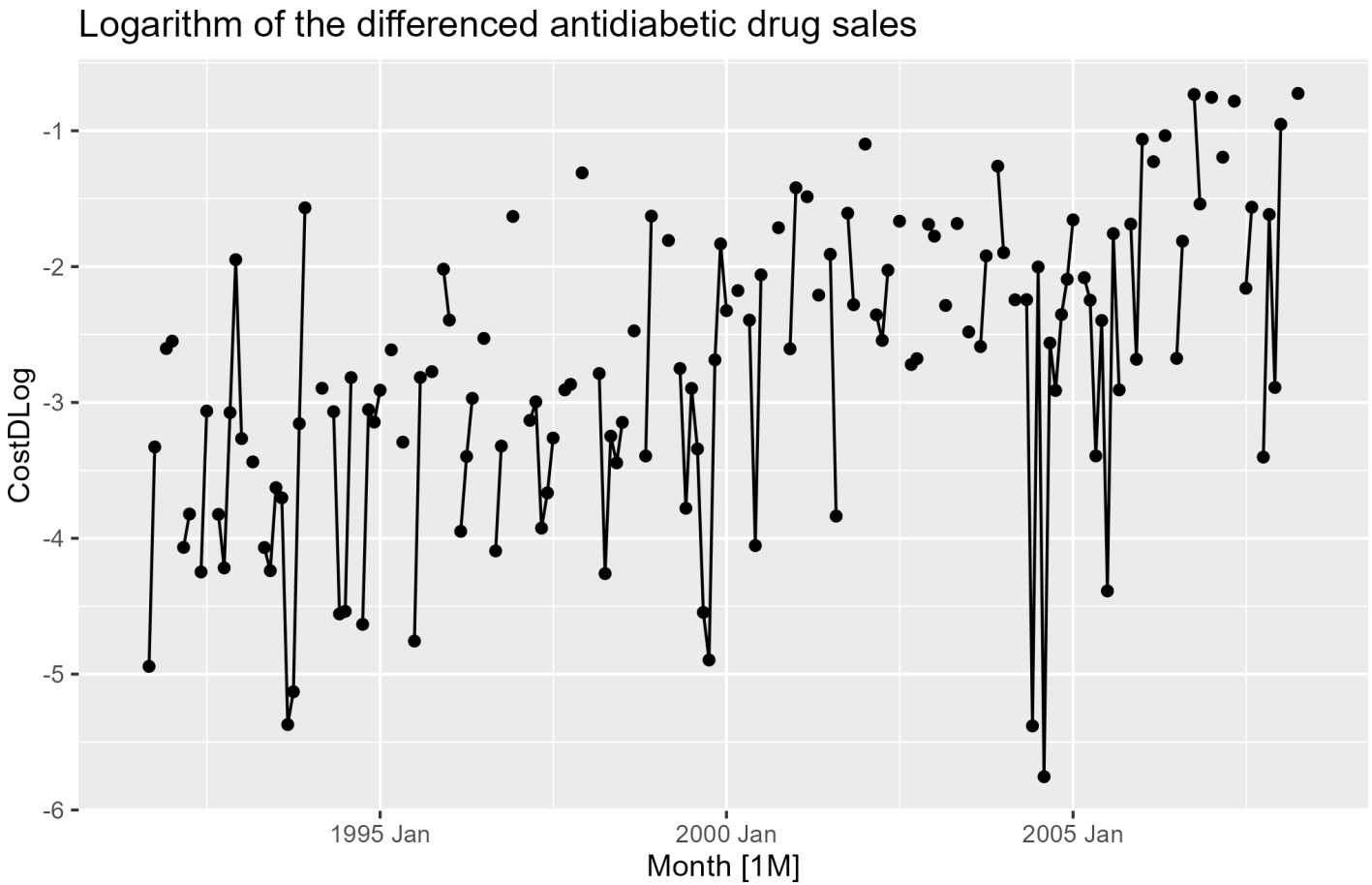
```
a10 |>
  mutate(CostD = difference(Cost)) -> a10

a10 |>
  autoplot(CostD) +
  geom_point() +
  labs(title = "Differenced antidiabetic drug sales")
```


Differenced antidiabetic drug sales
0.5
0.0
-0.5
-1.0
CostD
1995 Jan
2000 Jan
2005 Jan
Month [1M]

```
a10 |>
  mutate(CostDLog = log(CostD)) -> a10

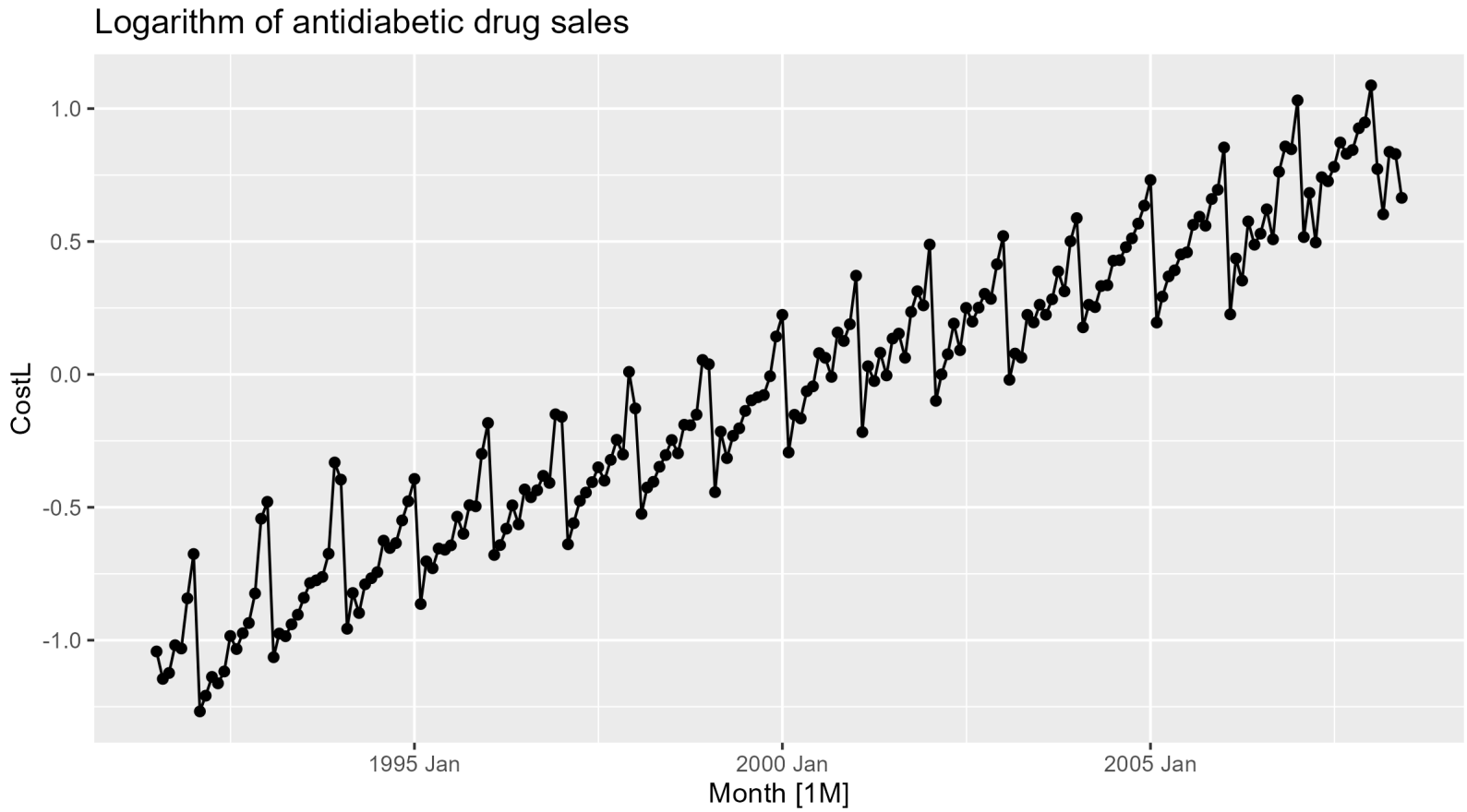
a10 |>
  autoplot(CostDLog) +
  geom_point() +
  labs(title = "Logarithm of the differenced antidiabetic drug sales")
```



- What happened with the data?

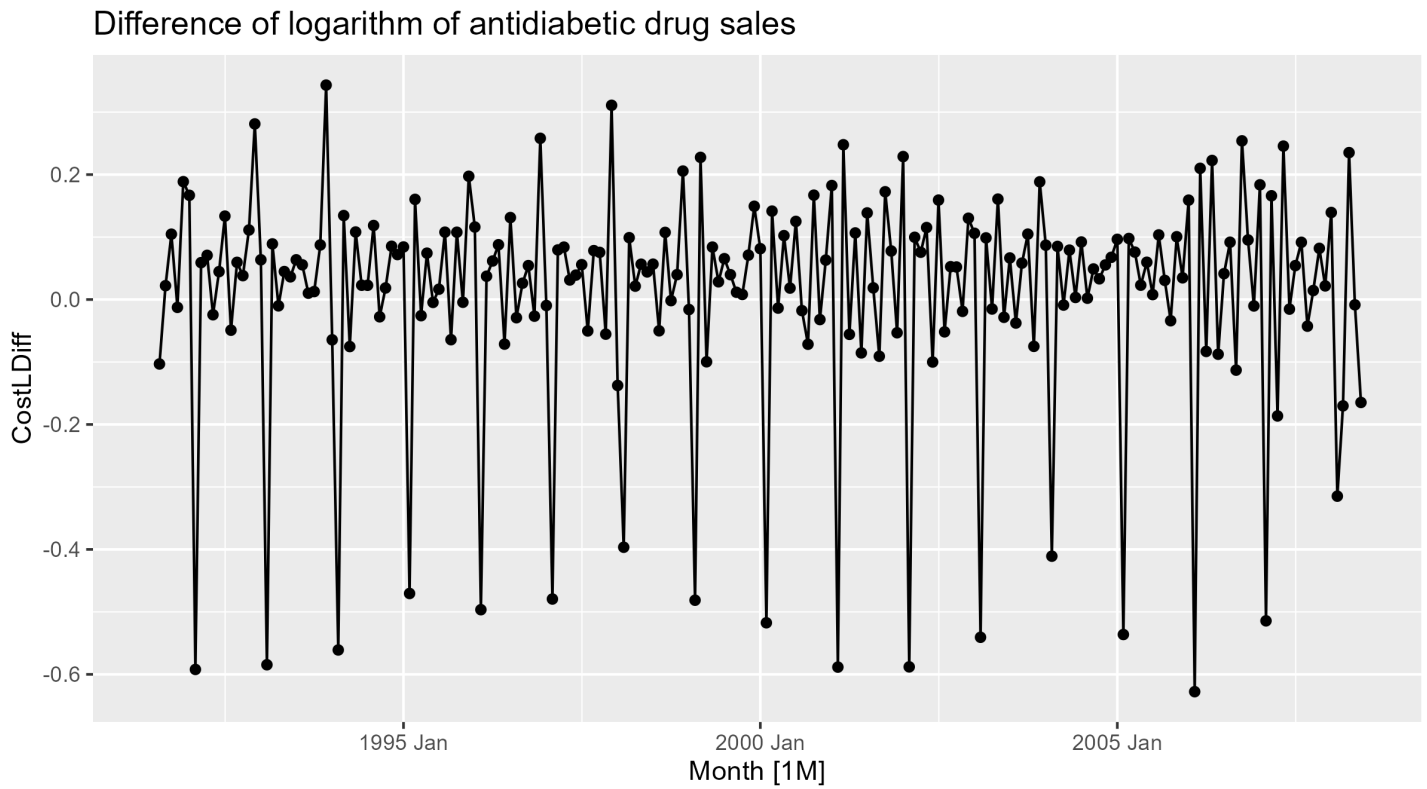
```
#let's do it the other way around
a10 |>
  mutate(CostL = log(Cost)) -> a10

a10 |>
  autoplot(CostL) +
  geom_point() +
  labs(title = "Logarithm of antidiabetic drug sales")
```


Logarithm of antidiabetic drug sales
CostL
1.0
0.5
0.0
-0.5
-1.0
1995 Jan
2000 Jan
2005 Jan
Month [1M]

```
a10 |>
  mutate(CostLDiff = difference(CostL)) -> a10

a10 |>
  autoplot(CostLDiff) +
  geom_point() +
  labs(title = "Difference of logarithm of antidiabetic drug sales")
```

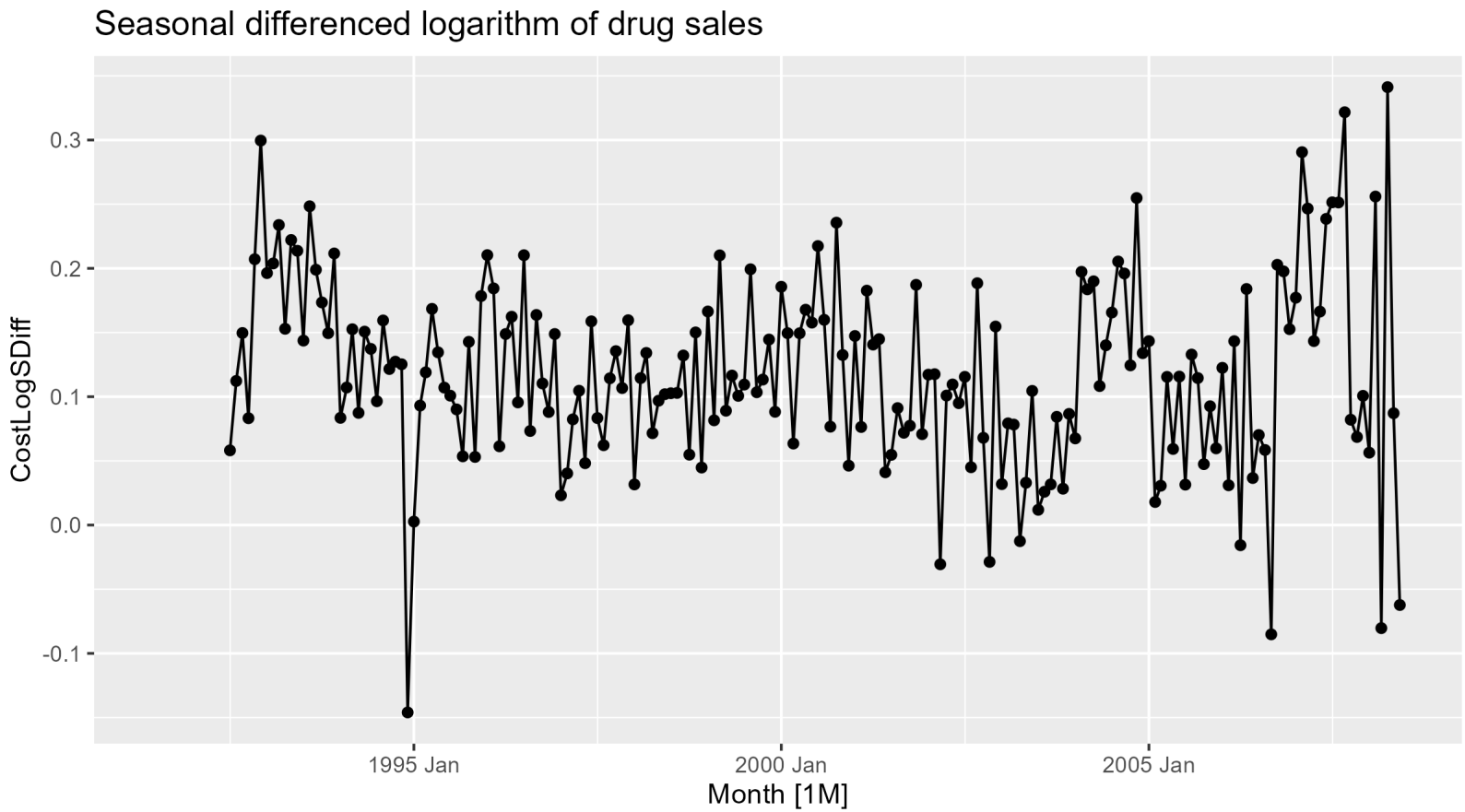


- Does this look stationary?

```
#Let's try it the normal way: logarithm + seasonal difference
a10 |>
  select(Cost) -> a10 #remove all the previous columns

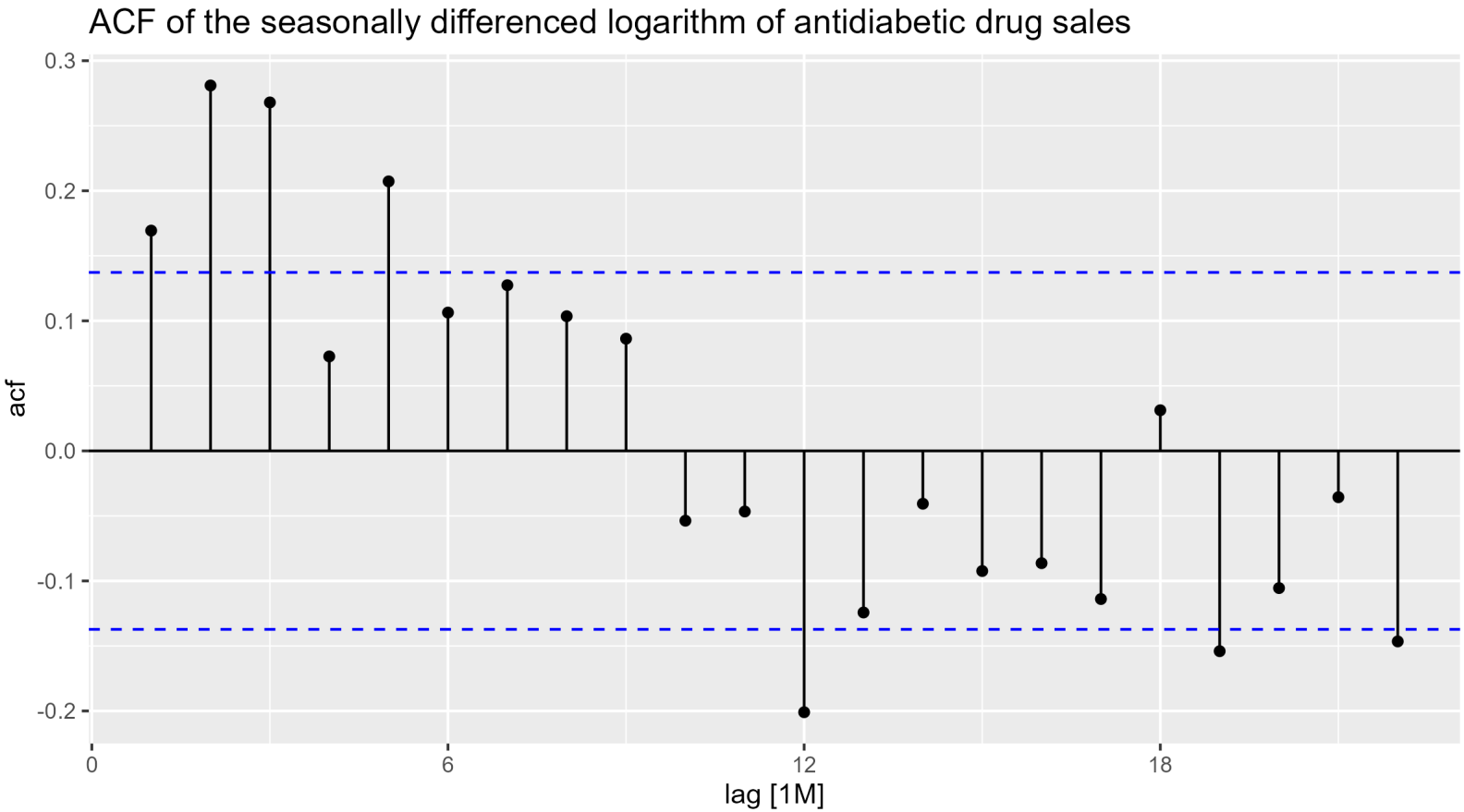
a10 |>
  mutate(CostLog = log(Cost)) |>
  mutate(CostLogSDiff = difference(CostLog, 12)) -> a10

a10 |>
  autoplot(CostLogSDiff) +
  geom_point() +
  labs(title = "Seasonal differenced logarithm of drug sales")
```



- Does this look stationary?

```
a10 |>
  ACF(CostLogSDiff) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of the seasonally differenced logarithm of antidiabetic drug
      sales")
```


ACF of the seasonally differenced logarithm of antidiabetic drug sales
acf
0.3
0.2
0.1
0.0
-0.1
-0.2
0
6
12
18
lag [1M]

```
a10 |>
  features(CostLogSDiff, ljung_box, lag = 24) # this is not white noise
```

```
  lb_stat lb_pvalue
    <dbl>     <dbl>
1    94.5  2.55e-10
```

- While it looked stationary, the ACF and the ljung_box tests show that the data is not white noise.
- However, stationary data is not necessarily white noise (but white noise is stationary).
- We can check if the data is stationary by the KPSS test (where a p-value above 0.05 means that the data is stationary)

```
a10 |>
  features(CostLogSDiff, unitroot_kpss)
```

```
  kpss_stat kpss_pvalue
      <dbl>       <dbl>
1     0.209         0.1
```

- We can check the number of regular and seasonal differences that are needed to make the data stationary
- The two values are independent, it might happen that doing just the seasonal difference is sufficient (like in our case)

```
a10 |>
  features(CostLog, c(unitroot_ndiffs, unitroot_nsdiffs))
```

```
# A tibble: 1 × 2
  ndiffs nsdiffs
   <int>   <int>
1      1       1
```

The backshift notation I

- The backward shifting operator, B, is useful when we are working with lags, it simply denotes the data shifted backwards a time unit.

$$By_t = y_{t-1}$$

- We can apply the operator twice:

$$B(By_t) = B^2 y_t = y_{t-2}$$

- When we are working with seasonal data, we can use the operator to reference values from the previous season. For example, for monthly data:

$$B^{12} y_t = y_{t-12}$$

The backshift notation II

- We can use it to describe differencing (and the resulting differenced data):

$$y'_t = y_t - y_{t-1} = y_t - By_t = (1 - B) * y_t$$

- If we want to difference the data twice, we can write it as:

$$(1 - B)^2 y_t$$

- and more generally, a d-th order difference is denoted by:

$$(1 - B)^d y_t$$

- We can use it to describe consecutive differencing as well:

$$(1 - B)(1 - B^m) y_t$$

- means a seasonal differencing (where m is the seasonal period) followed by a regular differencing.

Autoregressive models I

- We have talked about regression: when we want to forecast a variable using a linear combination of other predictor variables. In an *autoregressive* model, we try to forecast a variable using a linear combination of past values of that variable (hence the name *auto*regression)
- An important parameter of an autoregressive model is the number of past values to be considered, denoted by p.
- An autoregressive model of order p, denoted by **AR(p)** can be written as:

$$y_t = c + \phi_1 * y_{t-1} + \phi_2 * y_{t-2} + ... + \phi_p * y_{t-p} + \epsilon_t$$

- where ϵ_t is the white noise error.
- Autoregressive models are very flexible in handling a large variety of different time series.

Autoregressive models II

- For an AR(1) model ($y_t = c + \phi_1 * y_{t-1} + \epsilon_t$) we have:
  - if $\phi_1 = c = 0 \Rightarrow y_t$ is equivalent to white noise
  - if $\phi_1 = 1$ and $c = 0$, y_t is equivalent to a random walk
  - if $\phi = 1$ and $c \neq 0$, y_t is equivalent to a random walk with drift
  - if $\phi_1 < 0$, y_t in general oscillates around the mean.
- Usually, autoregressive models are used with stationary data, when some constraints are required for the parameter values:
  - AR(1): $-1 < \phi_1 < 1$
  - AR(2): $-1 < \phi_2 < 1$, $\phi_1 + \phi_2 < 1$, $\phi_2 - \phi_1 < 1$
  - For $p > 2$ the constraints are more complex, but the *fable* package in R can handle them.

Moving average models I

- The autoregressive model builds a regression model using the previous values of the variable as predictors.
- A moving average model builds a regression model using the past forecast errors as predictors.
- A moving average model of order q, denoted by **MA(q)** can be written as:

$$y_t = c + \epsilon_t + \theta_1 * \epsilon_{t-1} + \theta_2 * \epsilon_{t-2} + ... + \theta_q * \epsilon_{t-q}$$

- where ϵ_t is white noise.
- **Obs.** *moving average models* are not the same as *moving average smoothing*. The former is used to forecast the value of an observation, while the latter is used to estimate the trend-cycle component of a time series.

Moving average models II

- It is possible to write any stationary $AR(p)$ model as an $MA(\infty)$ model. For example, for an AR(1) model we could have:

$$y_t = \phi_1 * y_{t-1} + \epsilon_t =$$

$$= \phi_1 * (\phi_1 * y_{t-2} + \epsilon_{t-1}) + \epsilon_t =$$

$$= \phi_1^2 * y_{t-2} + \phi_1 * \epsilon_{t-1} + \epsilon_t =$$

$$= \phi_1^3 * y_{t-3} + \phi_1^2 * \epsilon_{t-2} + \phi_1 * \epsilon_{t-1} + \epsilon_t = ...$$

- Rearranging the terms a little, we will get:

$$y_t = \epsilon_t + \phi_1 * \epsilon_{t-1} + \phi_1^2 * \epsilon_{t-2} + \phi_1^3 * \epsilon_{t-3} + ...$$

- If $-1 < \phi_1 < 1$ the coefficients will be smaller and smaller and we will get an $MA(\infty)$ process.

Moving average models III

- In order for the reverse result to hold, we need some constraints on the parameter θ. If these hold, we call the model *invertible*, we can write any $MA(q)$ process as an $AR(\infty)$ process.
- Consider the $MA(1)$ process:

$$y_t = \epsilon_t + \theta_1 * \epsilon_{t-1}$$

- From here we can have:

$$\epsilon_t = y_t - \theta_1 * \epsilon_{t-1} =$$

$$= y_t - \theta_1 * (y_{t-1} - \theta_1 * \epsilon_{t-2}) =$$

$$= y_t - \theta_1 * y_{t-1} + \theta_1^2 * \epsilon_{t-2} =$$

$$= y_t - \theta_1 * y_{t-1} + \theta_1^2 * (y_{t-2} - \theta_1 * \epsilon_{t-3}) =$$

$$= y_t - \theta_1 * y_{t-1} + \theta_1^2 * y_{t-2} - \theta_1^3 * \epsilon_{t-3} = ... \Rightarrow$$

Moving average models IV

$$\epsilon_t = \sum_{j=0}^{\infty} (-\theta_1)^j * y_{t-j}$$

- If $|\theta_1| > 1$ the coefficients will be larger as we go further back in time; if $|\theta_1| = 1$ all terms will have the same *weight*. This is not what we want, we want the most recent observations to matter more, so we need $|\theta_1| < 1$.
- Thus we have the following invertibility conditions:
  - the MA(1) model: $-1 < \theta_1 < 1$
  - the MA(2) model: $-1 < \theta_2 < 1$, $\theta_1 + \theta_2 > -1$, $\theta_1 - \theta_2 < 1$

The non-seasonal ARIMA model I

- ARIMA stands for *AutoRegressive Integrated Moving Average* and it combines differencing with autoregression and moving averages.
- The model can be written in the following way:

$$y'_t = c + \phi_1 * y'_{t-1} + ... + \phi_p * y'_{t-p} + \theta_1 * \epsilon_{t-1} + ... + \theta_q * \epsilon_{t-q} + \epsilon_t$$

- where y'_t is the differenced series.
- This model has three important parameters:
  - p - the number of autoregressive terms
  - q - the number of moving average terms
  - d - the number of times the series was differenced (of order 1)
- so it is denoted as ARIMA(p, d, q)

The non-seasonal ARIMA model II

- For some values of p, d and q we get models that were already discussed:
  - ARIMA(0, 0, 0) with no constant is just white noise
  - ARIMA(0, 1, 0) with no constant is just a random walk
  - ARIMA(0, 1, 0) with a constant is a random walk with a drift
  - ARIMA(p, 0, 0) is autoregression
  - ARIMA(0, 0, q) is moving average
- If we use the backshift notation, we can express the ARIMA model easier:

$$(1-\phi_1*B-...-\phi_p*B^p)(1-B)^d y_t = c+(1+\theta_1*B+...+\theta_q*B^q)*\epsilon_t$$

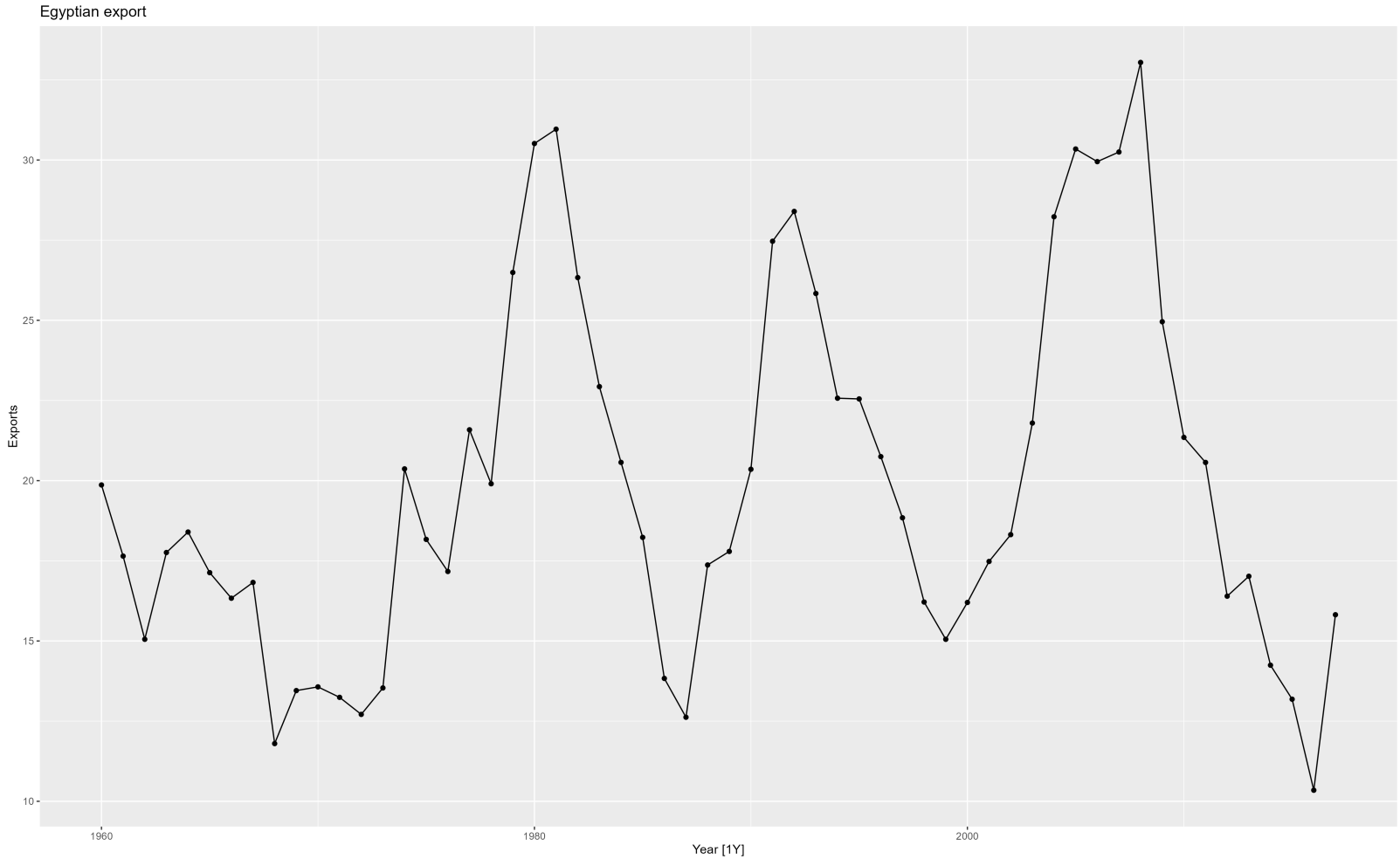
The non-seasonal ARIMA model III

- Selecting the best values for the three parameters, p, d, and q, can be complicated, but the *ARIMA* function from the fable package will do it automatically.
- Let's see an example, considering the exports for Egypt between 1960 and 2017.

```
egyptian_export <- global_economy |>
  filter(Code == "EGY") |>
  select(Exports)

egyptian_export |>
  autoplot(Exports) +
  geom_point() +
  labs(title = "Egyptian export")
```

The non-seasonal ARIMA model IV



The non-seasonal ARIMA model V

```
egyptian_arima_model <- egyptian_export |>
  model(ARIMA(Exports))

egyptian_arima_model |>
  report()
```

```
Series: Exports
Model: ARIMA(2,0,1) w/ mean

Coefficients:
         ar1      ar2      ma1  constant
      1.6764  -0.8034  -0.6896    2.5623
s.e.  0.1111   0.0928   0.1492    0.1161

sigma^2 estimated as 8.046:  log likelihood=-141.57
AIC=293.13   AICc=294.29   BIC=303.43
```

The non-seasonal ARIMA model VI

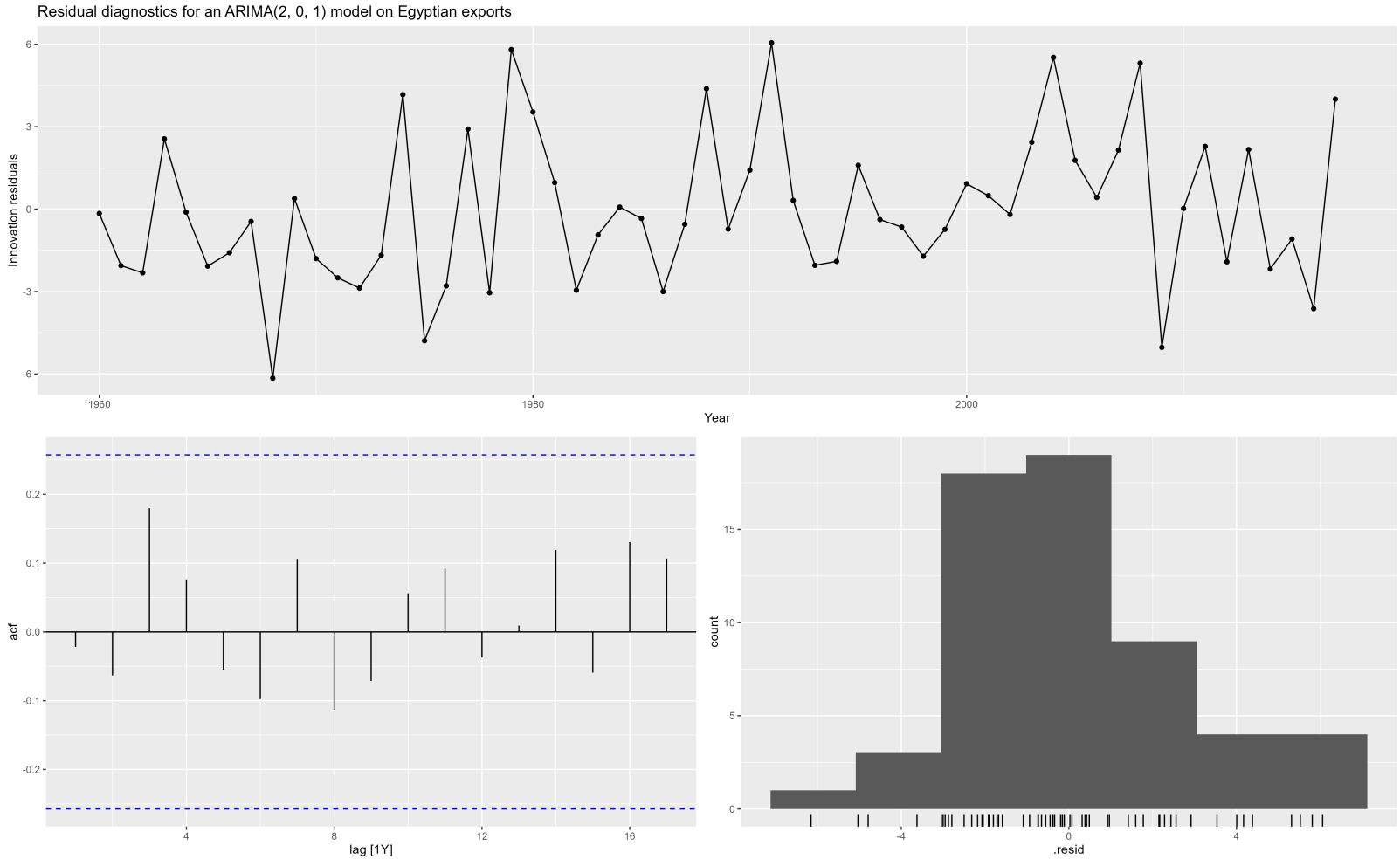
- We can see that we have an ARIMA(2, 0, 1) model and the *report* function provides the exact coefficients for the autoregressive and the moving average part as well.
- So we can see that the actual model is:

$$y_t = 2.56 + 1.68 * y_{t-1} - 0.8 * y_{t-2} - 0.69 * \epsilon_{t-1} + \epsilon_t$$

- If we look at the residuals, we can see that the model captures the information from the data quite well.
- Forecast for the next 10 years shows that the model captured the cyclic pattern in the data.

```
egyptian_arima_model |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for an ARIMA(2, 0, 1) model on Egyptian
      exports")
```

The non-seasonal ARIMA model VII

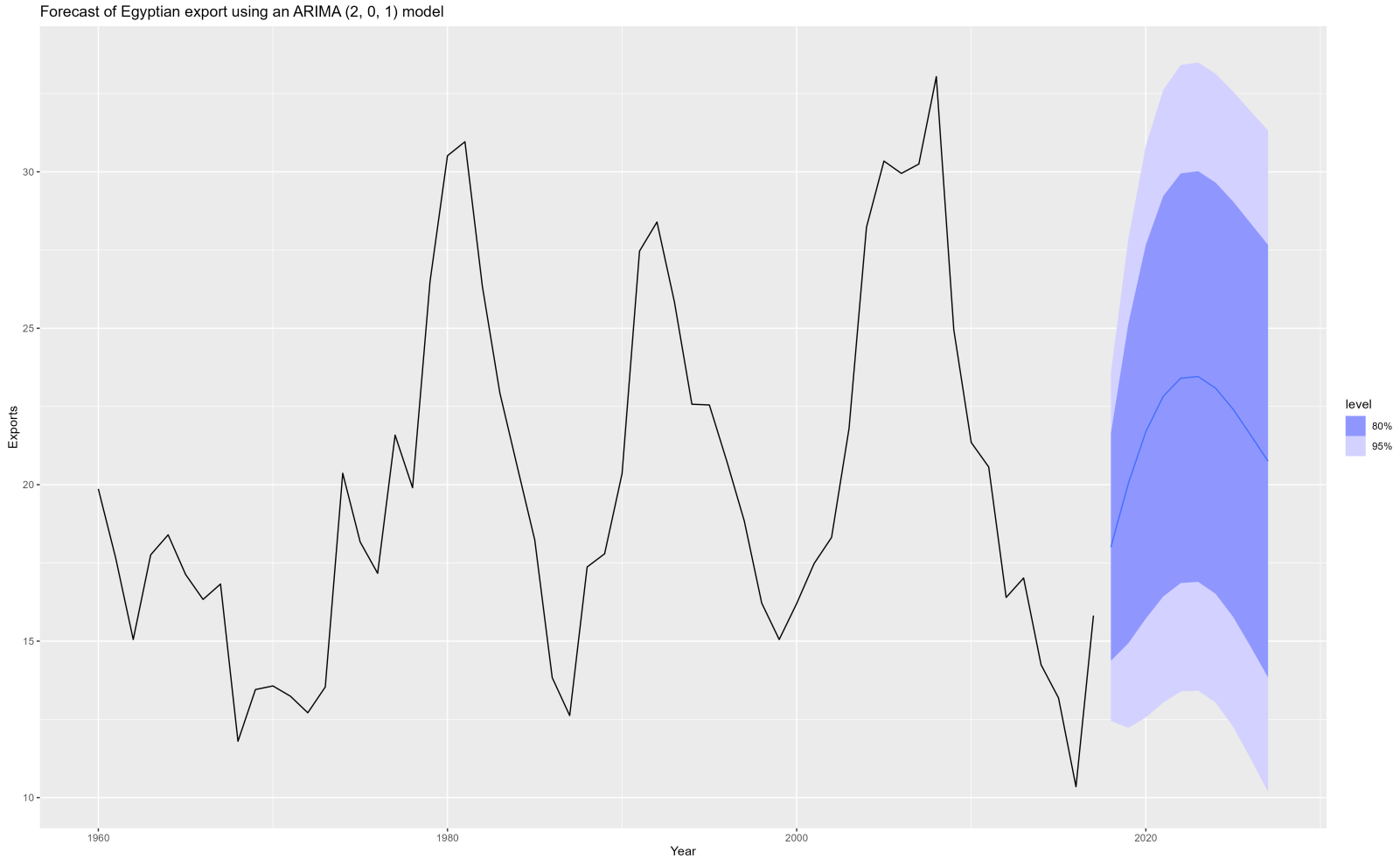


The non-seasonal ARIMA model VIII

```
egyptian_forecast <- egyptian_arima_model |>
  forecast(h = 10)

egyptian_forecast |>
  autoplot(egyptian_export) +
  labs(title = "Forecast of Egyptian export using an ARIMA (2, 0, 1) model")
```

The non-seasonal ARIMA model IX

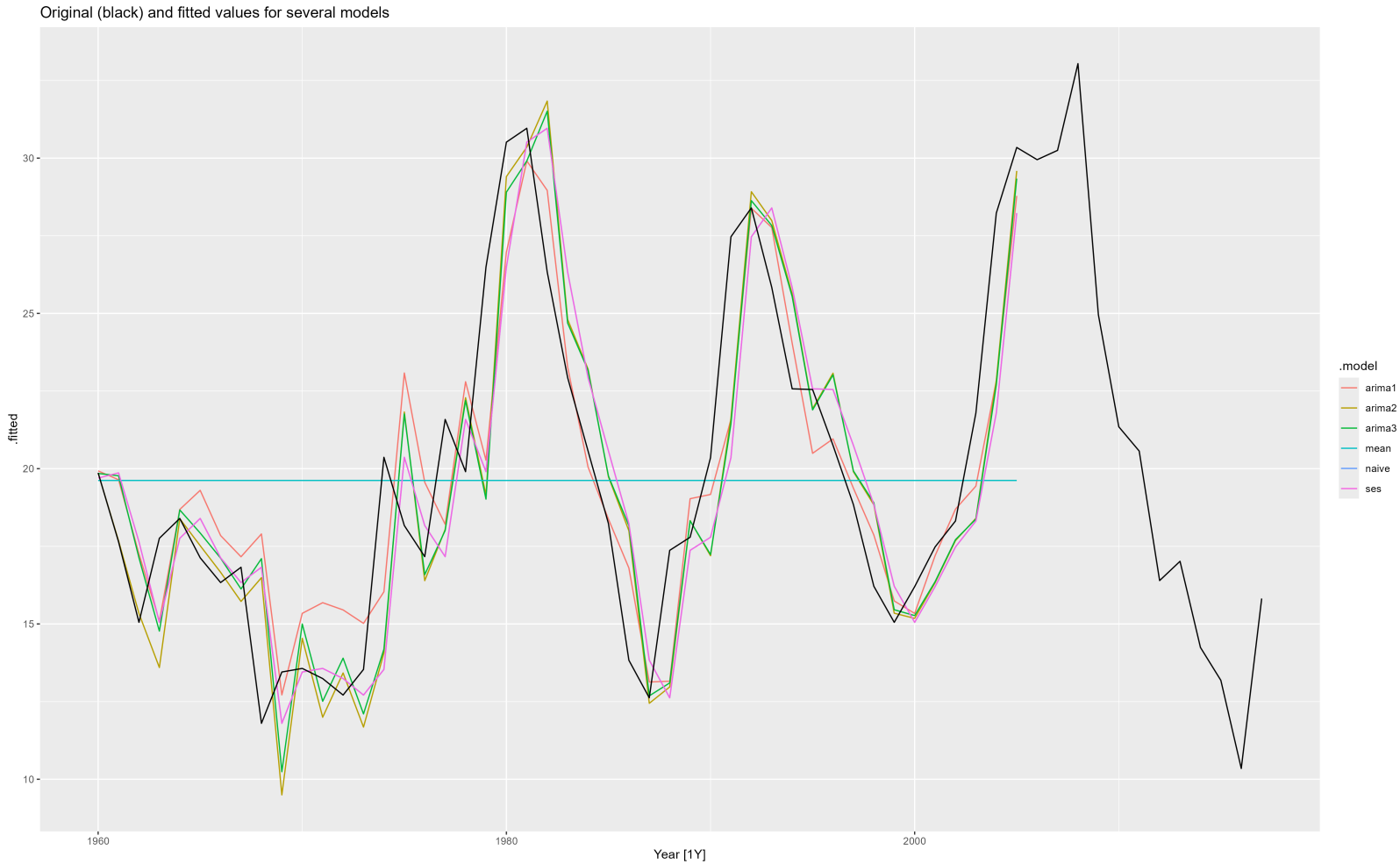


Example

- Let's compare several models for the Egyptian exports data. From the plot we saw that it has no seasonality and no trend, so we will consider the Naive model, the Mean model, three different ARIMA models (including the optimal one) and a Simple Exponential Smoothing model.
- **Obs:** for specifying the parameters p, d and q for the ARIMA model, we can use the *pdq* special.
- When we want to compare several models, we have a few options:
  - Look at the residuals - subjective.
  - Look at the fitted values - subjective.
  - If it is reported, look at the AIC, AICc and BIC measures - not every model can be compared by these measures
  - Do a train-test cross validation - this is what we will do

```r
egyptian_export
egyptian_train <- egyptian_export |>
  head(n = 46)
egyptian_test <- egyptian_export |>
  tail(n = 12)

egyptian_models <- egyptian_train |>
  model (
    naive = NAIVE(Exports),
    mean = MEAN(Exports),
    arima1 = ARIMA(Exports ~ pdq(2, 0, 1)),
    arima2 = ARIMA(Exports ~ pdq(1, 2, 2)),
    arima3 = ARIMA(Exports ~ pdq(1, 1, 1)),
    ses = ETS(Exports)
  )
#plot fitted values
egyptian_models |>
  augment() |>
  autoplot(.fitted) +
  autolayer(egyptian_export) +
  labs(title = "Original (black) and fitted values for several models")
```


Original (black) and fitted values for several models
.fitted
30
25
20
15
10
1960
1980
2000
Year [1Y]
.model
arima1
arima2
arima3
mean
naive
ses

```
egyptian_models |> select(naive) |> report()
```

```
Series: Exports
Model: NAIVE

sigma^2: 9.6079
```

```
egyptian_models |> select(mean) |> report()
```

```
Series: Exports
Model: MEAN

Mean: 19.6209
sigma^2: 26.979
```

```
egyptian_models |> select(arima1) |> report()
```

```
Series: Exports
Model: ARIMA(2,0,1) w/ mean

Coefficients:
         ar1      ar2      ma1  constant
      1.6064  -0.7655  -0.5672    3.1827
s.e.  0.1555   0.1289   0.1990    0.1747

sigma^2 estimated as 7.71:  log likelihood=-111.02
AIC=232.04   AICc=233.54   BIC=241.18
```

- Why are these values different from the ones we saw previously?

```
egyptian_models |> select(arima2) |> report()
```

```
Series: Exports
Model: ARIMA(1,2,2)

Coefficients:
          ar1      ma1      ma2
      -0.4110  -0.3057  -0.6942
s.e.   0.3481   0.2891   0.2830

sigma^2 estimated as 9.428:  log likelihood=-112.05
AIC=232.1   AICc=233.12   BIC=239.23
```

```
egyptian_models |> select(arima3) |> report()
```

```
Series: Exports
Model: ARIMA(1,1,1)

Coefficients:
          ar1     ma1
      -0.4332  0.7047
s.e.   0.3246  0.2563

sigma^2 estimated as 9.028:  log likelihood=-112.42
AIC=230.84   AICc=231.43   BIC=236.26
```

```
egyptian_models |> select(ses) |> report()
```

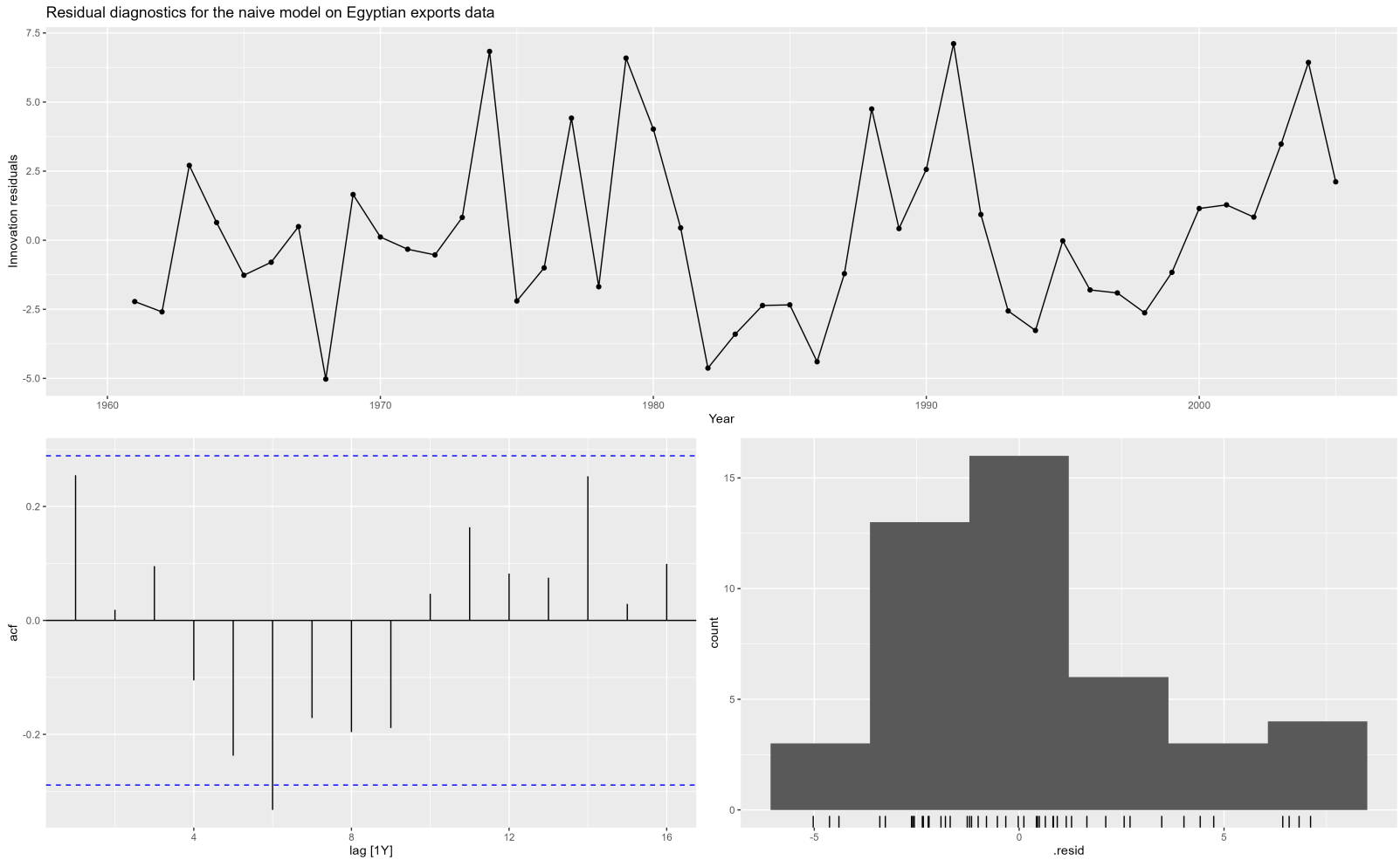
```
Series: Exports
Model: ETS(A,N,N)
  Smoothing parameters:
    alpha = 0.9999

  Initial states:
    l[0]
 19.6961

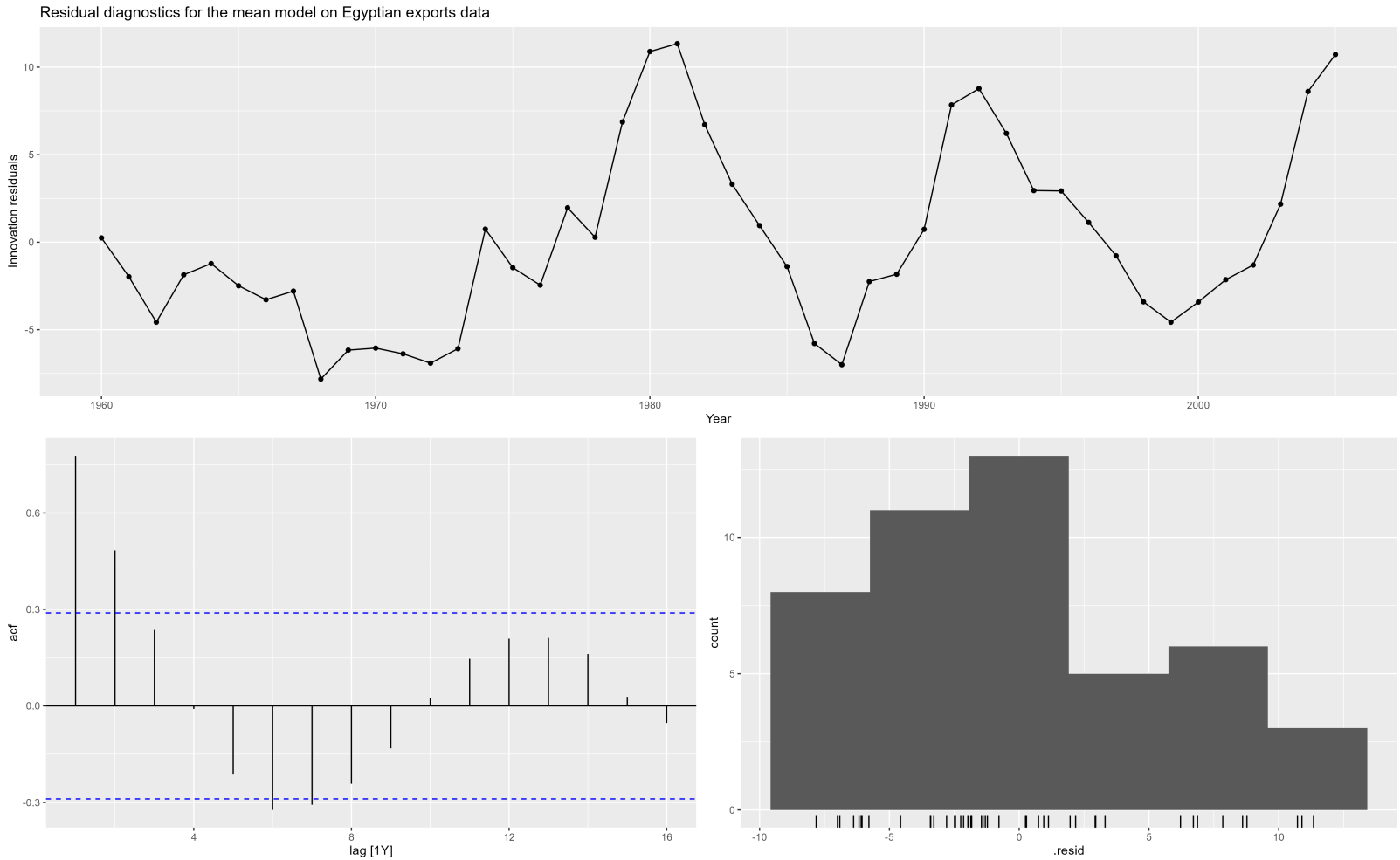
  sigma^2:  9.6645

     AIC     AICc      BIC
284.4220 284.9934 289.9079
```

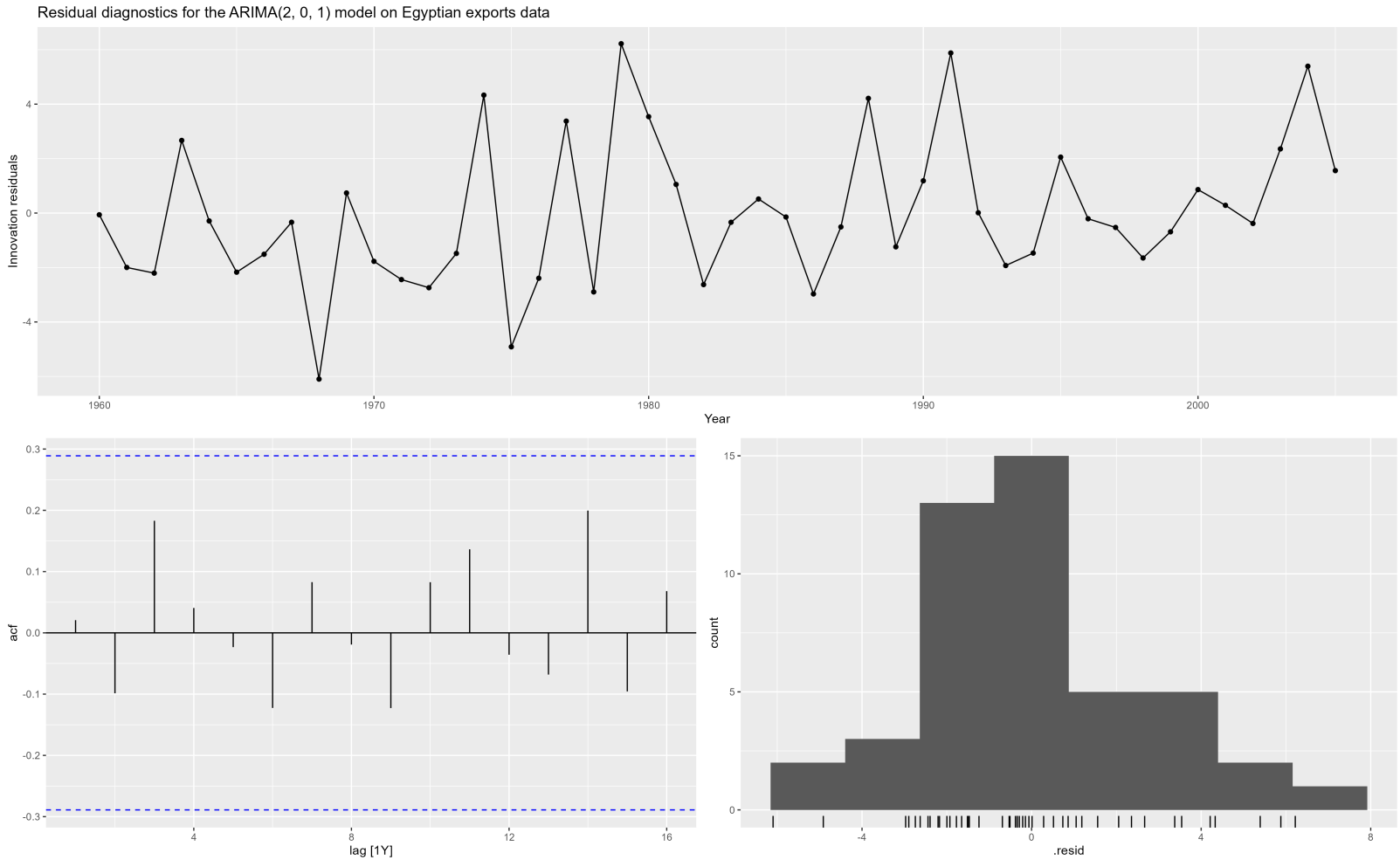
```
#residual diagnostics
egyptian_models |>
  select(naive) |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for the naive model on Egyptian exports
      data")
```


Residual diagnostics for the naive model on Egyptian exports data
Innovation residuals
7.5
5.0
2.5
0.0
-2.5
-5.0
1960
1970
1980
1990
2000
Year
acf
0.2
0.0
-0.2
4
8
12
16
lag [1Y]
count
15
10
5
0
-5
0
5
.resid

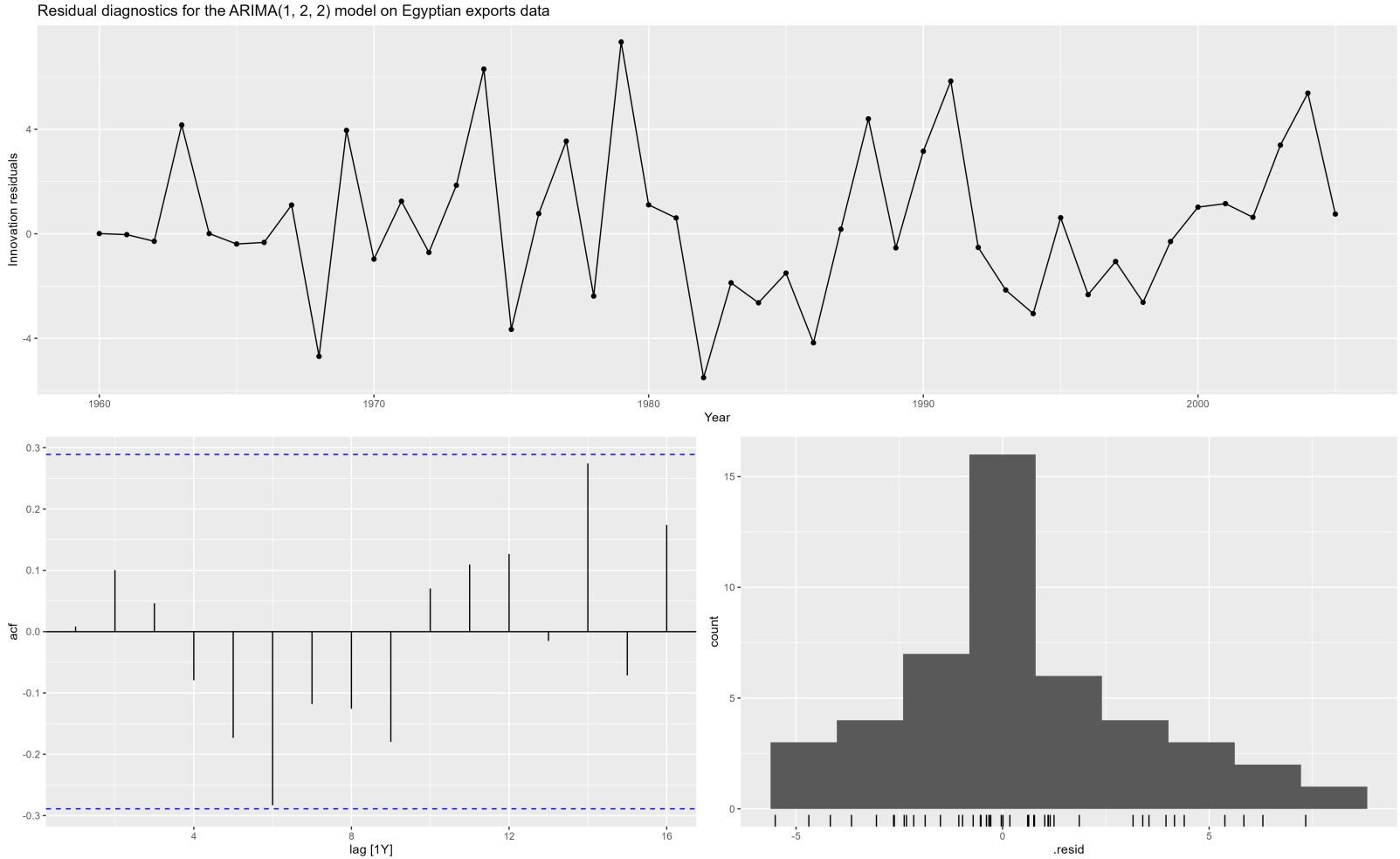
```
egyptian_models |>
  select(mean) |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for the mean model on Egyptian exports data
     ")
```


Residual diagnostics for the mean model on Egyptian exports data
Innovation residuals
10
5
0
-5
1960
1970
1980
1990
2000
Year
acf
0.6
0.3
0.0
-0.3
4
8
12
16
lag [1Y]
count
10
5
0
-10
-5
0
5
10
.resid

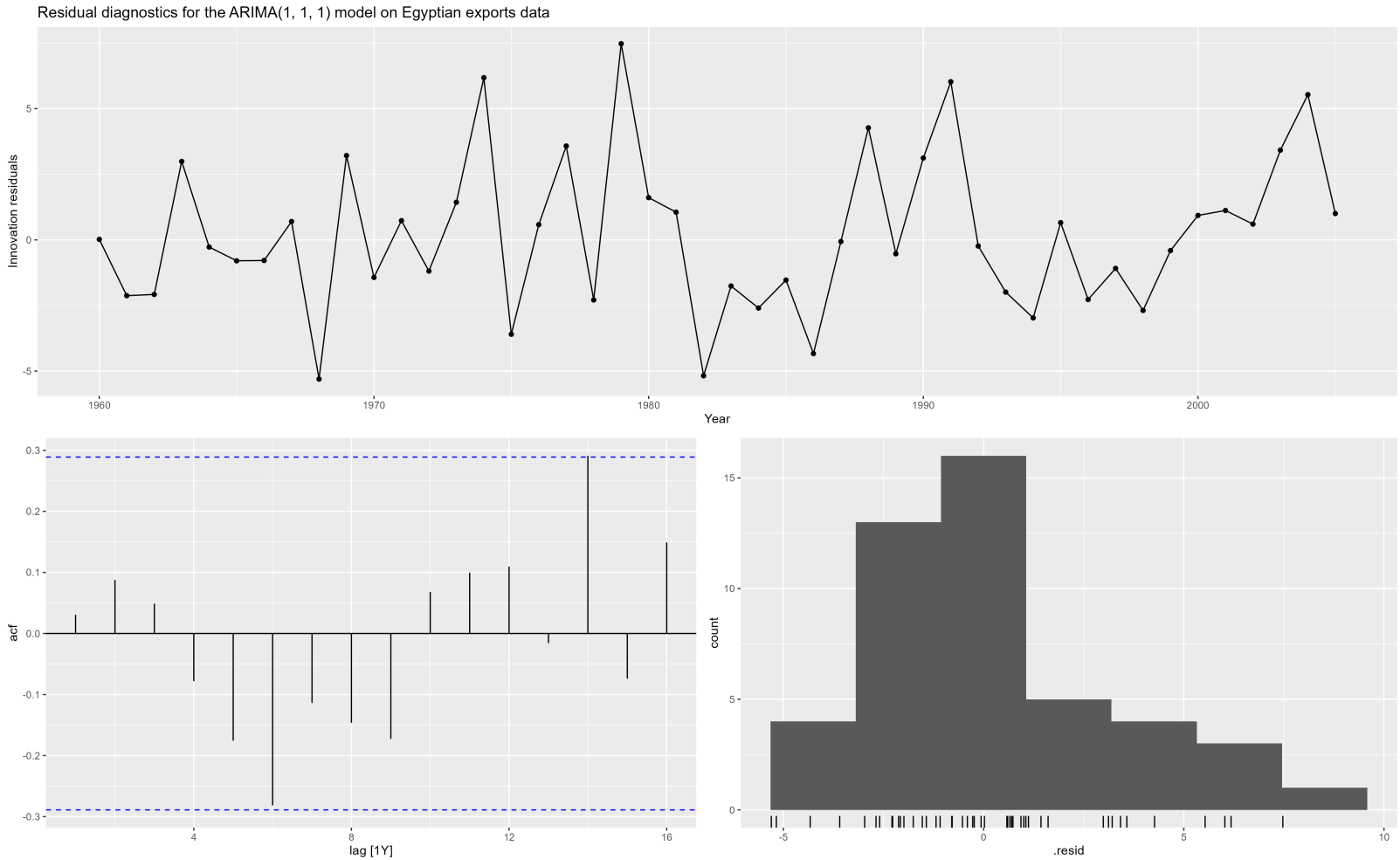
```
egyptian_models |>
  select(arima1) |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for the ARIMA(2, 0, 1) model on Egyptian
      exports data")
```


Residual diagnostics for the ARIMA(2, 0, 1) model on Egyptian exports data
Innovation residuals
4
0
-4
1960
1970
1980
1990
2000
Year
acf
0.3
0.2
0.1
0.0
-0.1
-0.2
-0.3
4
8
12
16
lag [1Y]
count
15
10
5
0
-4
0
4
8
.resid

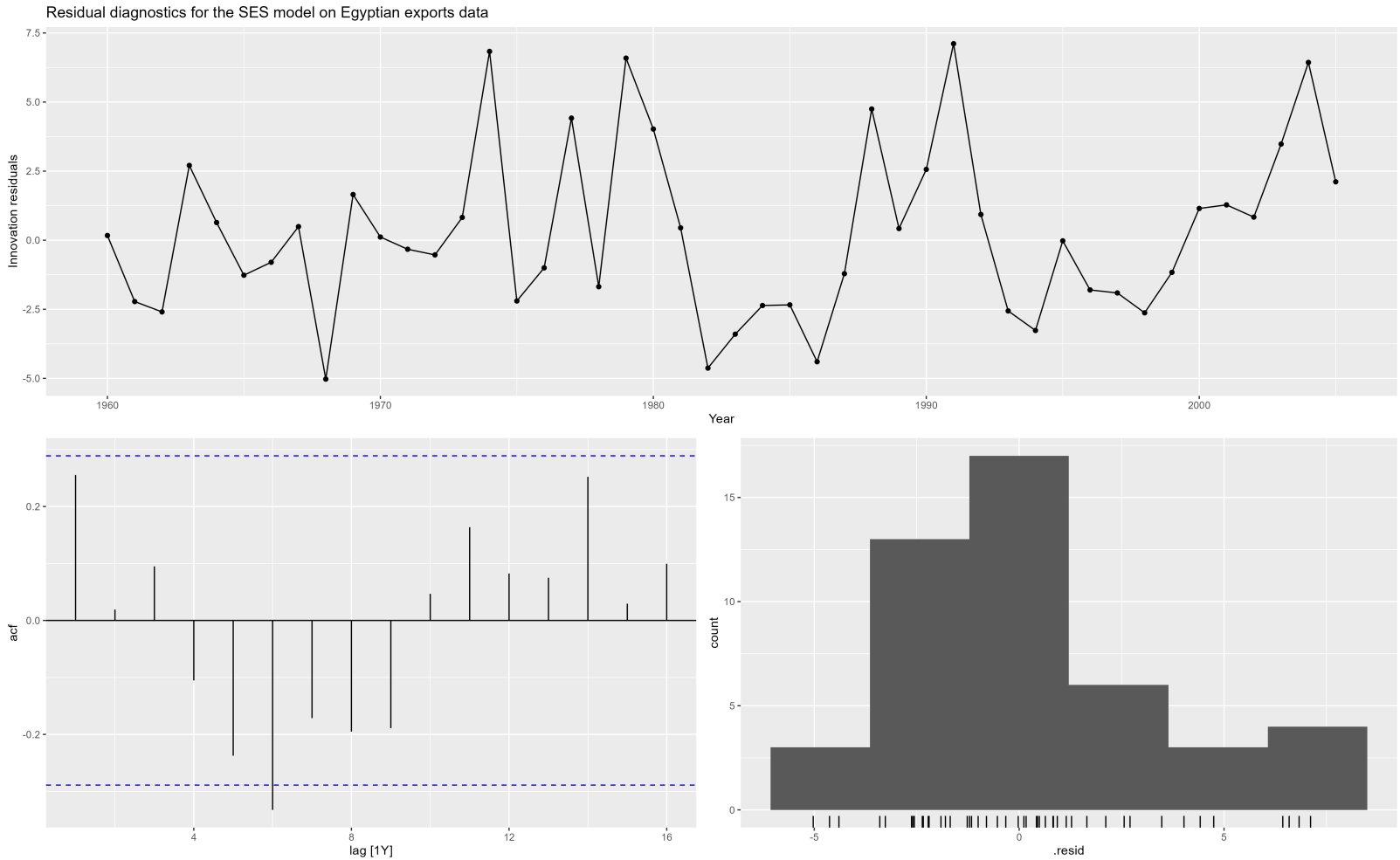
```
egyptian_models |> select(arima2) |>
egyptian_models |>
  select(arima2) |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for the ARIMA(1, 2, 2) model on Egyptian
      exports data")
```


Residual diagnostics for the ARIMA(1, 2, 2) model on Egyptian exports data
Innovation residuals
4
0
-4
1960
1970
1980
1990
2000
Year
acf
0.3
0.2
0.1
0.0
-0.1
-0.2
-0.3
4
8
12
16
lag [1Y]
count
15
10
5
0
-5
0
5
.resid

```
egyptian_models |>
  select(arima3) |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for the ARIMA(1, 1, 1) model on Egyptian
      exports data")
```


Residual diagnostics for the ARIMA(1, 1, 1) model on Egyptian exports data
Innovation residuals
5
0
-5
1960
1970
1980
1990
2000
Year
acf
0.3
0.2
0.1
0.0
-0.1
-0.2
-0.3
4
8
12
16
lag [1Y]
count
15
10
5
0
-5
0
5
10
.resid

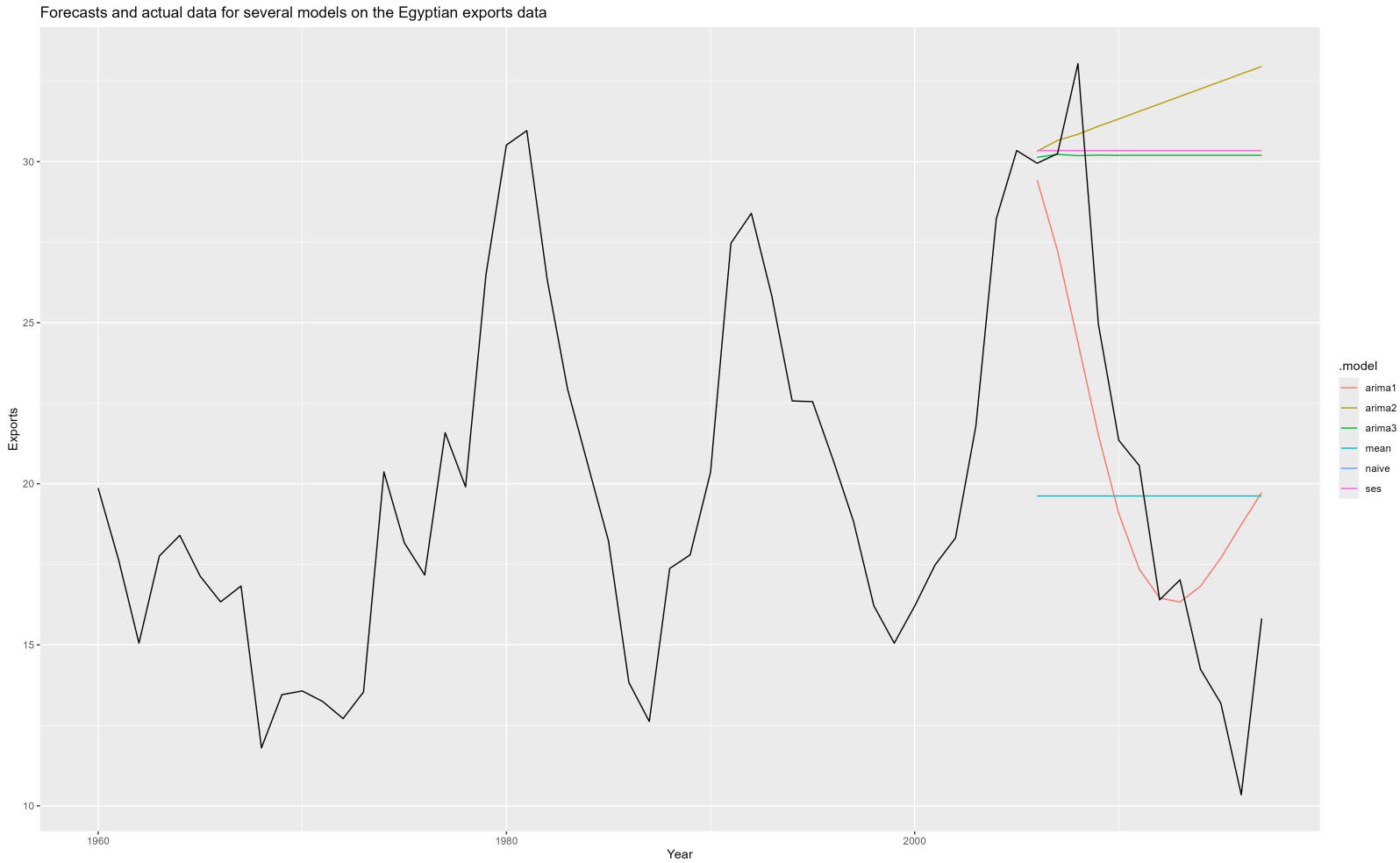
```
egyptian_models |>
  select(ses) |>
  gg_tsresiduals() +
  labs(title = "Residual diagnostics for the SES model on Egyptian exports data"
    )
```


Residual diagnostics for the SES model on Egyptian exports data
Innovation residuals
7.5
5.0
2.5
0.0
-2.5
-5.0
1960
1970
1980
1990
2000
Year
acf
0.2
0.0
-0.2
4
8
12
16
lag [1Y]
count
15
10
5
0
-5
0
5
.resid

```r
#forecast and compute performance measures
egyptian_forecasts <- egyptian_models |>
  forecast(h = 12)
accuracy(egyptian_forecasts, egyptian_test)
```

.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 arima1	Test	0.197	4.33	3.44	-6.37	20.0	NaN	NaN	0.676
2 mean	Test	0.973	7.19	6.09	-7.58	31.6	NaN	NaN	0.785
3 arima3	Test	-9.60	12.0	10.1	-65.6	67.0	NaN	NaN	0.784
4 ses	Test	-9.75	12.1	10.2	-66.4	67.7	NaN	NaN	0.785
5 naive	Test	-9.75	12.1	10.2	-66.4	67.7	NaN	NaN	0.785
6 arima2	Test	-11.1	13.6	11.4	-75.0	76.1	NaN	NaN	0.791

```
#plot
egyptian_forecasts |> autoplot(egyptian_export, level = NULL) +
  labs(title = "Forecasts and actual data for several models on the Egyptian
      exports data")
```


Forecasts and actual data for several models on the Egyptian exports data
Exports
Year
30
25
20
15
10
1960
1980
2000
.model
arima1
arima2
arima3
mean
naive
ses

- As seen, the *ARIMA* function will automatically find the best parameters. However, it is important to understand these parameters and how they influence the behaviour of the model.
- The constant c (together with d) has an effect on the long-term forecasts.
  - If $c = 0$ and $d = 0$ the long-term forecast will go to zero.
  - If $c = 0$ and $d = 1$ the long-term forecast will go to a non-zero constant.
  - If $c = 0$ and $d = 2$ the long-term forecast will follow a straight line
  - If $c \neq 0$ and $d = 0$ the long-term forecast will go to the mean of the data
  - If $c \neq 0$ and $d = 1$, the long-term forecast will follow a straight line
  - If $c \neq 0$ and $d = 2$, the long-term forecast will follow a quadratic trend (not recommended)

- d also has an effect on the prediction intervals: the larger the value of d the faster the prediction intervals grow.
- The value of p is important in case of cyclic data. For cyclic data $p \geq 2$, and some extra conditions are imposed on the parameters. For AR(2) this is, for example:

$$\phi_1^2 + 4 * \phi_2 < 0$$

Example I

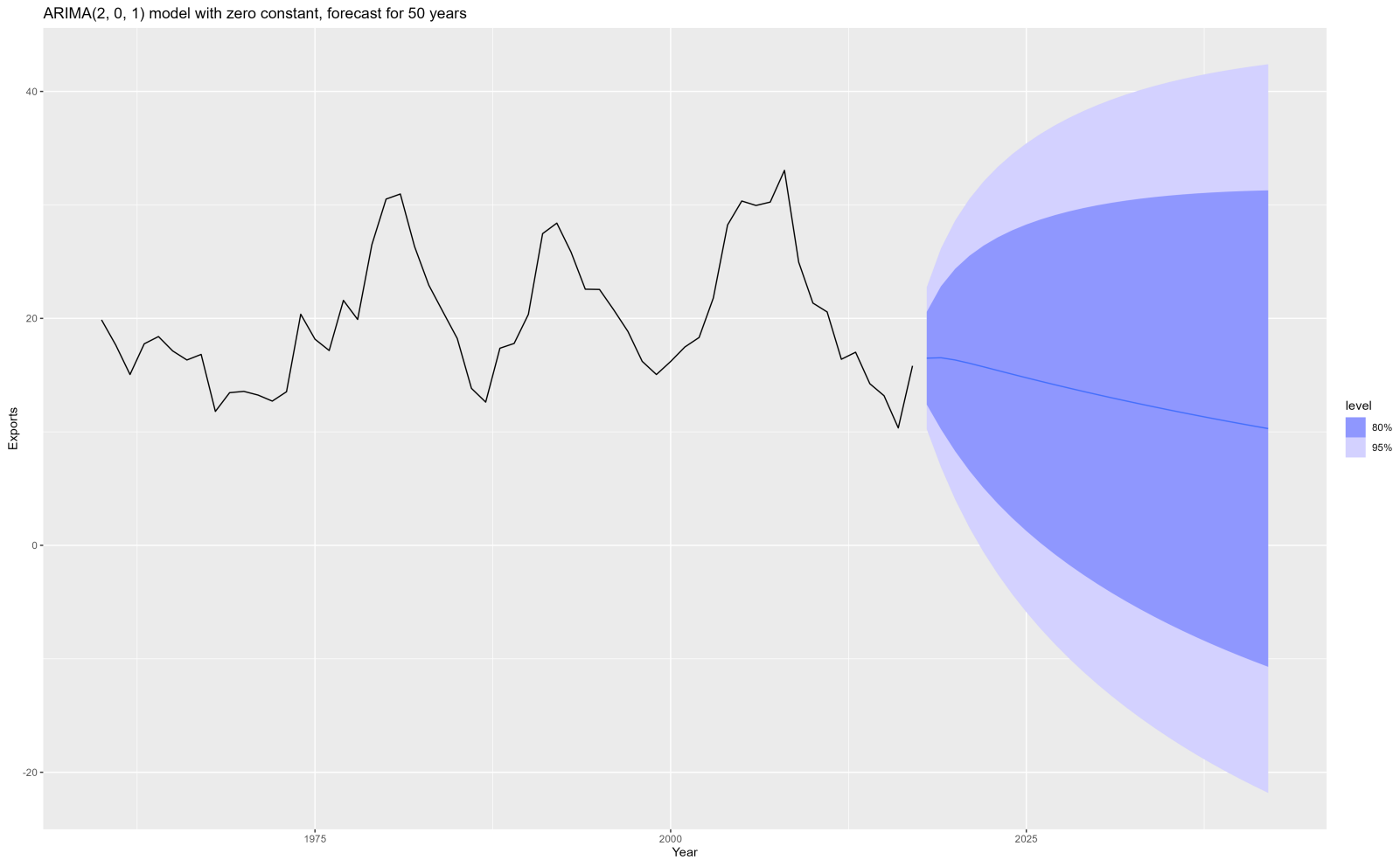
- Let's see a few examples about the effect of these parameter values: (+0 in an ARIMA model means we want the constant to be zero, while +1 means we want it to be non-zero - it will not be automatically 1):

```
egyptian_export |>
  model(m1 = ARIMA(Exports ~ pdq(2, 0, 1) + 0),
        m2 = ARIMA(Exports ~ pdq(2, 1, 1) + 0),
        m3 = ARIMA(Exports ~ pdq(2, 2, 1) + 0),
        m4 = ARIMA(Exports ~ pdq(1, 0, 1) + 1),
        m5 = ARIMA(Exports ~ pdq(4, 0, 1) + 1),
        m6 = ARIMA(Exports ~ pdq(3, 1, 1) + 1)) -> egyptian_models2

egyptian_models2 |> forecast(h = 50) -> egyptian_forecast2

#c = 0, d = 0, p = 2 => expect long term forecast going to 0
egyptian_forecast2 |>
  filter(.model == "m1") |>
  autoplot(egyptian_export) +
  labs(title = "ARIMA(2, 0, 1) model with zero constant, forecast for 50 years")
```

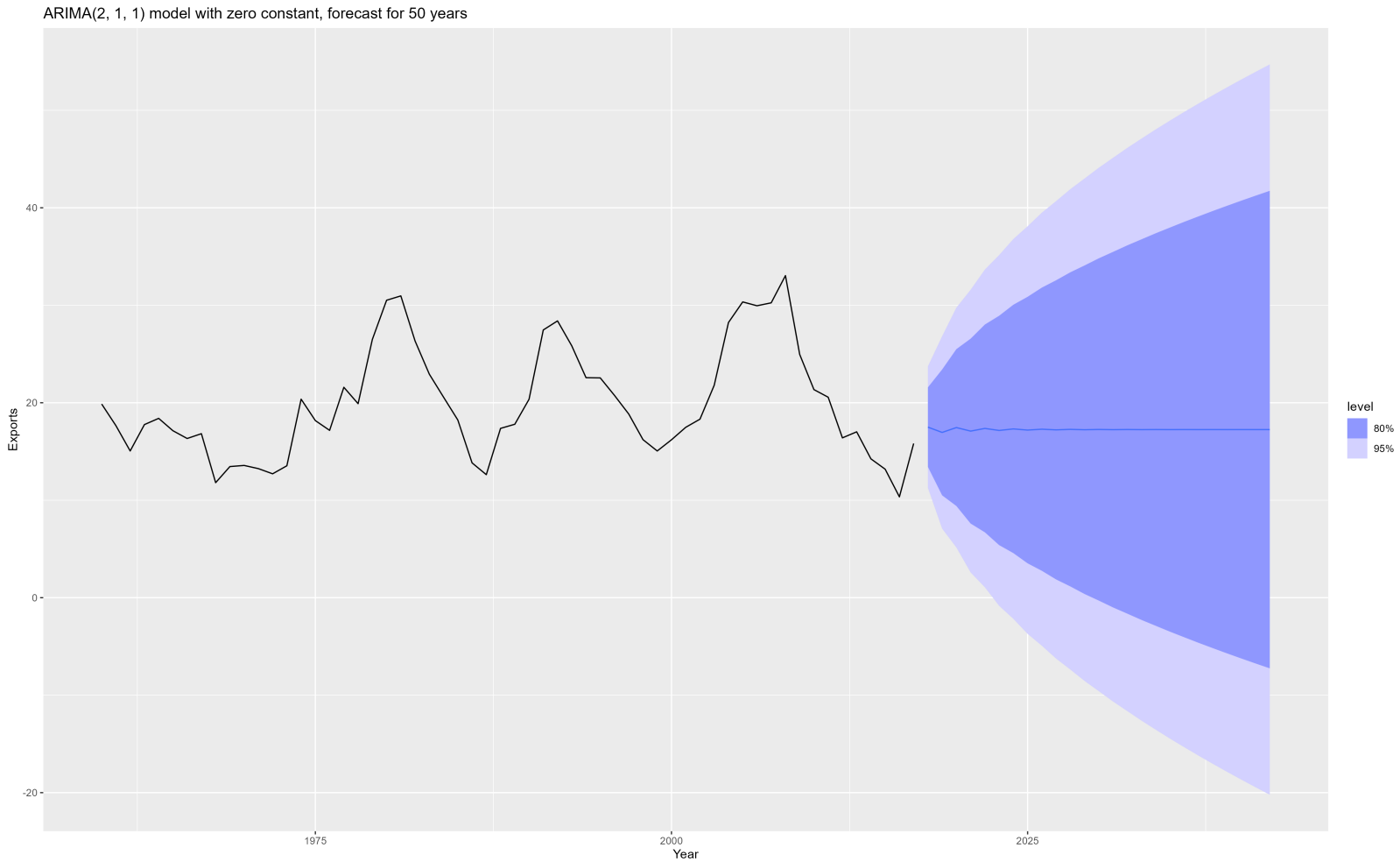
Example II


ARIMA(2, 0, 1) model with zero constant, forecast for 50 years
Exports
40
20
0
-20
1975
2000
2025
Year
level
80%
95%

Example III

```
#c = 0, d = 1, p = 2 => long-term forecast goes to a constant value
egyptian_forecast2 |>
  filter(.model == "m2") |>
  autoplot(egyptian_export) +
  labs(title = "ARIMA(2, 1, 1) model with zero constant, forecast for 50 years")
```

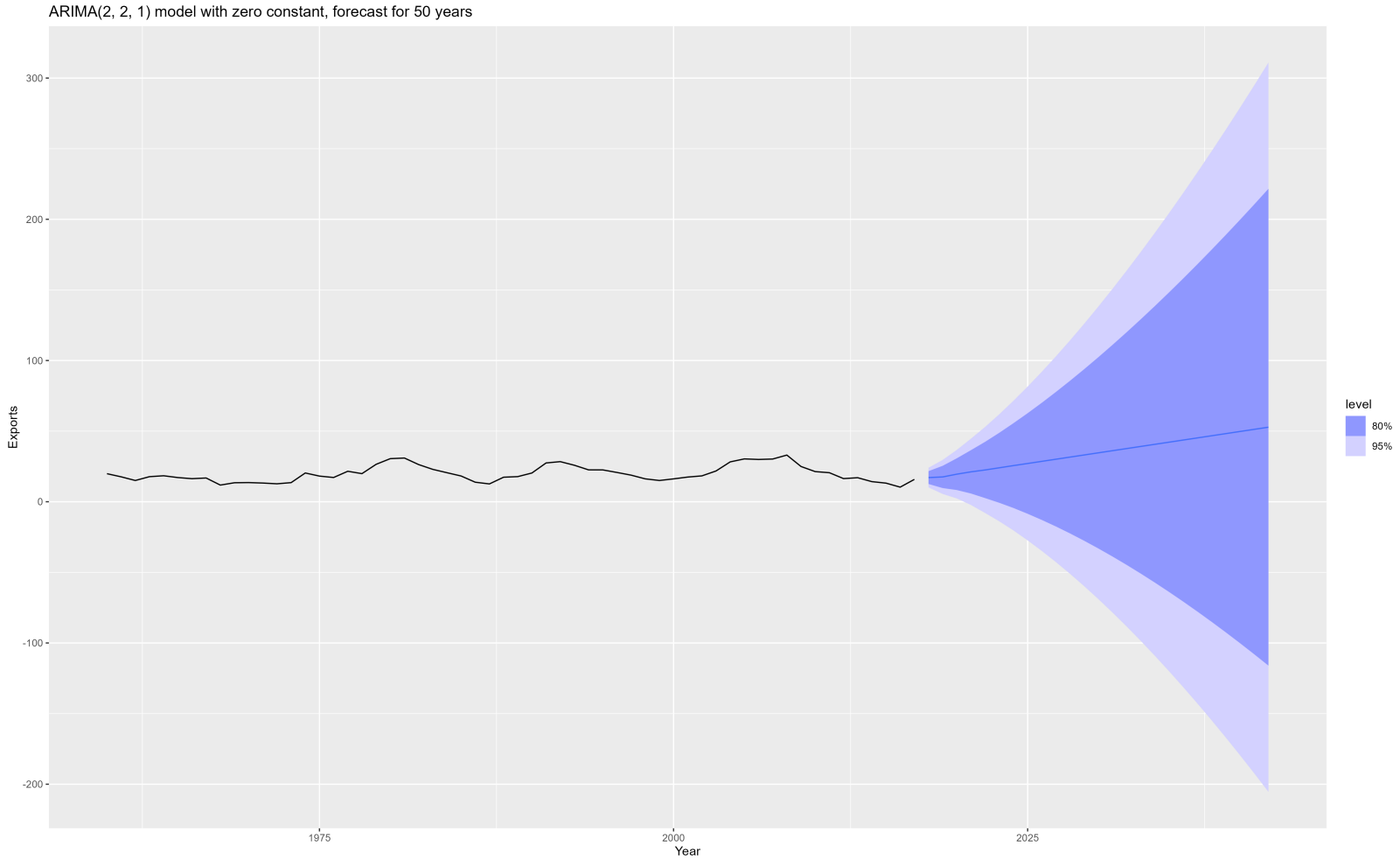
Example IV


ARIMA(2, 1, 1) model with zero constant, forecast for 50 years
Exports
40
20
0
-20
1975
2000
2025
Year
level
80%
95%

Example V

```
#c = 0, d = 2, p = 2 => expect rapidly growing prediction interval long-term
egyptian_forecast2 |>
  filter(.model == "m3") |>
  autoplot(egyptian_export) +
  labs(title = "ARIMA(2, 2, 1) model with zero constant, forecast for 50 years")
```

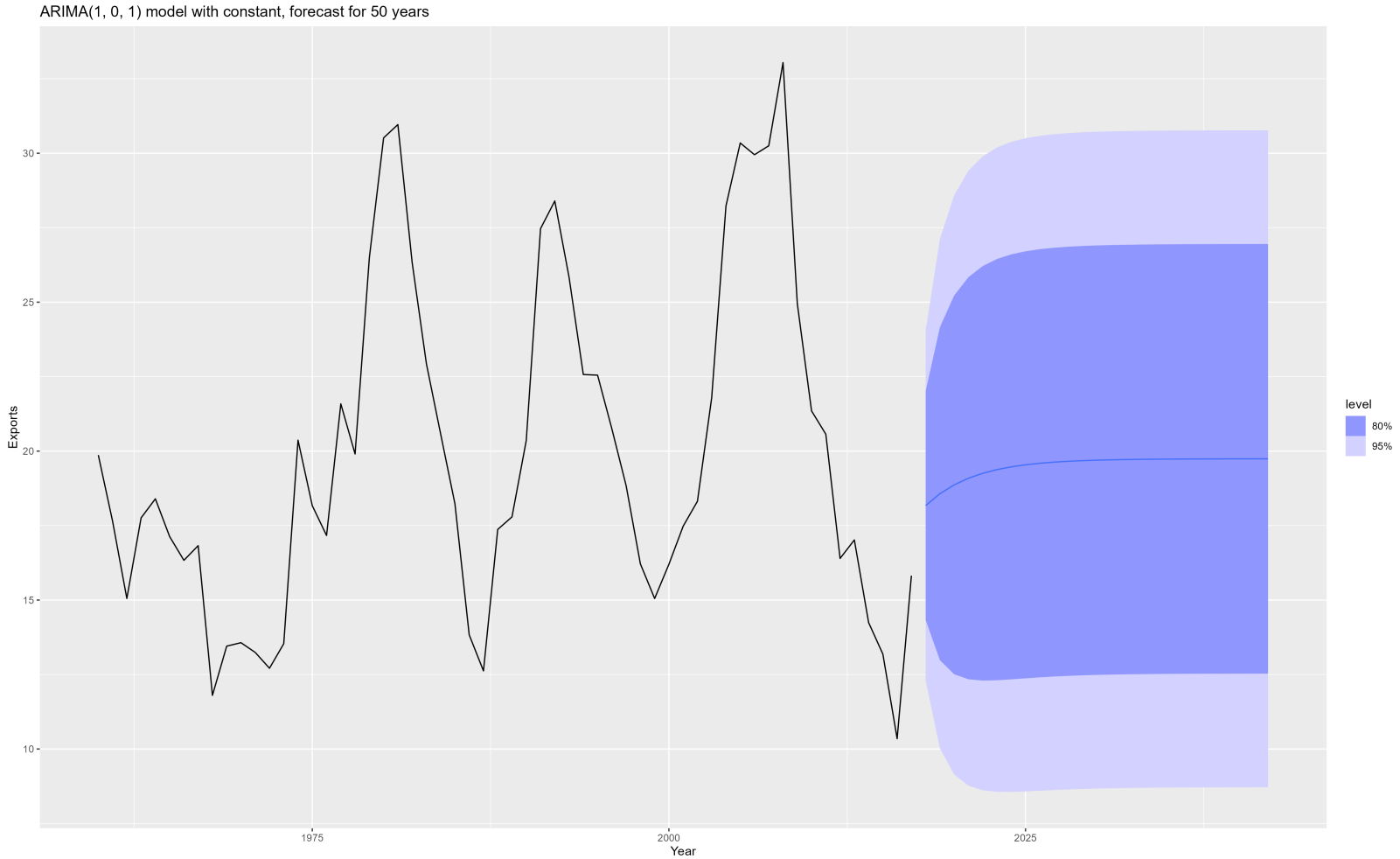
Example VI


ARIMA(2, 2, 1) model with zero constant, forecast for 50 years
Exports
300
200
100
0
-100
-200
1975
2000
2025
Year
level
80%
95%

Example VII

```
#c != 0, d = 0, p = 1 => long-term forecast will go to mean
egyptian_forecast2 |>
  filter(.model == "m4") |>
  autoplot(egyptian_export) +
  labs(title = "ARIMA(1, 0, 1) model with constant, forecast for 50 years")
mean(egyptian_export$Exports)
```

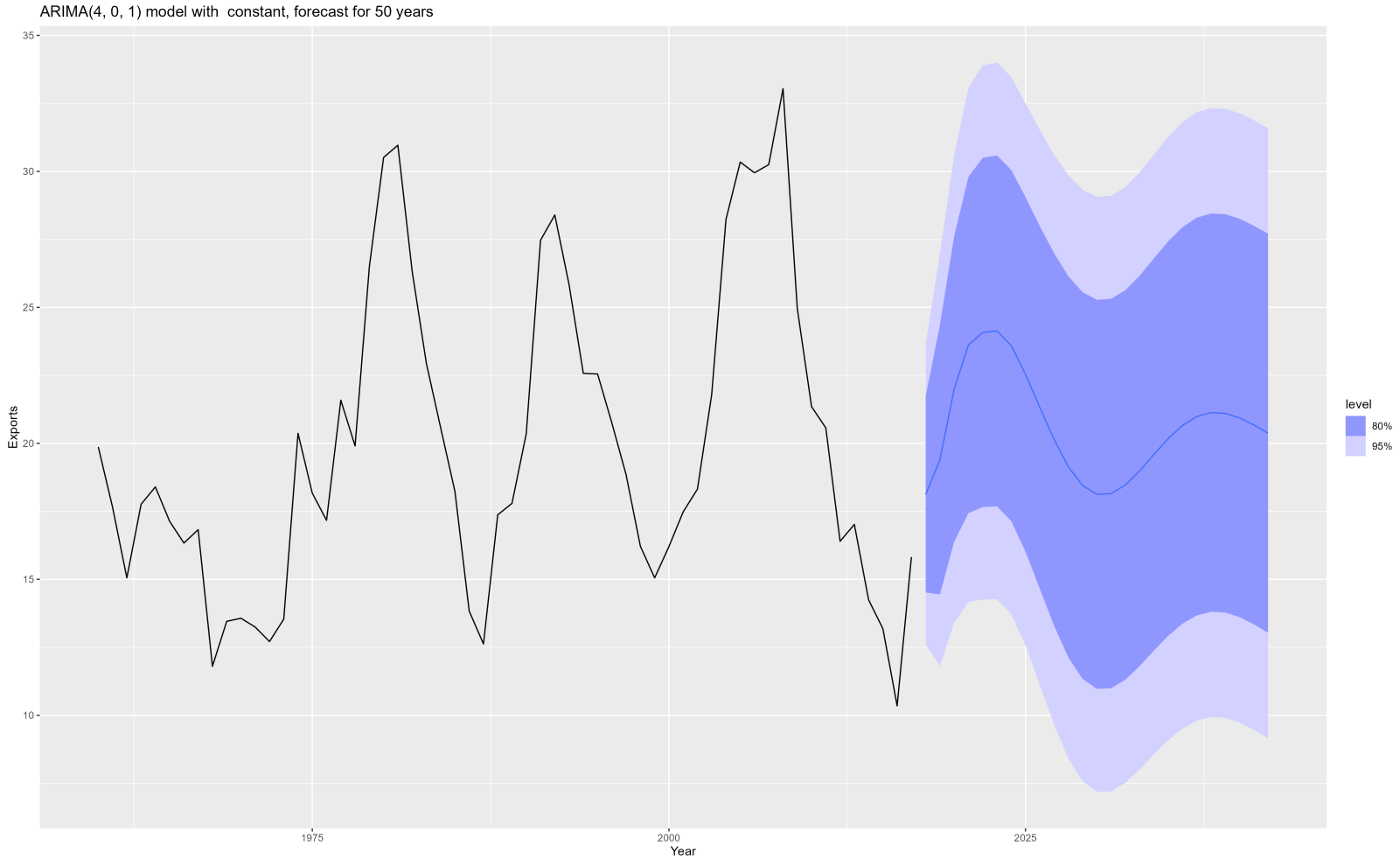
Example VIII



Example IX

```
#c != 0, d = 0, p = 4 => expect long-term forecast will go to mean
egyptian_forecast2 |>
  filter(.model == "m5") |>
  autoplot(egyptian_export) +
  labs(title = "ARIMA(4, 0, 1) model with  constant, forecast for 50 years")
```

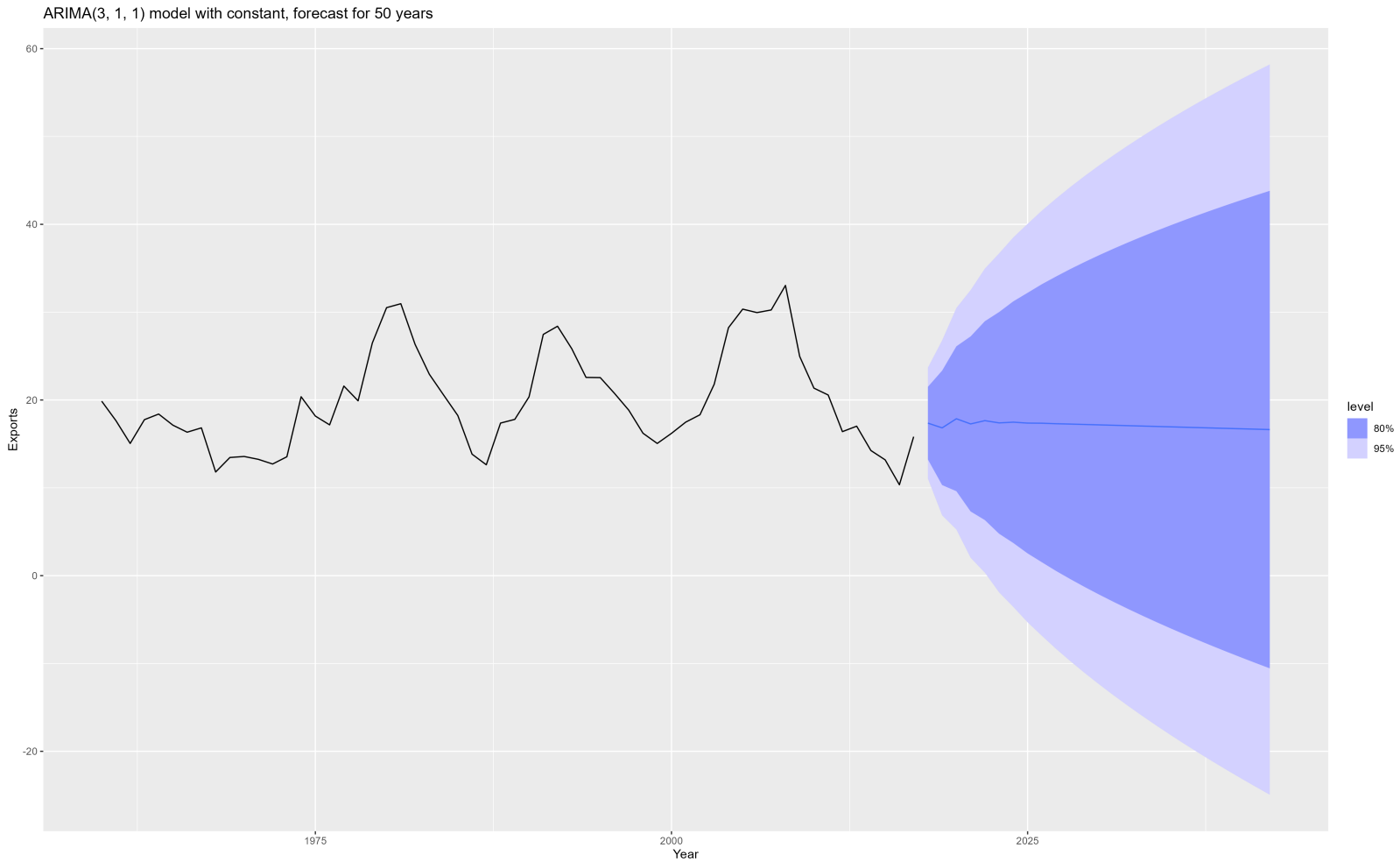
Example X


ARIMA(4, 0, 1) model with constant, forecast for 50 years
Exports
35
30
25
20
15
10
1975
2000
2025
Year
level
80%
95%

Example XI

```
#c != 0, d = 1, p = 3 => expect growing prediction interval, cycle, long-term
     forecast will be a straight line
egyptian_forecast2 |>
  filter(.model == "m6") |>
  autoplot(egyptian_export) +
  labs(title = "ARIMA(3, 1, 1) model with constant, forecast for 50 years")
```

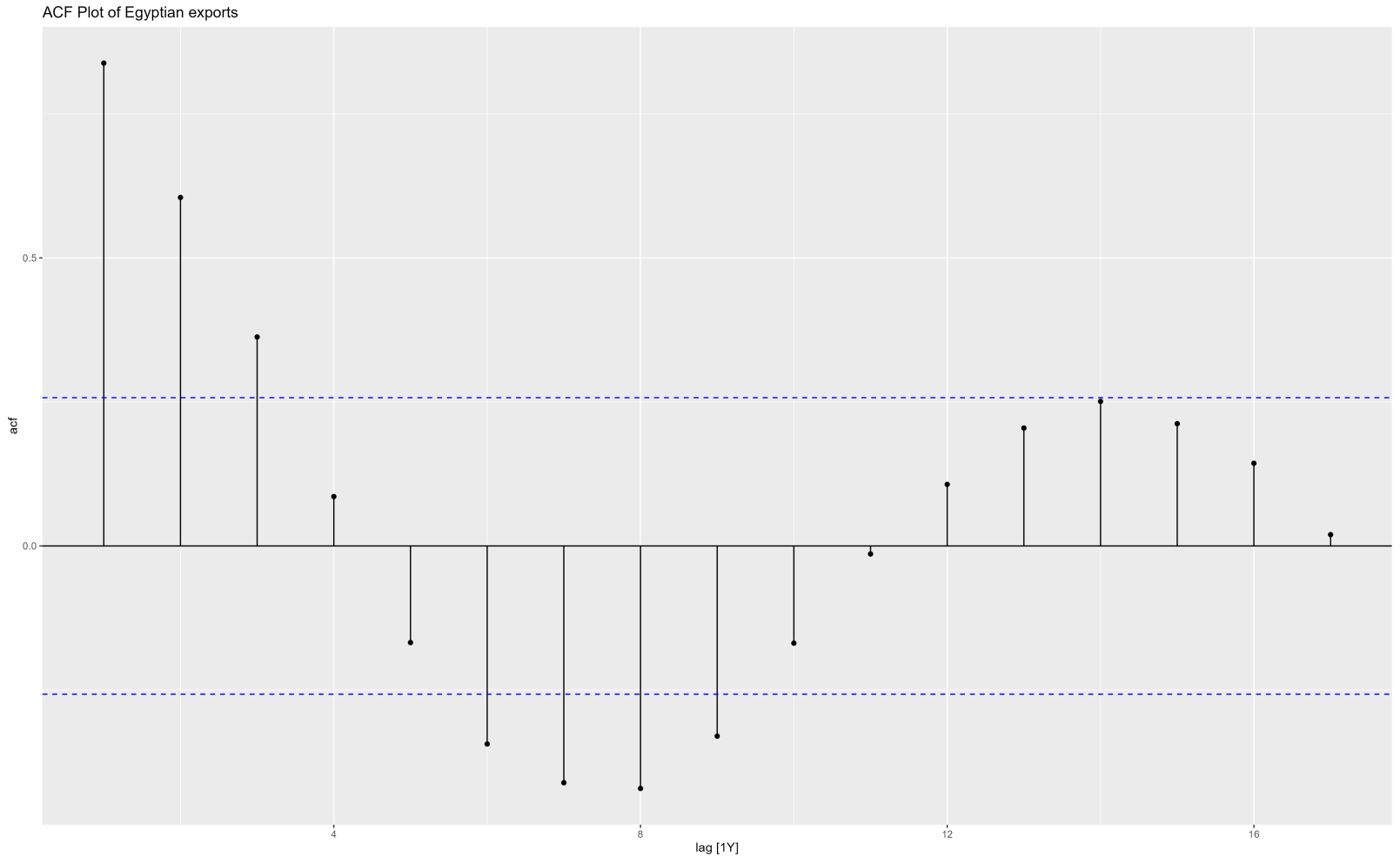
Example XII


ARIMA(3, 1, 1) model with constant, forecast for 50 years
60
40
20
0
-20
Exports
1975
2000
2025
Year
level
80%
95%

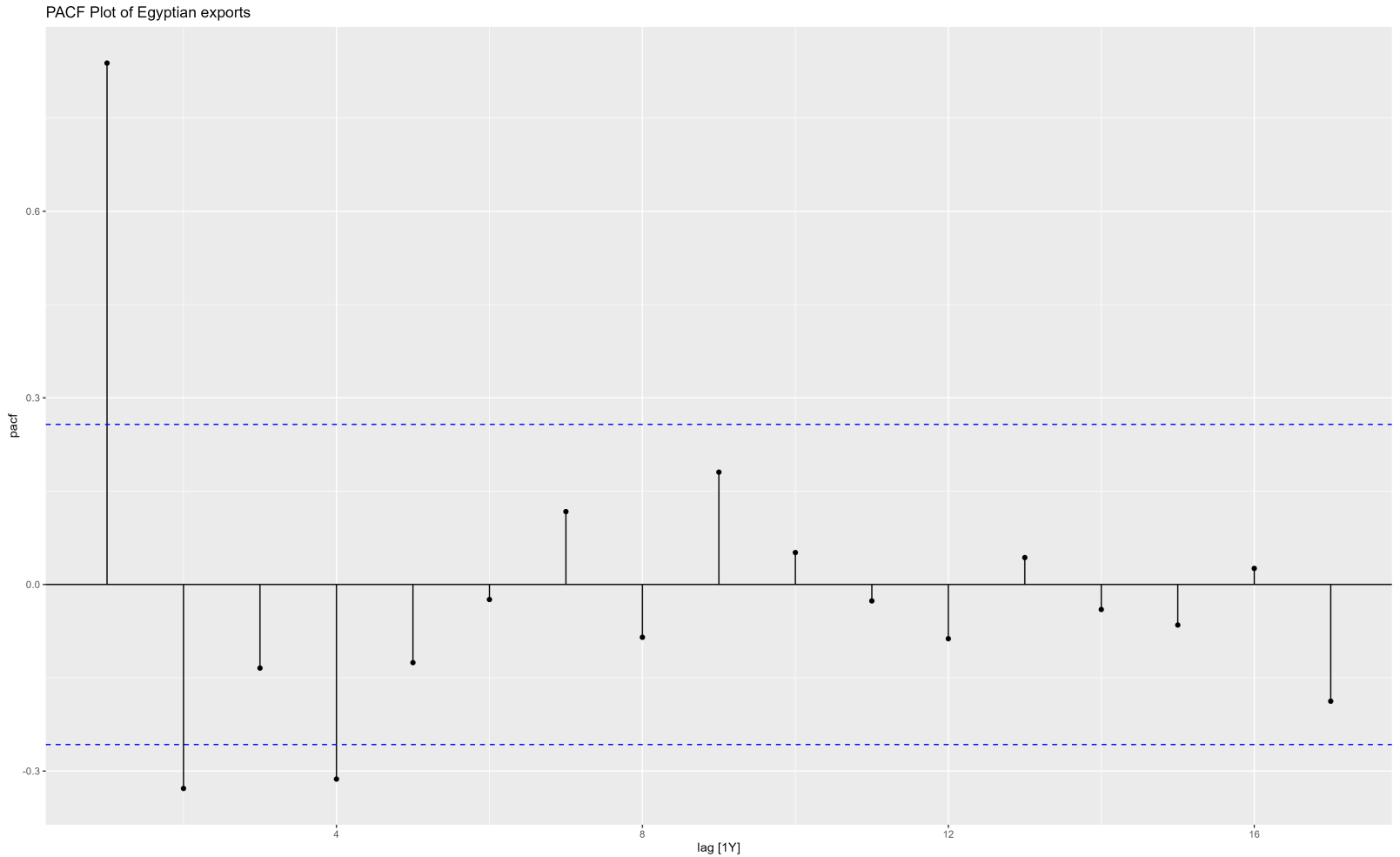
- For determining the optimal values for p and q, the ACF and the related PACF plots might be helpful.
- The ACF plot shows the autocorrelation in a time series, i.e., the correlation between y_t and y_{t-k} for different values of k.
- Consider a time series where y_t and y_{t-1} are correlated. This means that y_{t-1} and y_{t-2} will also be correlated. But this means that y_t and y_{t-2} might also be correlated, not because y_{t-2} holds information useful for forecasting y_t, but simply because they are both connected to y_{t-1}.
- To overcome this problem, the idea of *partial autocorrelation* was introduced. Partial autocorrelation measures the relation between y_t and y_{t-k} after removing the effect of the previous lags (lags $1, 2, 3, ..., k-1$). More specifically, α_k the value of the kth partial autocorrelation is equal to the estimate of ϕ_k in an AR(k) model.

- We can get the partial autocorrelation in *R* with the *PACF* function (and plot it with *autoplot*).
- Similar to the *ACF* plot, the *PACF* plot also has a critical value, at the value $\pm 1.96 * \sqrt{T}$
- We can display the time plot, the ACF and PACF plots in *R* using the *gg_tsdisplay* function with the *plot_type = "partial"* parameter.

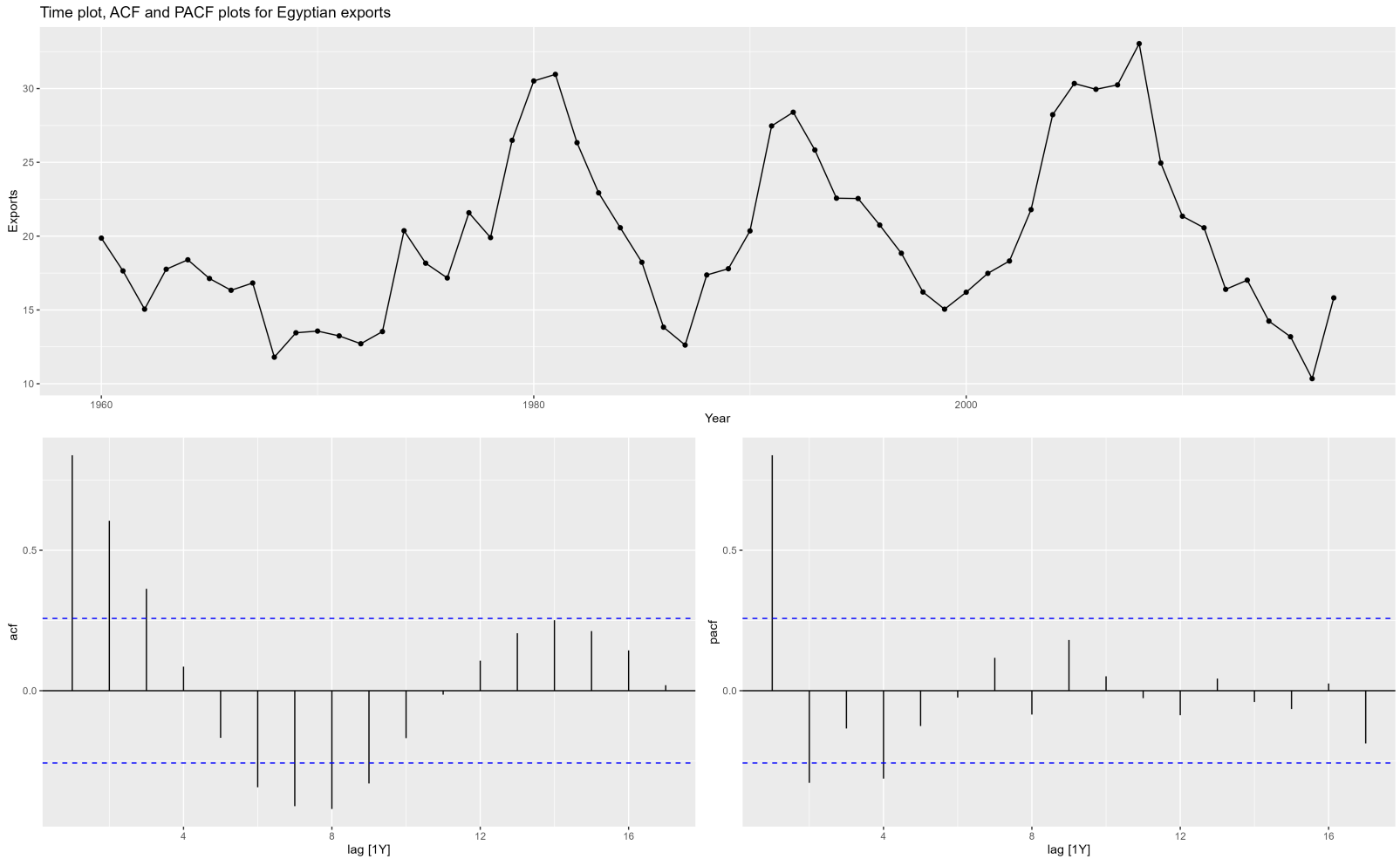
```
egyptian_export |>
  ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF Plot of Egyptian exports")
```


ACF Plot of Egyptian exports
acf
0.5
0.0
4
8
12
16
lag [1Y]

```
egyptian_export |>
  PACF() |>
  autoplot() +
  geom_point() +
  labs(title = "PACF Plot of Egyptian exports")
```


PACF Plot of Egyptian exports
pacf
0.6
0.3
0.0
-0.3
4
8
12
16
lag [1Y]

```
egyptian_export |>
  gg_tsdisplay(plot_type = "partial") +
  labs(title = "Time plot, ACF and PACF plots for Egyptian exports")
```


Time plot, ACF and PACF plots for Egyptian exports
Exports
Year
1960
1980
2000
10
15
20
25
30
acf
pacf
lag [1Y]
0.0
0.5
4
8
12
16

- The ACF and PACF plots can help us if the value of p or q is zero, i.e. we have either an ARIMA(p, d, 0) or an ARIMA(0, d, q) model.
- We have an ARIMA(p, d, 0) model if, for the differenced data:
  - The ACF plot is exponentially decreasing or sinusoidal
  - The last significant autocorrelation in the PACF plot is at lag p.
- We have an ARIMA(0, d, q) model if, for the differenced data:
  - The PACF is exponentially decaying or sinusiodal
  - The last significant autocorrelation in the ACF plot is at lag q

- From the plots of the Egyptian exports we can see that:
  - ACF plot is sinusoidal
  - Last significant autocorrelation on PACF is at lag 4.
- This suggests an ARIMA(4, 0, 0) model. Let's see it compared to the ARIMA(2, 0, 1) returned by the ARIMA function.

```
egyptian_export |>
  model(m1 = ARIMA(Exports ~ pdq(4, 0, 0)),
        m2 = ARIMA(Exports ~ pdq(2, 0 ,1))) -> egyptian_model3

egyptian_model3 |> select(m1) |> report()
```

```
Series: Exports
Model: ARIMA(4,0,0) w/ mean

Coefficients:
         ar1      ar2     ar3      ar4  constant
      0.9861  -0.1715  0.1807  -0.3283    6.6922
s.e.  0.1247   0.1865  0.1865   0.1273    0.3562

sigma^2 estimated as 7.885:  log likelihood=-140.53
AIC=293.05   AICc=294.7   BIC=305.41
```

```
egyptian_model4 |> select(m2) |> report()
```

```
Series: Exports
Model: ARIMA(2,0,1) w/ mean

Coefficients:
         ar1      ar2      ma1  constant
      1.6764  -0.8034  -0.6896    2.5623
s.e.  0.1111   0.0928   0.1492    0.1161

sigma^2 estimated as 8.046:  log likelihood=-141.57
AIC=293.13   AICc=294.29   BIC=303.43
```

- The AICc value of ARIMA(4,0,0) is 294.7 while for the optimal ARIMA(2,0,1) it is 294.29, so we can see that the ARIMA(4, 0,0) model is close to the optimal one.

How does the ARIMA function work?

- The ARIMA function from *R* tries to find the best values for p, d and q by minimizing AICc and the *Maximum Likelihood Estimation (MLE)* (the value which maximizes the probability of obtaining the data that we have observed).
- The algorithm, called the *Hyndman-Khandakar* algorithm, works in the following way:
  1. Find the value of d (0, 1 or 2), by using the KPSS test.
  2. Difference the data d times.
  3. Find the value of p and q to minimize the value of AICc.
     1. Fit four initial models: ARIMA(0, d, 0), ARIMA(2, d, 2), ARIMA(1, d, 0) and ARIMA(0, d, 1).
     2. If $d < 2$ include the model ARIMA(0, d, 0) without a constant as well.
     3. The best (according to AICc) model is set to be the *current model*.
     4. Variations of the current model are considered: modify p and/or q with ± 1 and include/exclude the constant from the current model.
     5. Keep the best model and repeat the steps until no model with better AICc is found.

- The previously described algorithm is a *stepwise* algorithm, and there is no guarantee that it will find the model with the best AICc.
- The behaviour of the algorithm can be modified using some parameters for the ARIMA function:
  - *greedy* - if set to true, the algorithm will switch to a new current model once it finds a neighbour with a better AICc (and will not search all neighbours).
  - *approximation* - if set to true some approximations will be used to seep up the search.
  - *stepwise* - if set to FALSE, instead of the previously described stepwise model a much larger set of models is searched.

Steps of modeling non-seasonal data with ARIMA

1. Plot the data, look for patterns and unusual observations
2. If the data does not have constant variance, use a transformation (most likely Box-Cox with a well-selected λ)
3. If the data is non-stationary, difference it, until is becomes stationary. If not sure, use statistical tests (for example KPSS).
4. Look at ACF and PACF, to see if ARIMA(p, d, 0) or ARIMA(0, d, q) would be suitable.
5. Try the chosen model(s), use AICc to search for better models.
6. Check the ACF plot of the residuals to see if they are white noise. Do a statistical test as well (for example: *ljung_box)*. If the residuals are not white noise, try a modified model.
7. When the residuals look like white noise, you can use the model for forecasting.

- The Hyndman-Khandakar algorithm only covers steps 3-5, so even if it is used, the rest of the steps need to be performed.
- Regarding the statistical test of the residuals, up until now we have used the *ljung_box* test with default parameters, we have only specified the number of lags to consider for seasonal data. However, it has another important parameter, the *degrees of freedom, dof*. We set this parameter to be $p + q$.

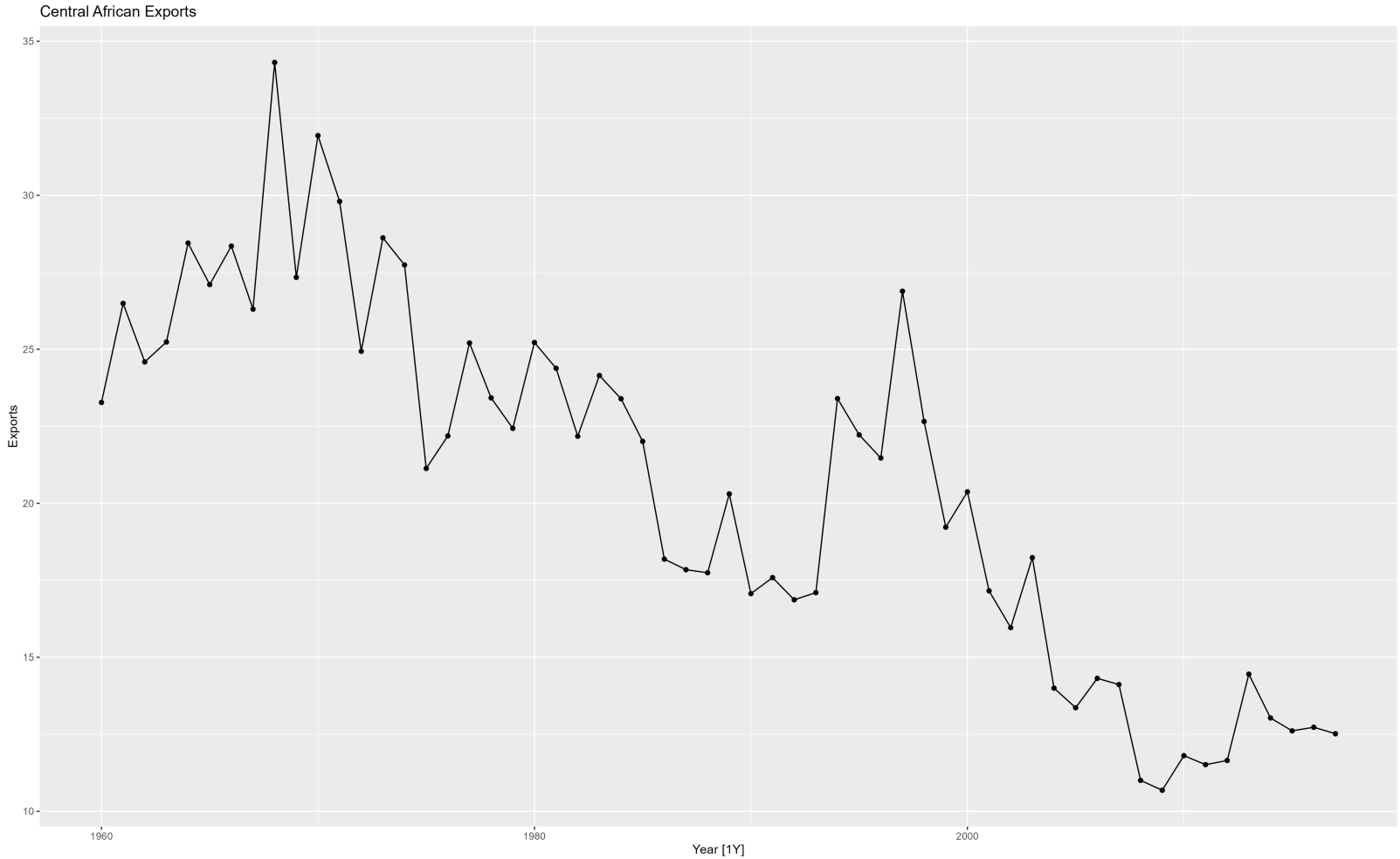
Example I

- Let's see the entire ARIMA process on the Central African Republic exports, which comes from the same data set as the Egyptian exports, *global_economy*.
- Let's first plot the data

```
african_exports <- global_economy |>
  filter(Code == "CAF") |>
  select(Exports)

african_exports |>
  autoplot() +
  geom_point() +
  labs(title = "Central African Exports")
```

Example II


Central African Exports
Exports
35
30
25
20
15
10
1960
1980
2000
Year [1Y]

Example III

- We have an overall decreasing trend, so data is non-stationary.
- Variance does not really seem to change, so no transformation is needed.
- We need to difference the data. To see how many times, we can use the *unitroot_ndiff* function.

```
african_exports |> features(Exports, unitroot_ndiffs)
```

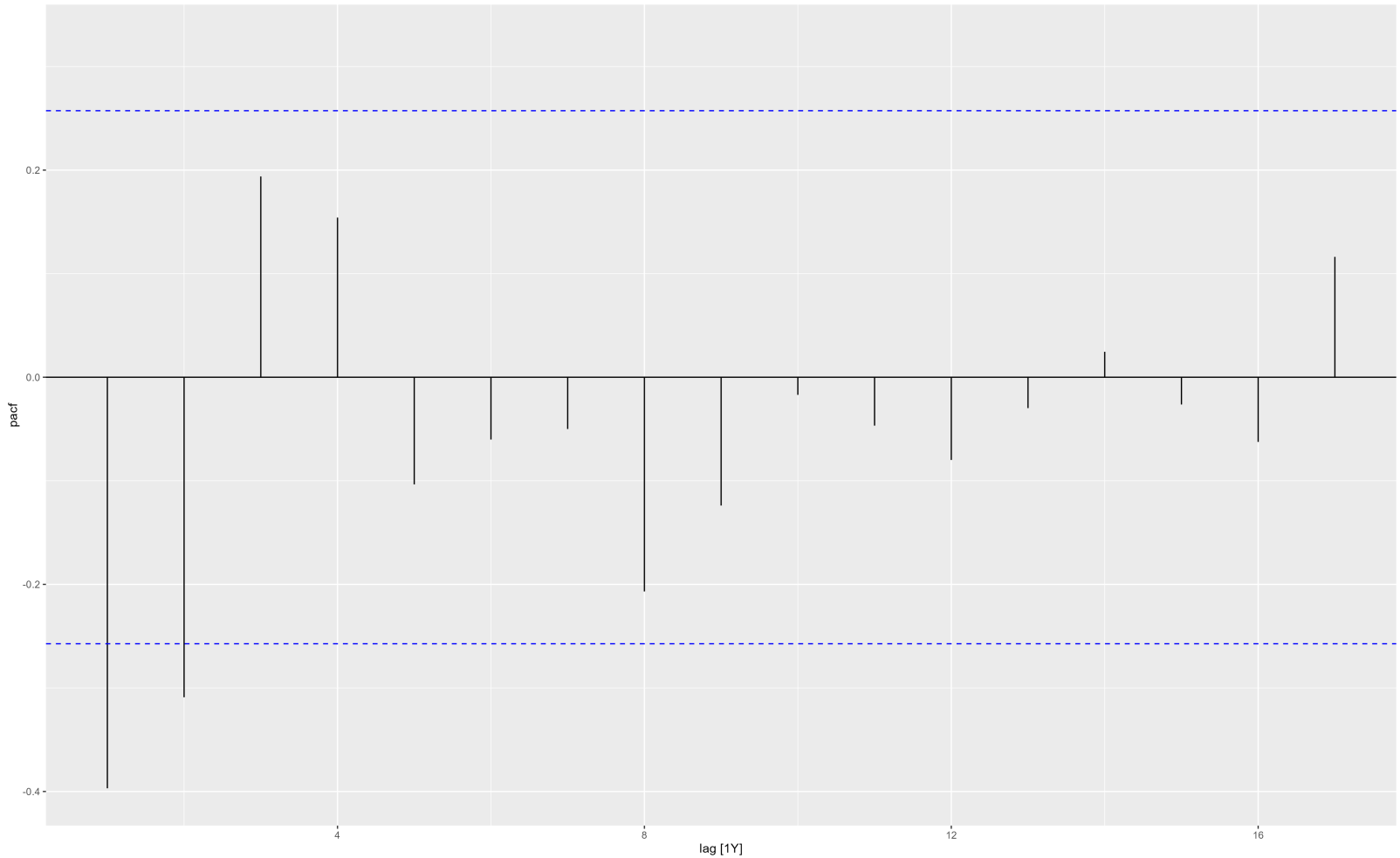
```
  ndiffs
   <int>
1      1
```

- Let's difference the data and visualize its ACF and PACF (we can use the *gg_tsdisplay* function).

Example IV

```
african_exports |>
  mutate(ExpDiff = difference(Exports)) |>
  gg_tsdisplay(ExpDiff, plot_type = "partial")
```

Example V



Example VI

- The differenced data seems stationary.
- However, neither ACF nor PACF is exponentially decaying or sinusoidal, so ARIMA(0, d, q) or ARIMA(p, d, 0) is not an optimal model. Nevertheless, looking at the last significant correlations, we can experiment with an ARIMA(0, 1, 3) and an ARIMA(2, 1, 0) model. Let's add two automatic parameter searching procedures as well: the stepwise one and the one which searches more models.

```
african_models <- african_exports |>
  model(m1 = ARIMA(Exports ~ pdq(0, 1, 3)),
        m2= ARIMA(Exports ~ pdq(2, 1, 0)),
        m3 = ARIMA(Exports),
        m4 = ARIMA(Exports, stepwise = FALSE ))
african_models

african_models |>
  glance() |>
  view()
```

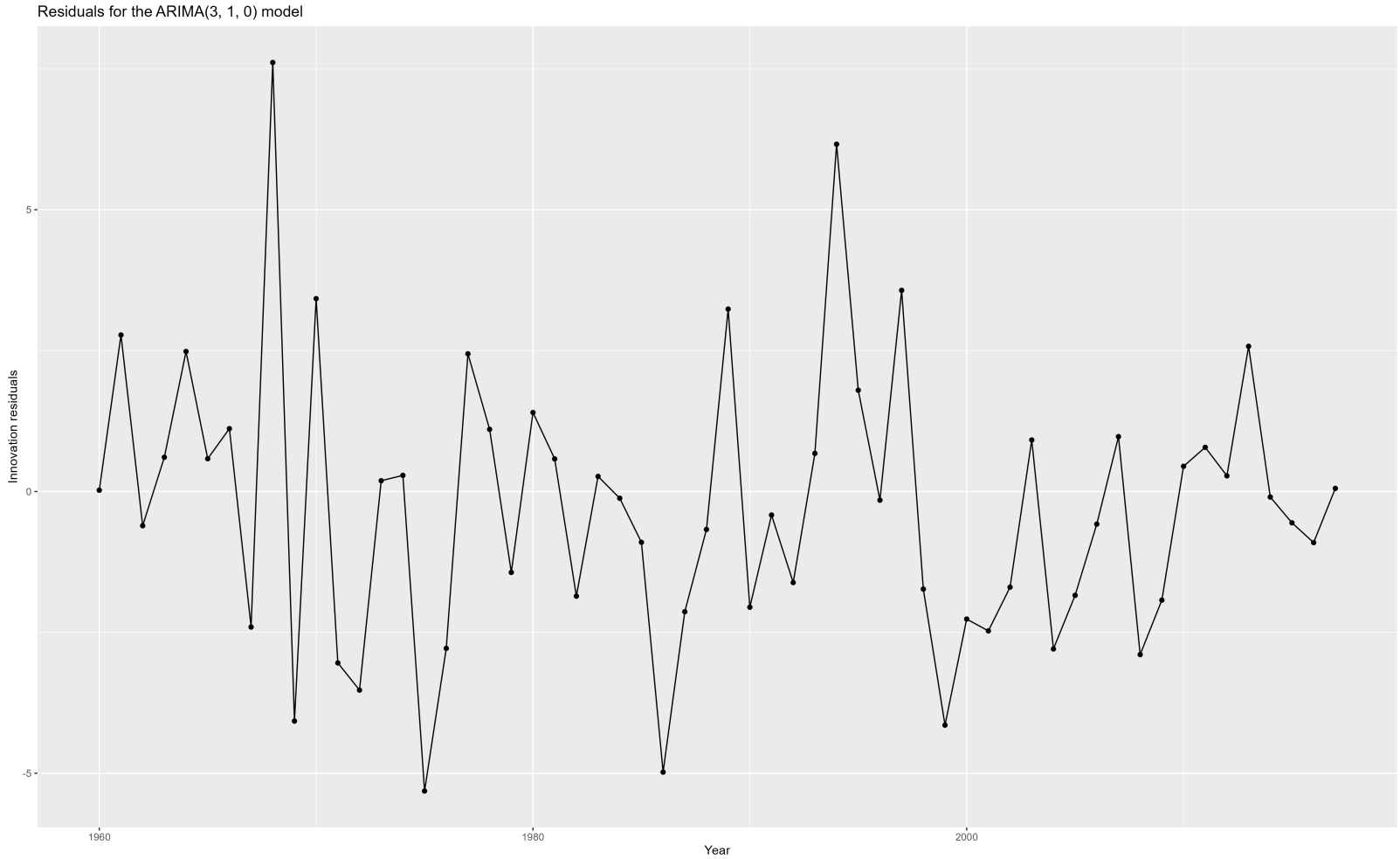
Example VII

- The best AICc value (274.77) is achieved by the $m4$ model (an ARIMA(3,1,0)), followed by $m2$ (274.99) and $m1$ (275.02). The $m3$ model, which is ARIMA(2,1,2) has AICc of 275.37.
- Let's continue with the ARIMA(3, 1, 0) and look at its residuals.

```
selected <- african_models |> select(m4)

selected |>
  gg_tsresiduals() +
  labs(title = "Residuals for the ARIMA(3, 1, 0) model")
```

Example VIII



Example IX

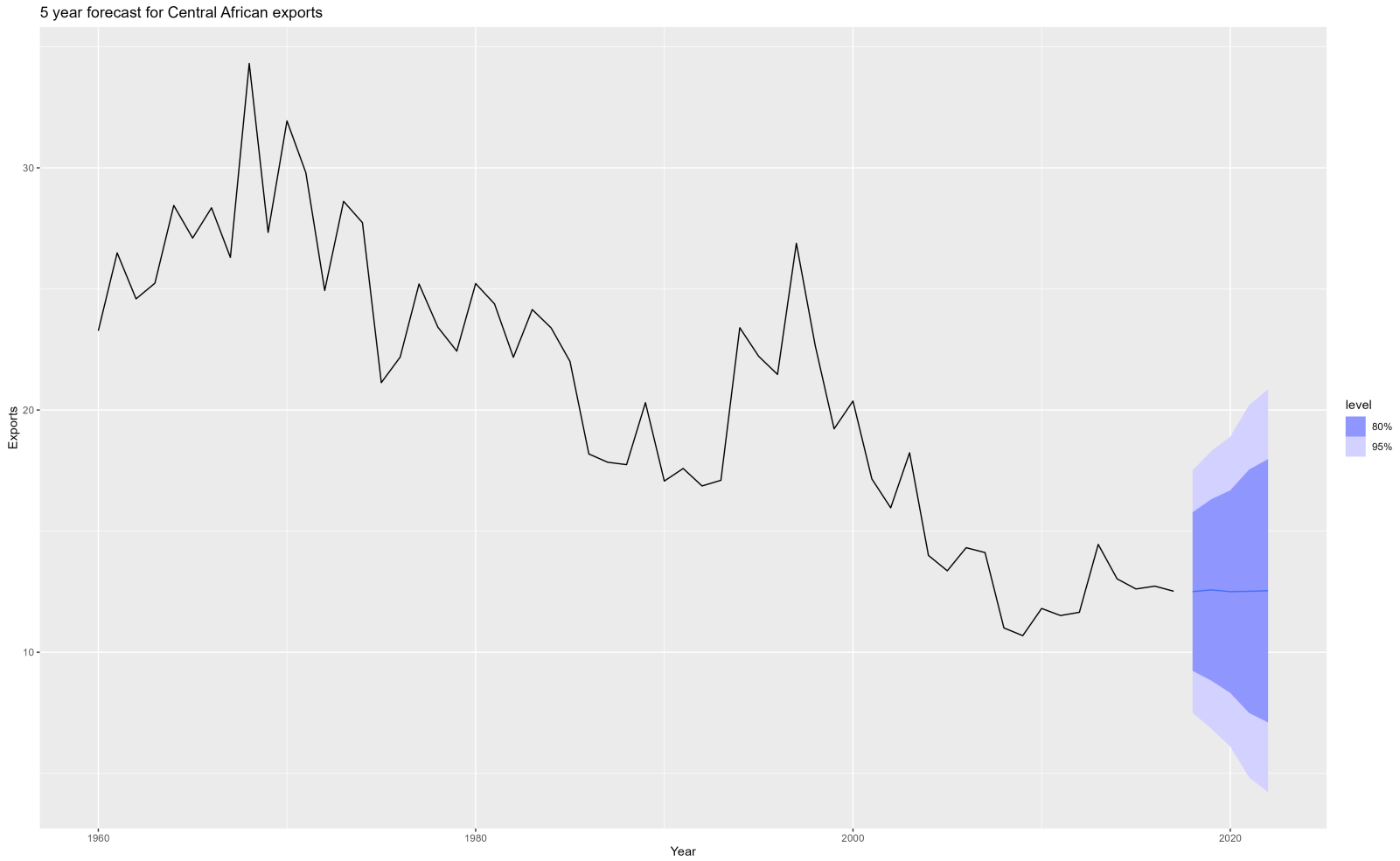
- Residuals look fine, but let's do a ljung_box test as well. Since we have an ARIMA(3, 1, 0) model, we set the *dof* parameter to 3.

```
selected |>
  augment()  |>
  features(.innov, ljung_box, lag = 10, dof = 3)
```

```
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 m4        5.75     0.569
```

- Large p value shows that the residuals are indeed white noise. We have our best model. Let's forecast.

```
selected |>
  forecast(h = 5) |>
  autoplot(african_exports) +
  labs(title = "5 year forecast for Central African exports")
```



- The forecast is quite flat, similar to what we would get for a

How are forecasts computed? I

- The *forecast* function provides us with the forecasts of the model, but how are they computed?
- Let's take as example an ARIMA(3, 1, 1) model and discuss the steps for forecasting for moment T+1. In backshift notation, it can be written as:

$$(1 - \hat{\phi}_1 * B - \hat{\phi}_2 * B^2 - \hat{\phi}_3 * B^3)(1 - B) * y_t = (1 + \hat{\theta}_1 * B) * \epsilon_t$$

- STEP 1: expand the ARIMA equation and rearrange terms, so that you have y_t on one side (left) and all the other terms on the other (right)

$$(1-(1+\hat{\phi}_1)*B+(\hat{\phi}_1-\hat{\phi}_2)*B^2+(\hat{\phi}_2-\hat{\phi}_3)*B^3+\hat{\phi}_3*B^4)*y_t =$$

$$= (1 + \hat{\theta}_1 * B) * \epsilon_t$$

How are forecasts computed? II

- Let's do the multiplication with y_t and apply the backshift operator where necessary.

$$y_t - (1+\hat{\phi}_1) * y_{t-1} + (\hat{\phi}_1 - \hat{\phi}_2) * y_{t-2} + (\hat{\phi}_2 - \hat{\phi}_3) * y_{t-3} + \hat{\phi}_3 * y_{t-4} =$$

$$= \epsilon_t + \hat{\theta}_1 * \epsilon_{t-1}$$

- Finally, leave just y_t on the left hand side

$$y_t = (1+\hat{\phi}_1) * y_{t-1} - (\hat{\phi}_1 - \hat{\phi}_2) * y_{t-2} - (\hat{\phi}_2 - \hat{\phi}_3) * y_{t-3} - \hat{\phi}_3 * y_{t-4} +$$

$$+ \epsilon_t + \hat{\theta}_1 * \epsilon_{t-1}$$

- STEP 2: Replace t with $T+1$.

$$y_{T+1} = (1+\hat{\phi}_1) * y_T - (\hat{\phi}_1 - \hat{\phi}_2) * y_{T-1} - (\hat{\phi}_2 - \hat{\phi}_3) * y_{T-2} - \hat{\phi}_3 * y_{T-3} +$$

$$+ \epsilon_{T+1} + \hat{\theta}_1 * \epsilon_T$$

How are forecasts computed? III

- STEP 3: Assuming we have observations up until moment T, the unknown values from the previous equation are ϵ_{T+1} and ϵ_T. We will consider ϵ_{T+1} to be 0 and we will replace ϵ_T with the last observed residual, e_t. Thus, the equation becomes:

$$\hat{y}_{T+1|T} =$$

$$= (1+\hat{\phi}_1)*y_T-(\hat{\phi}_1-\hat{\phi}_2)*y_{T-1}-(\hat{\phi}_2-\hat{\phi}_3)*y_{T-2}-\hat{\phi}_3*y_{T-3}+\hat{\theta}_1*e_T$$

- If we want to forecast the observation at moment $T+2$, we go back to step 2, and replace t with $T+2$ (instead of $T+1$). We will get:

$$\hat{y}_{T+2|T} = (1+\hat{\phi}_1)*y_{T+1}-(\hat{\phi}_1-\hat{\phi}_2)*y_T-(\hat{\phi}_2-\hat{\phi}_3)*y_{T-1}-\hat{\phi}_3*y_{T-2}+$$

$$+\epsilon_{T+2}+\hat{\theta}_1*\epsilon_{T+1}$$

How are forecasts computed? IV

- We do not know the value of y_{T+1} so we will replace it with the previous forecast, $\hat{y}_{T+1|T}$ and the values of ϵ_{T+2} and ϵ_{T+1}, which will be replaced by 0. So we will get:

$$\hat{y}_{T+2|T} = (1+\hat{\phi}_1)*\hat{y}_{T+1|T} - (\hat{\phi}_1 - \hat{\phi}_2)*y_T - (\hat{\phi}_2 - \hat{\phi}_3)*y_{T-1} - \hat{\phi}_3*y_{T-2}$$

- This process can be continued for any value h into the future.

Seasonal ARIMA models I

- Up until now, we have talked about ARIMA modeling of non-seasonal time series. However, ARIMA can model seasonal time series as well.
- In case of a seasonal model we add extra terms to the regular ARIMA model to deal with with the seasonality: ARIMA(p, d, q) becomes $ARIMA(p, d, q)(P, D, Q)_m$, where m is the seasonal period.
- The seasonal part contains terms similar to the non-seasonal one, but the differencing is seasonal.
- For example, the backshift notation of an $ARIMA(1, 1, 1)(1, 1, 1)_4$ model, which has no constants, is:

$$(1-\phi_1 * B)(1-\Phi_1 * B^4)(1-B)(1-B^4)y_t = (1+\theta_1 * B)(1+\Theta_1 * B^4)\epsilon_t$$

Seasonal ARIMA models II

- We have discussed that the ACF and PACF plots can help determine the value of p pr q in some cases. Similarly, we can look at the autocorrelations/partial autocorrelations for seasonal lags to determine, in some cases, the values of P or Q.
- For example, an $ARIMA(0,0,0)(0,0,1)_{12}$ will have:
  - A large spike in the ACF at lag 12 (and no other significant spikes)
  - Exponentially decaying spikes on the PACF at seasonal lags (12, 24, 36, etc.)
- Similarly, an $ARIMA(0,0,0)(1,0,0)_{12}$ model will have:
  - A large spike in the PACF at lag 12 (and no other significant spikes)
  - Exponentially decaying spikes on the ACF plot at seasonal lags(12, 4, 36, etc.)

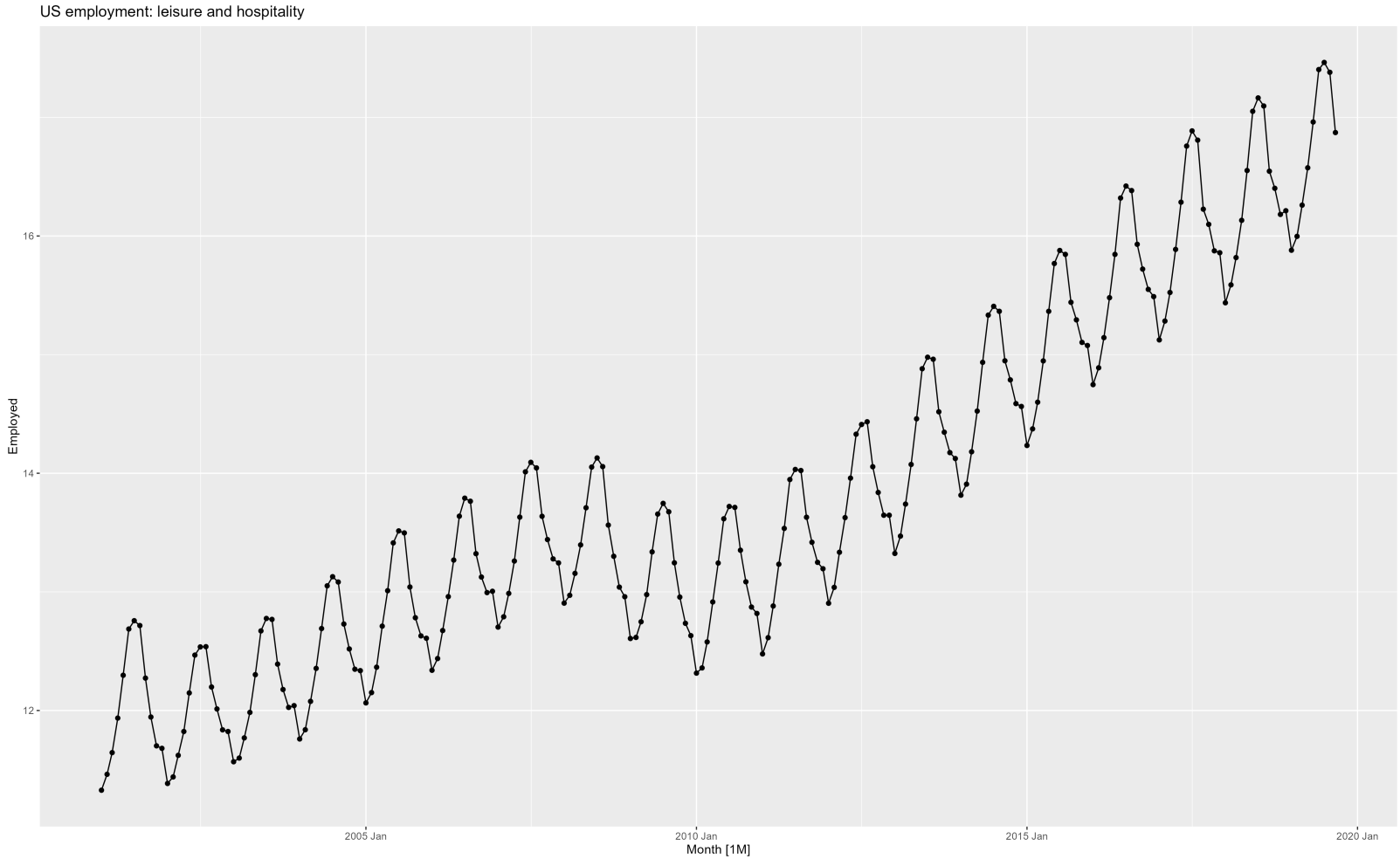
Example I

- Let's see the process through some examples. First we will consider the US employment data for the leisure and hospitality jobs, between January 2001 and September 2019.

```
leisure <- us_employment |>
  filter(Title == "Leisure and Hospitality", year(Month) > 2000) |>
  mutate(Employed = Employed / 1000) |>  #to work with smaller values
  select(Employed, Month)

leisure |> autoplot() +
  geom_point() +
  labs(title = "US employment: leisure and hospitality")
```

Example II

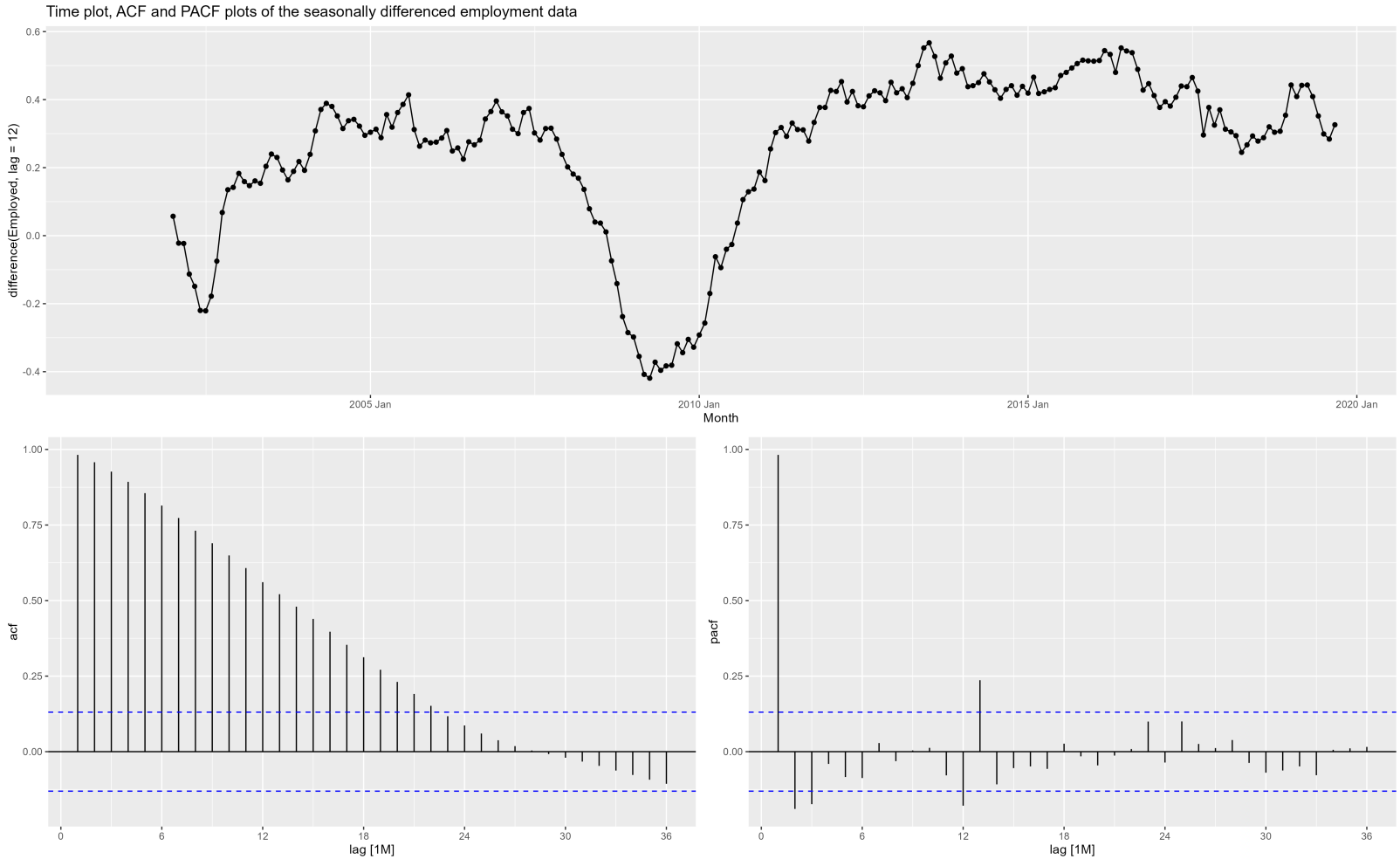

US employment: leisure and hospitality
Employed
16
14
12
2005 Jan
2010 Jan
2015 Jan
2020 Jan
Month [1M]

Example III

- Clearly, non-stationary seasonal data with an increasing trend. Let's do a seasonal differencing.

```
leisure |>
  gg_tsdisplay(difference(Employed, lag = 12), plot_type = "partial", lag = 36)
    +
  labs(title = "Time plot, ACF and PACF plots of the seasonally differenced
    employment data")
```

Example IV

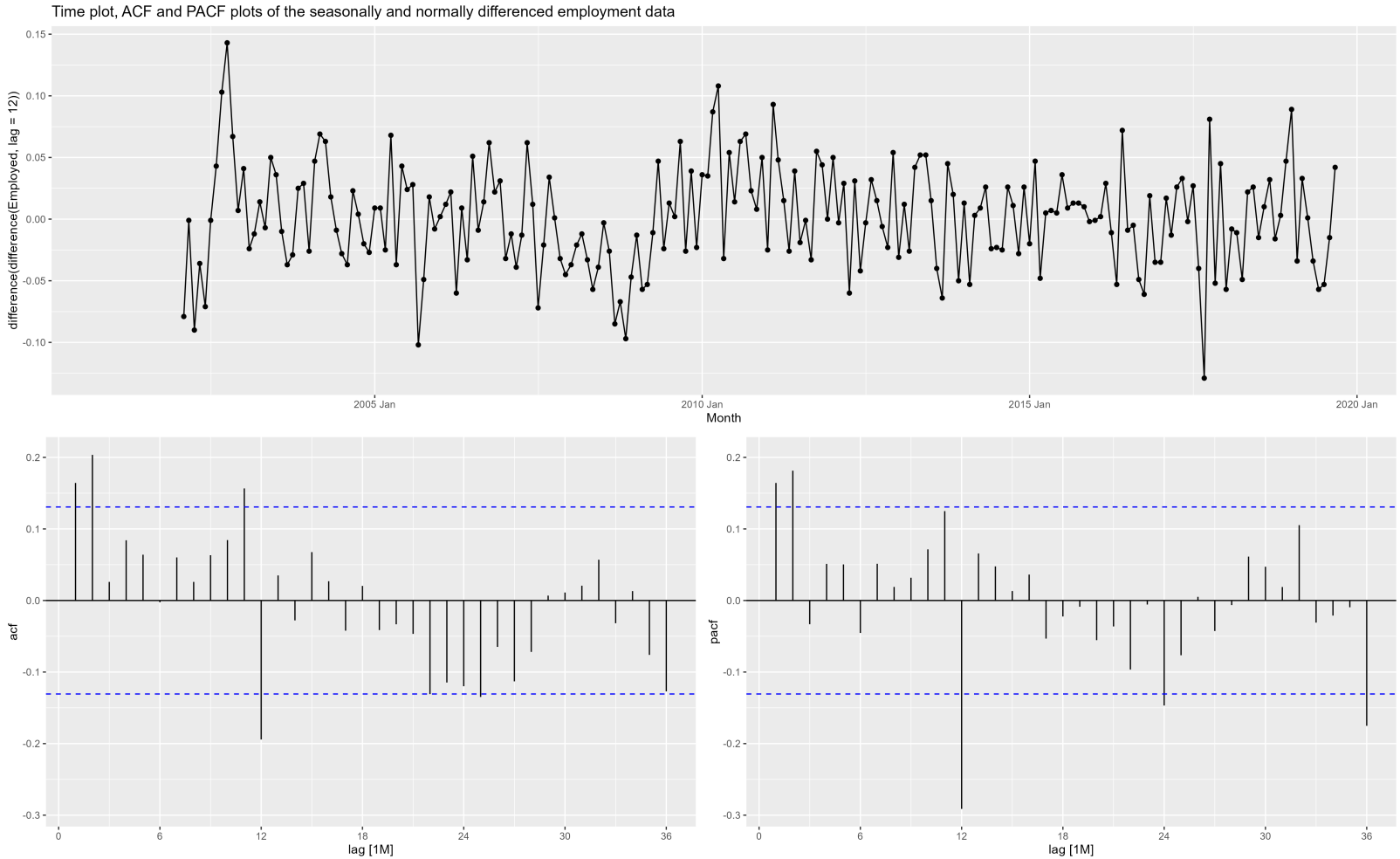

Time plot, ACF and PACF plots of the seasonally differenced employment data
difference(Employed, lag = 12)
0.6
0.4
0.2
0.0
-0.2
-0.4
2005 Jan
2010 Jan
2015 Jan
2020 Jan
Month
acf
1.00
0.75
0.50
0.25
0.00
0
6
12
18
24
30
36
lag [1M]
pacf
1.00
0.75
0.50
0.25
0.00
0
6
12
18
24
30
36
lag [1M]

Example V

- Still not stationary. Let's do a regular differencing as well.

```
leisure |>
  gg_tsdisplay(difference(difference(Employed, lag = 12)), plot_type = "partial"
      , lag = 36) +
  labs(title = "Time plot, ACF and PACF plots of the seasonally and normally
      differenced employment data")
```

Example VI


Time plot, ACF and PACF plots of the seasonally and normally differenced employment data
difference(difference(Employed, lag = 12))
0.15
0.10
0.05
0.00
-0.05
-0.10
2005 Jan
2010 Jan
2015 Jan
2020 Jan
Month
acf
0.2
0.1
0.0
-0.1
-0.2
-0.3
0
6
12
18
24
30
36
lag [1M]
pacf
0.2
0.1
0.0
-0.1
-0.2
-0.3
0
6
12
18
24
30
36
lag [1M]

Example VII

- Now, this looks like stationary data.
- **Obs.** We could have determined the number of differencings needed, by using statistical tests as well:

```
leisure |> features(Employed, c(unitroot_ndiffs, unitroot_nsdiffs))
```

```
  ndiffs nsdiffs
  <int>   <int>
1     1       1
```

Example VIII

- Now, let's look at the ACF and PACF plots:
  - On the ACF plot we have a significant spike at lag 2, suggesting a non-seasonal MA(2) component.
  - The significant spike at lag 12 on the ACF plot suggests a seasonal MA(1) component.
  - So one model to try out will be $ARIMA(0,1,2)(0,1,1)_{12}$
  - If we look at the PACF plots, we have a significant spike at lag 2, suggesting a non-seasonal AR(2) term, so another model that we can try out is $ARIMA(2,1,0)(0,1,1)_{12}$.
  - We will also add the model where the parameters are found automatically.

```
leisure_models <- leisure |>
  model(arima012011 = ARIMA(Employed ~ pdq(0, 1, 2) + PDQ(0, 1,1)),
        arima210011 = ARIMA(Employed ~ pdq(2, 1, 0) + PDQ(0, 1, 1)),
        auto = ARIMA(Employed, stepwise = FALSE, approx = FALSE))

leisure_models
```

Example IX

```
arima012011                arima210011                    auto
<model>                        <model>                  <model>
1 <ARIMA(0,1,2)(0,1,1)[12]> <ARIMA(2,1,0)(0,1,1)[12]> <ARIMA(2,1,0)(1,1,1)[12]>
```

```
leisure_models |>
  glance()
```

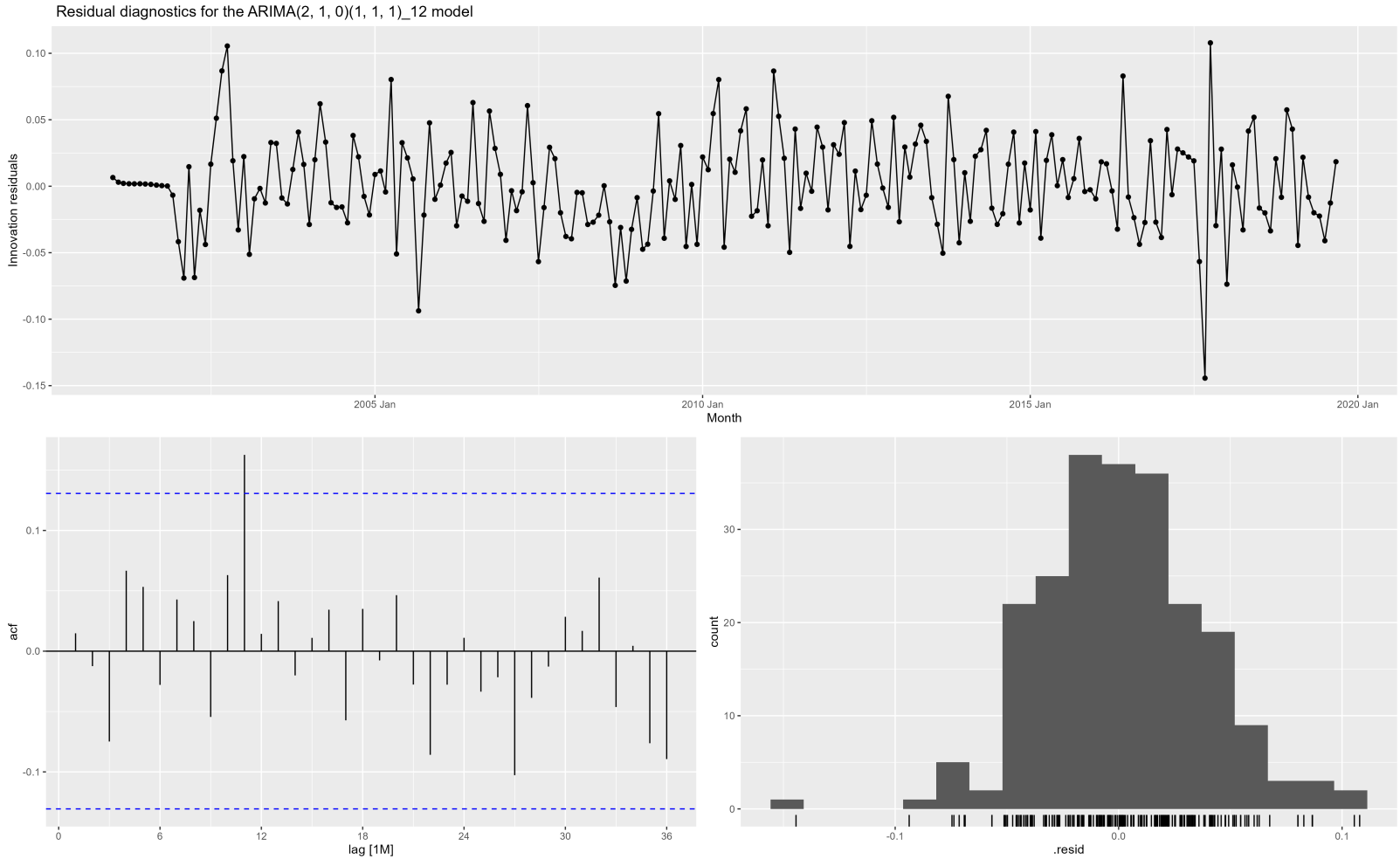
```
.model       sigma2 log_lik   AIC  AICc   BIC ar_roots   ma_roots
  <chr>       <dbl>   <dbl> <dbl> <dbl> <dbl> <list>     <list>
1 arima012011 0.00146    391. -775. -775. -761. <cpl [0]>  <cpl [14]>
2 arima210011 0.00145    392. -776. -776. -763. <cpl [2]>  <cpl [12]>
3 auto        0.00142    395. -780. -780. -763. <cpl [14]> <cpl [12]>
```

- We can see that the automatically found parameters are $(2,1,0)(1,1,1)$, very close to our second guess.
- Looking at the AICc values, we can see that the best is the automatically found model. Let's look at its residuals.

Example X

```
leisure_models |>
  select(auto) |>
  gg_tsresiduals(lag = 36) +
  labs(title = "Residual diagnostics for the ARIMA(2, 1, 0)(1, 1, 1)_12 model")
```

Example XI


Residual diagnostics for the ARIMA(2, 1, 0)(1, 1, 1)_12 model
Innovation residuals
0.10
0.05
0.00
-0.05
-0.10
-0.15
2005 Jan
2010 Jan
2015 Jan
2020 Jan
Month
acf
0.1
0.0
-0.1
0
6
12
18
24
30
36
lag [1M]
count
30
20
10
0
-0.1
0.0
0.1
.resid

Example XII

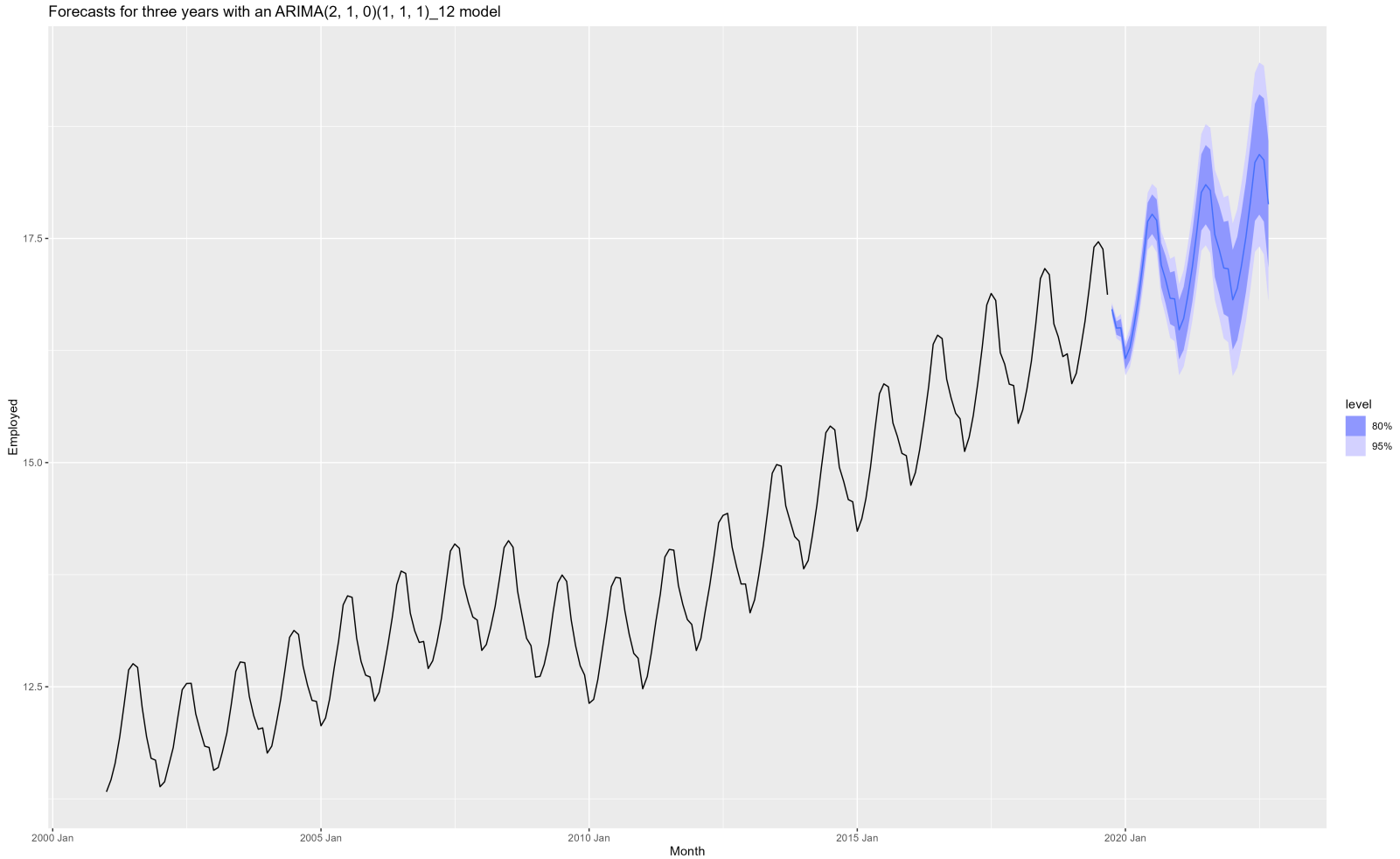
```
leisure_models |>
  select(auto) |>
  augment() |>
  features(.innov, ljung_box, dof = 4, lag = 24) #white noise
```

```
.model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 auto      16.6     0.680
```

- Since the Ljung-Box tests shows that the residuals are white noise, we can select this model as the best one. Let's forecast (and plot) the values for the next 3 years.

```
leisure_models |>
  select(auto) |>
  forecast(h = 36) |>
  autoplot(leisure) +
  labs(title = "Forecasts for three years with an ARIMA(2, 1, 0)(1, 1, 1)_12
      model")
```

Example XIII



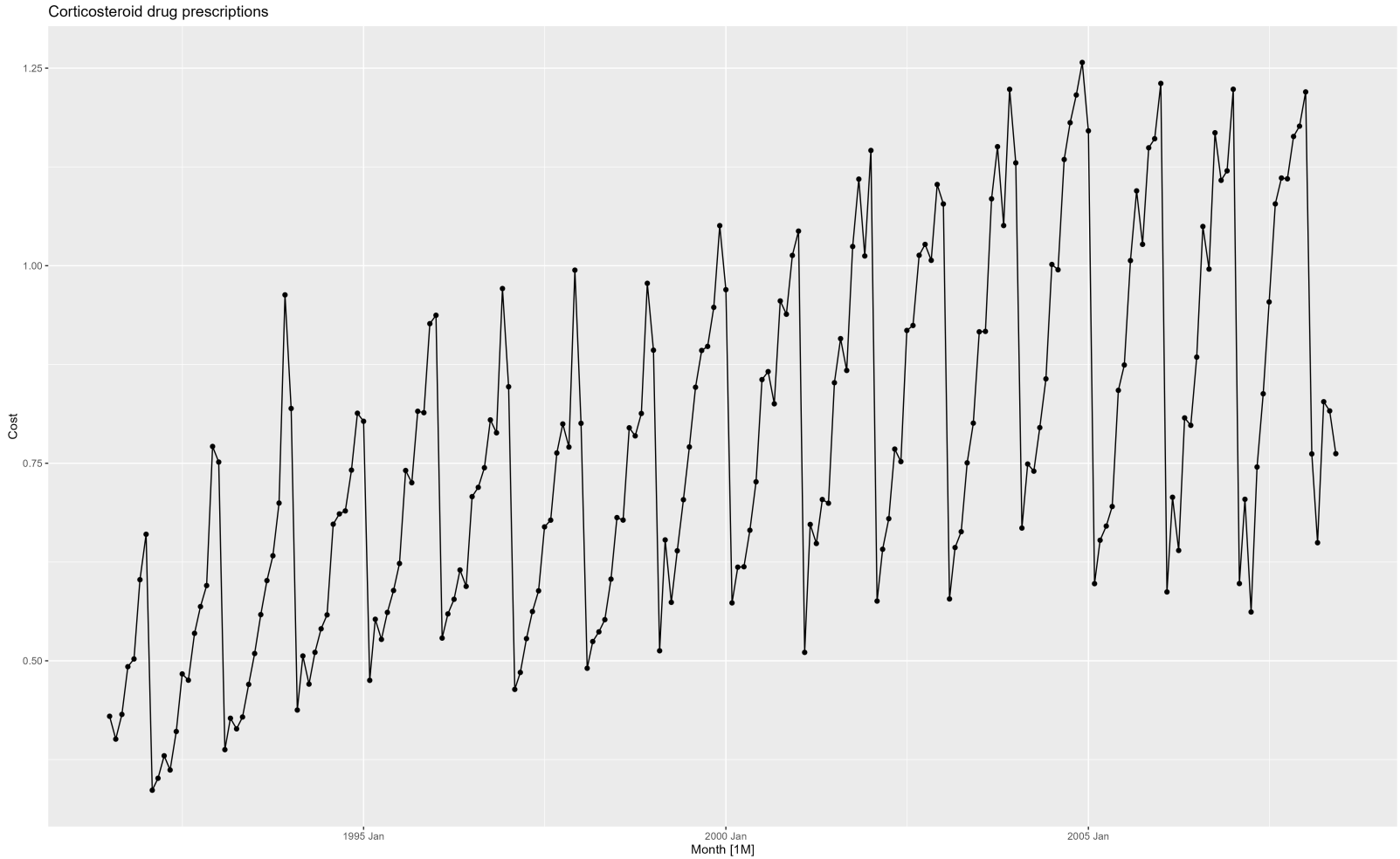
Example 2 I

- For the second example, let's consider a part of the PBS data set (monthly Medicare prescription data from Australia), the one for the corticosteroid drug sales.

```
corticosteroid <- PBS |>
  filter(ATC2 == "H02") |>
  summarise(Cost = sum(Cost)/1e6) # to have smaller values

corticosteroid |> autoplot() +
    geom_point() +
    labs(title = "Corticosteroid drug prescriptions")
```

Example 2 II

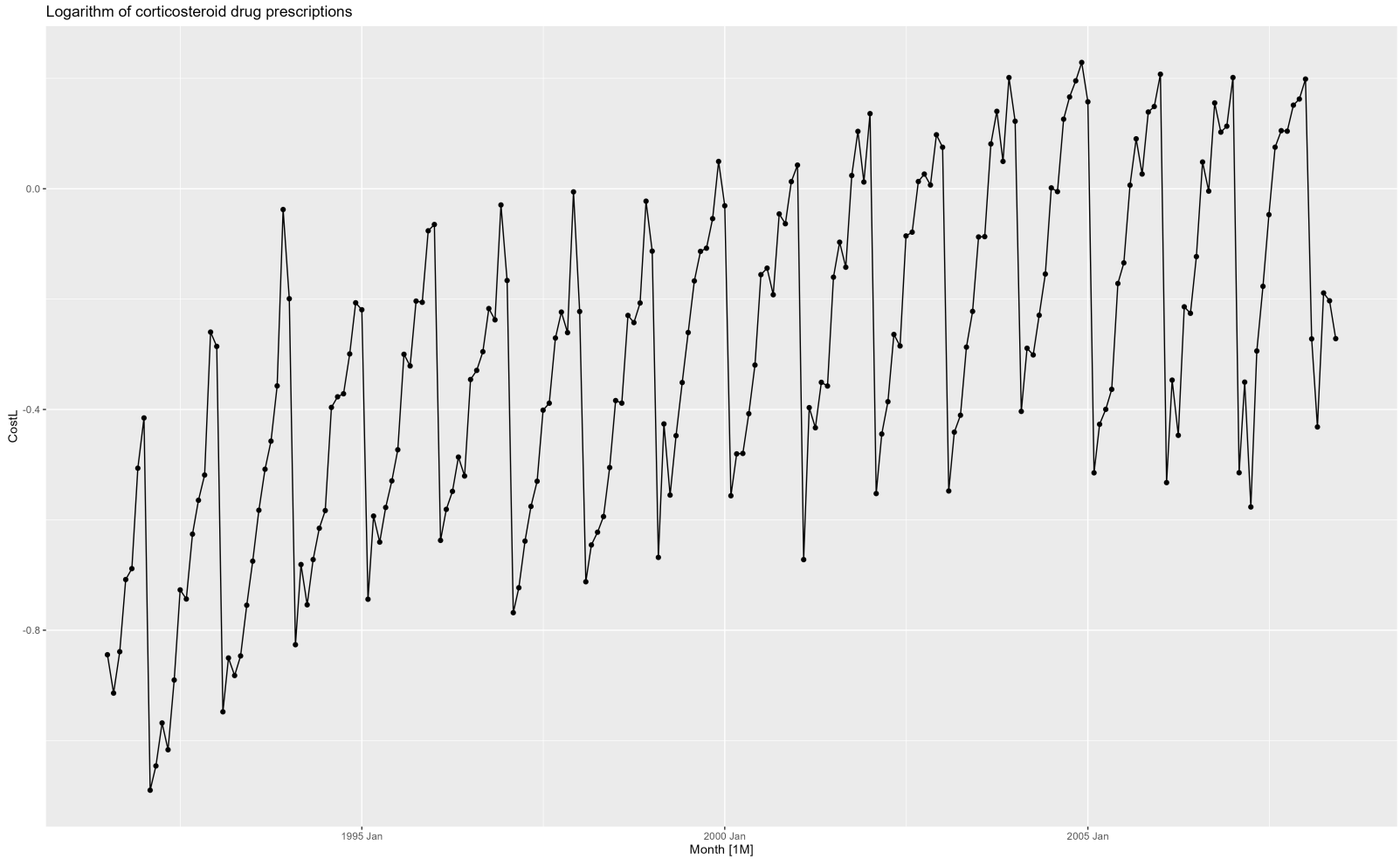

Corticosteroid drug prescriptions
Cost
1.25
1.00
0.75
0.50
1995 Jan
2000 Jan
2005 Jan
Month [1M]

Example 2 III

- It looks like the variance of the time series increases with time, so we probably need a transformation. Logarithm seems suitable.

```
corticosteroid <- corticosteroid |>
  mutate(CostL = log(Cost))
corticosteroid |> autoplot(CostL) +
  geom_point() +
  labs(title = "Logarithm of corticosteroid drug prescriptions")
```

Example 2 IV


Logarithm of corticosteroid drug prescriptions
CostL
0.0
-0.4
-0.8
1995 Jan
2000 Jan
2005 Jan
Month [1M]

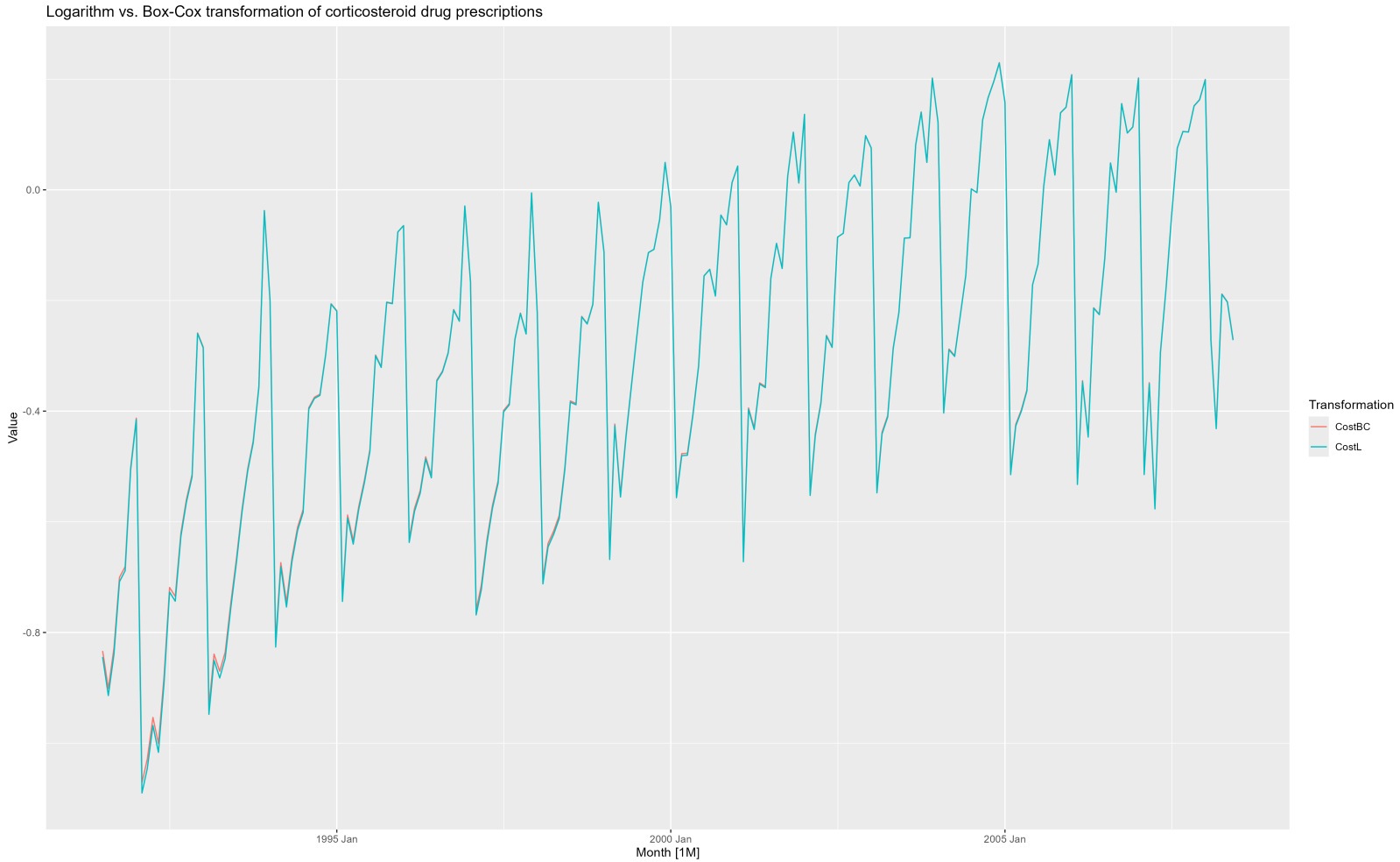
Example 2 V

- Looks good, it seems to have stable variance.
- **Obs.** If the log transformation does not seem to work, we could do a Box-Cox transformation. This has a parameter λ which determines how to change the data. As discussed in a previous lecture, we can find the optimal value of λ with the *guerrero* feature.

```
lambda <- corticosteroid |>
    features(Cost, guerrero)
lambda #will print 0.0315
corticosteroid <- corticosteroid |>
    mutate(CostBC = box_cox(Cost, lambda))

corticosteroid |>
    select(-Cost) |>
    pivot_longer(cols = c(CostL, CostBC), names_to= "Transformation", values_to=
      "Value") |>
    autoplot() +
    labs(title = "Logarithm vs. Box-Cox transformation of corticosteroid drug
     prescriptions")
```

Example 2 VI


Logarithm vs. Box-Cox transformation of corticosteroid drug prescriptions
Value
0.0
-0.4
-0.8
1995 Jan
2000 Jan
2005 Jan
Month [1M]
Transformation
CostBC
CostL

Example 2 VII

- We can see that the two plots are almost identical, so the log transformation is OK.
- Data is definitely not stationary and highly seasonal. We should start with a seasonal differencing.

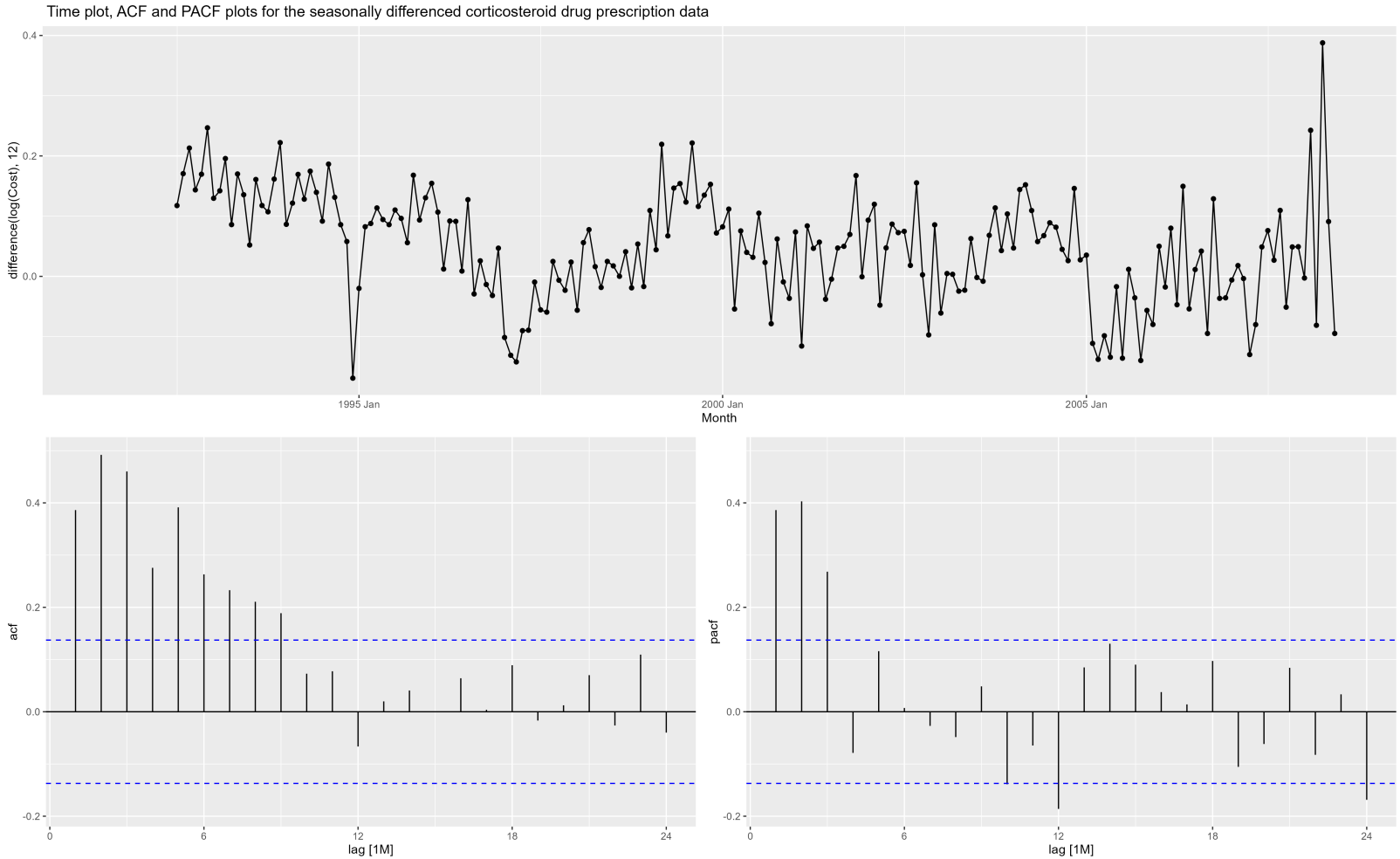
```
corticosteroid |> features(log(Cost), c(unitroot_ndiffs, unitroot_nsdiffs))
```

```
 ndiffs nsdiffs
   <int>   <int>
1      1       1
```

- Let's see if a seasonal difference is enough

```
corticosteroid |>
  gg_tsdisplay(difference(log(Cost), 12), plot_type = "partial", lag_max = 24) +
  labs(title = "Time plot, ACF and PACF plots for the seasonally differenced
     corticosteroid drug prescription data")
```

Example 2 VIII

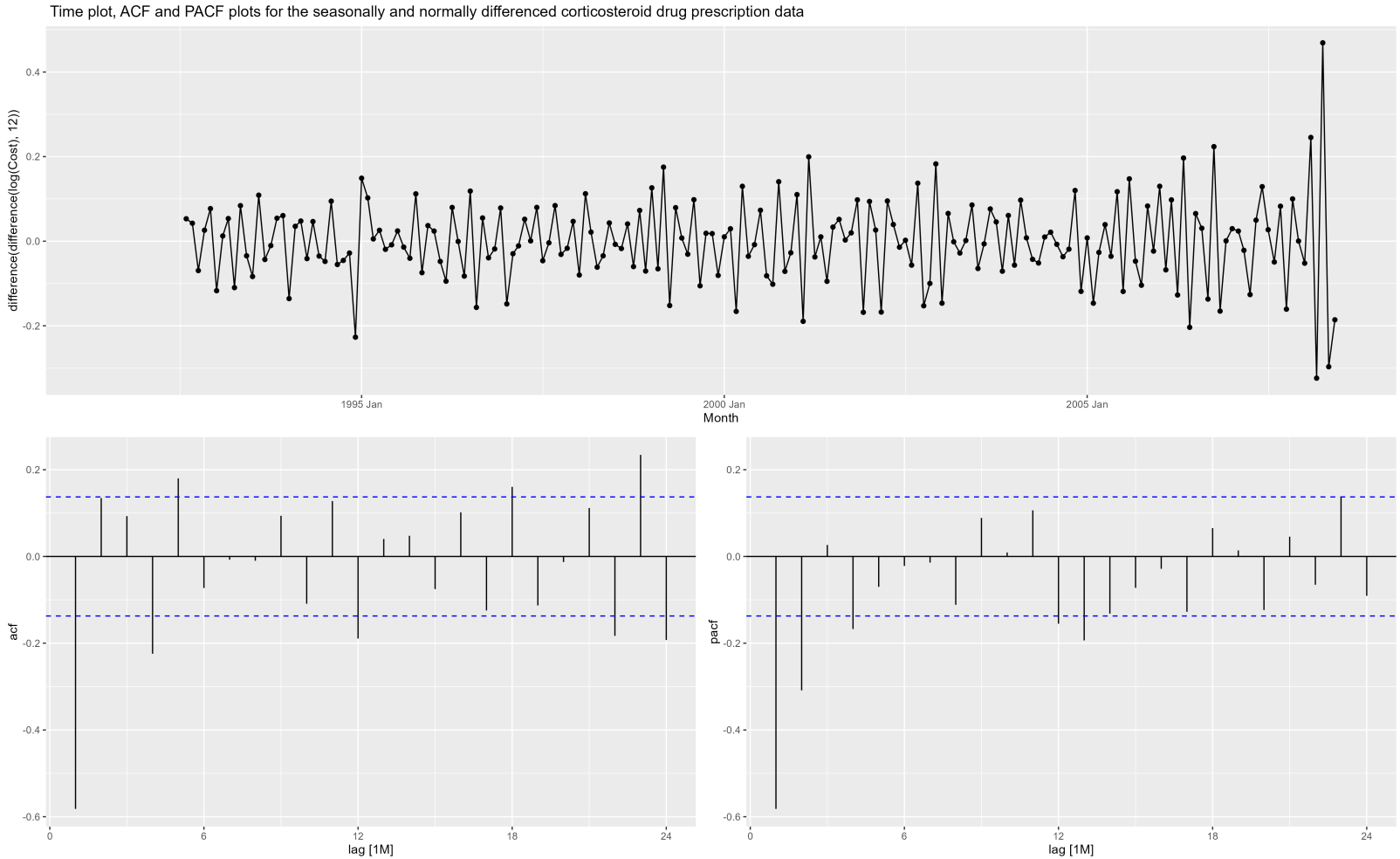


Example 2 IX

- It is not that obvious whether the seasonal differencing is enough.

```
corticosteroid |> gg_tsdisplay(difference(difference(log(Cost), 12)), plot_type
    = "partial", lag_max = 36) +
  labs(title = "Time plot, ACF and PACF plots for the seasonally and non-
    seasonally differenced drug prescription data")
```

Example 2 X


Time plot, ACF and PACF plots for the seasonally and normally differenced corticosteroid drug prescription data
difference(difference(log(Cost), 12))
0.4
0.2
0.0
-0.2
1995 Jan
2000 Jan
2005 Jan
Month
acf
pacf
0.2
0.0
-0.2
-0.4
-0.6
0
6
12
18
24
lag [1M]

Example 2 XI

- The PACF plot does not really have significant spikes at seasonal lags (well, maybe a tiny one at lag 12), but the ACF plot has significant spikes at lag 12 and 24, suggesting an MA(2) model for the seasonal component.
- Looking at non-seasonal lags, the PACF plot has two significant spikes, suggesting an AR(2) model for the non-seasonal component.
- So, one possible model would be $ARIMA(2,1,0)(0,1,2)_{12}$. Let's fit this model, together with a few possible variations.

Example 2 XII

```
corticosteroid |> model(
  arima210012 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(0, 1, 2)),
  arima211012 = ARIMA(log(Cost) ~ pdq(2, 1, 1) + PDQ(0, 1, 2)),
  arima310012 = ARIMA(log(Cost) ~ pdq(3, 1, 0) + PDQ(0, 1, 2)),
  arima210011 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(0, 1, 1)),
  arima210111 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(1, 1, 1)),
  arima311012 = ARIMA(log(Cost) ~ pdq(3, 1, 1) + PDQ(0, 1, 2)),
  arima210210 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(2, 1, 0))
) -> corticosteroid_models

corticosteroid_models |> glance() |> arrange(AICc)
```

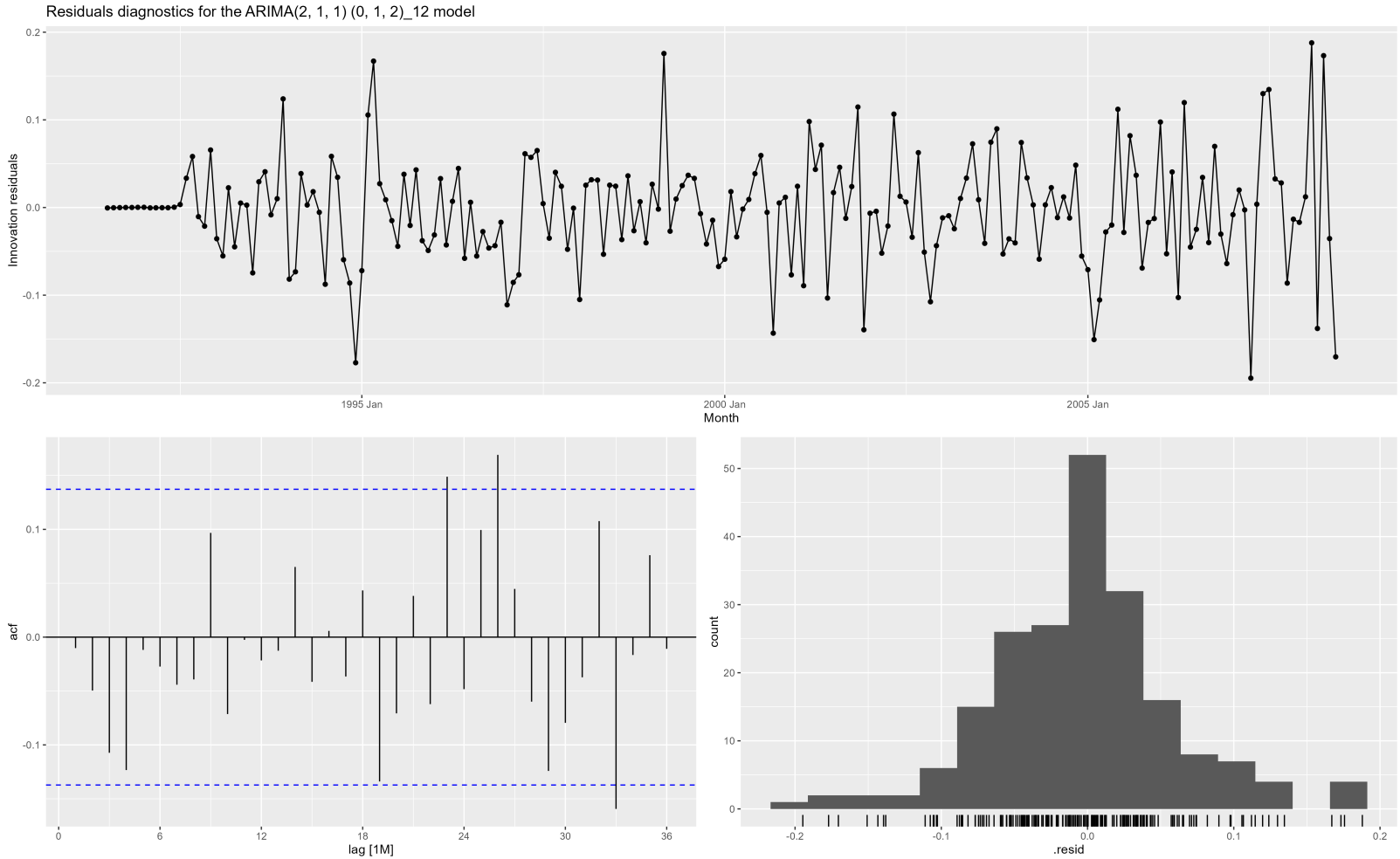
	.model	sigma2	log_lik	AIC	AICc	BIC	ar_roots	ma_roots
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<list>	<list>
1	arima211012	0.00428	248.	-485.	-484.	-465.	<cpl [2]>	<cpl [25]>
2	arima210012	0.00433	247.	-484.	-483.	-467.	<cpl [2]>	<cpl [24]>
3	arima310012	0.00431	248.	-483.	-483.	-464.	<cpl [3]>	<cpl [24]>
4	arima210111	0.00435	246.	-483.	-483.	-467.	<cpl [14]>	<cpl [12]>
5	arima210011	0.00439	245.	-483.	-483.	-470.	<cpl [2]>	<cpl [12]>
6	arima311012	0.00430	248.	-483.	-482.	-460.	<cpl [3]>	<cpl [25]>
7	arima210210	0.00465	242.	-473.	-473.	-457.	<cpl [26]>	<cpl [0]>

Example 2 XIII

- According to AICc, the best model is $ARIMA(2,1,1)(0,1,2)_{12}$. Let's look at its residuals.

```
corticosteroid_models |>
  select(arima211012) |>
  gg_tsresiduals(lag_max = 36) +
   labs(title = "Residuals diagnostics for the ARIMA(2, 1, 1) (0, 1, 2)_12 model
     ")
```

Example 2 XIV



Example 2 XV

```
corticosteroid_models |>
  select(arima211012) |>
  augment() |>
  features(.innov, ljung_box, lag = 36, dof = 5) #fails, it is not white noise.
```

```
  .model      lb_stat lb_pvalue
  <chr>         <dbl>     <dbl>
1 arima211012    51.1    0.0130
```

- The Ljung-Box test tells us, that the residuals are not white noise.
- Running automatic ARIMA disabling the approximations and stepwise search, we get that the best model is an $ARIMA(2,1,3)(0,1,1)_{12}$. However, this fails the Ljung-Box test as well.

Example 2 XVI

```
corticosteroid |>
   model(ARIMA(log(Cost), stepwise = FALSE, approx = FALSE)) -> corticosteroid_
     best_model
corticosteroid_best_model
```

```
  `ARIMA(log(Cost), stepwise = FALSE, approx = FALSE)`
                                                <model>
1                           <ARIMA(2,1,3)(0,1,1)[12]>
```

```
corticosteroid_best_model |>
  augment() |>
  features(.innov, ljung_box, lag = 36, dof = 6)
```

```
  .model                                                lb_stat lb_pvalue
  <chr>                                                   <dbl>     <dbl>
1 ARIMA(log(Cost), stepwise = FALSE, approx = FALSE)       46.1    0.0301
```

Example 2 XVII

- Using the AICc measure we can only compare models, where the order of the differencing is the same. If we want to test models with different orders of differencing, we need other methods, for example, train-test validation.
- The *corticosteroid* time series contains 204 observations, let's use the last two years as test and the remaining instances as train.

Example 2 XVIII

```
corticosteroid_train <- corticosteroid |> head(n = 180)
corticosteroid_test <- corticosteroid |> tail(n = 24)

corticosteroid_train |> model(
  arima210012 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(0, 1, 2)),
  arima211012 = ARIMA(log(Cost) ~ pdq(2, 1, 1) + PDQ(0, 1, 2)),
  arima310012 = ARIMA(log(Cost) ~ pdq(3, 1, 0) + PDQ(0, 1, 2)),
  arima210011 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(0, 1, 1)),
  arima210111 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(1, 1, 1)),
  arima311012 = ARIMA(log(Cost) ~ pdq(3, 1, 1) + PDQ(0, 1, 2)),
  arima210210 = ARIMA(log(Cost) ~ pdq(2, 1, 0) + PDQ(2, 1, 0)),
  arima301111 = ARIMA(log(Cost) ~ pdq(3, 0, 1) + PDQ(1, 1, 1)),
  arima213011 = ARIMA(log(Cost) ~ pdq(2, 1, 3) + PDQ(0, 1, 1)),
  arima301012 = ARIMA(log(Cost) ~ pdq(3, 0, 1) + PDQ(0, 1, 2))
) -> corticosteroid_train_models

corticosteroid_forecasts <- corticosteroid_train_models |>
  forecast(h = 24)

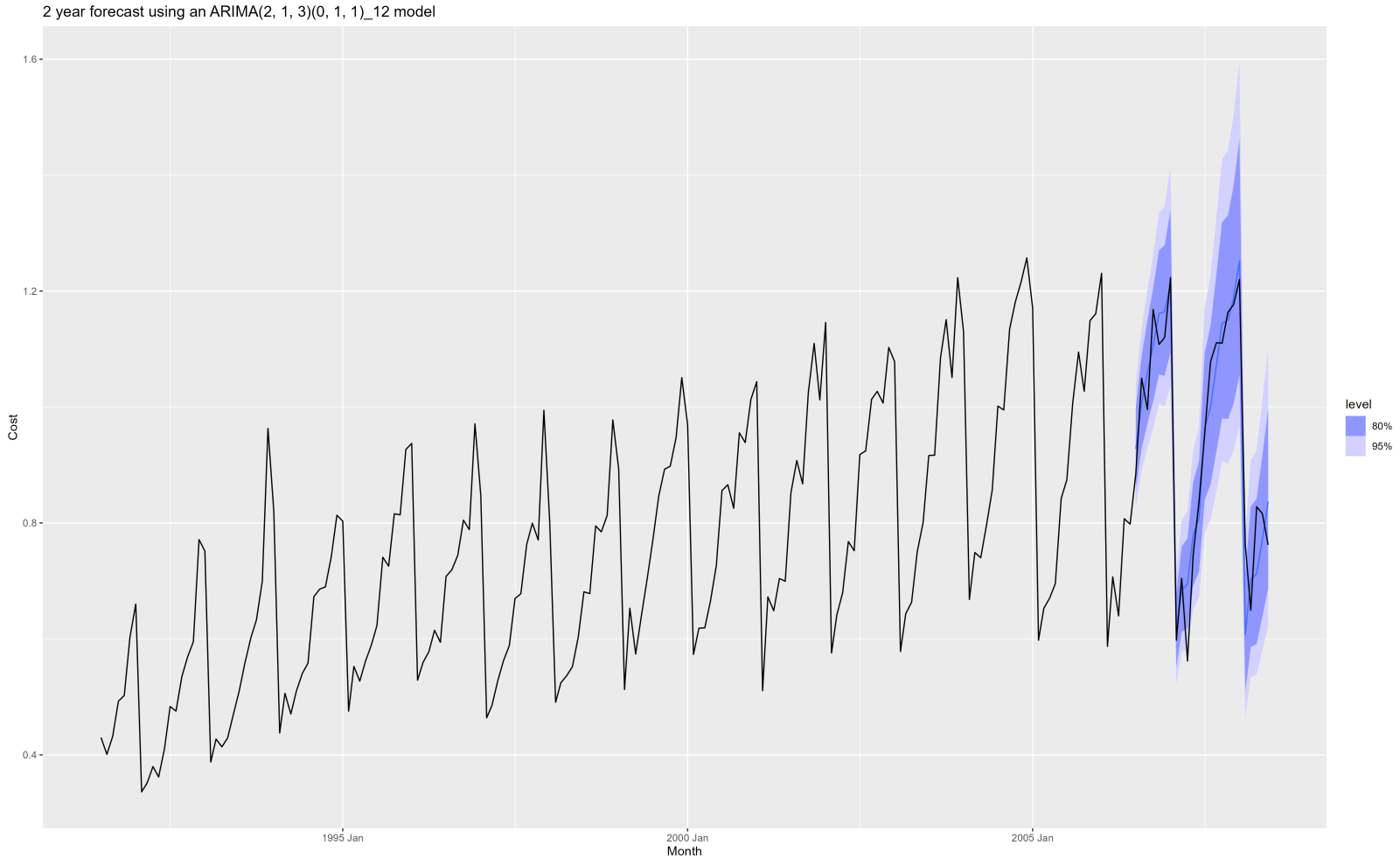
accuracy(corticosteroid_forecasts, corticosteroid_test) |> arrange(RMSE)
```

Example 2 XIX

	.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	arima213011	Test	0.000308	0.0628	0.0505	-0.350	6.04	NaN	NaN	-0.282
2	arima210011	Test	0.000462	0.0630	0.0479	-0.377	5.68	NaN	NaN	-0.253
3	arima210111	Test	-0.0127	0.0636	0.0499	-1.89	5.97	NaN	NaN	-0.281
4	arima211012	Test	-0.0236	0.0663	0.0540	-3.14	6.48	NaN	NaN	-0.247
5	arima310012	Test	-0.0242	0.0667	0.0545	-3.20	6.53	NaN	NaN	-0.252
6	arima210012	Test	-0.0225	0.0667	0.0542	-3.02	6.49	NaN	NaN	-0.267
7	arima311012	Test	-0.0261	0.0684	0.0567	-3.40	6.76	NaN	NaN	-0.272
8	arima210210	Test	-0.0221	0.0687	0.0566	-2.80	6.70	NaN	NaN	-0.223
9	arima301111	Test	-0.0680	0.0960	0.0829	-7.87	9.53	NaN	NaN	-0.187
10	arima301012	Test	-0.0761	0.103	0.0888	-8.83	10.2	NaN	NaN	-0.128

```
corticosteroid_forecasts |>
  filter(.model == "arima213011") |>
  autoplot(corticosteroid) +
  labs(title = "Figure 67: 2 year forecast using an ARIMA(2, 1, 3)(0, 1, 1)_12
      model")
```

Example 2 XX


2 year forecast using an ARIMA(2, 1, 3)(0, 1, 1)_12 model
Cost
1.6
1.2
0.8
0.4
1995 Jan
2000 Jan
2005 Jan
Month
level
80%
95%